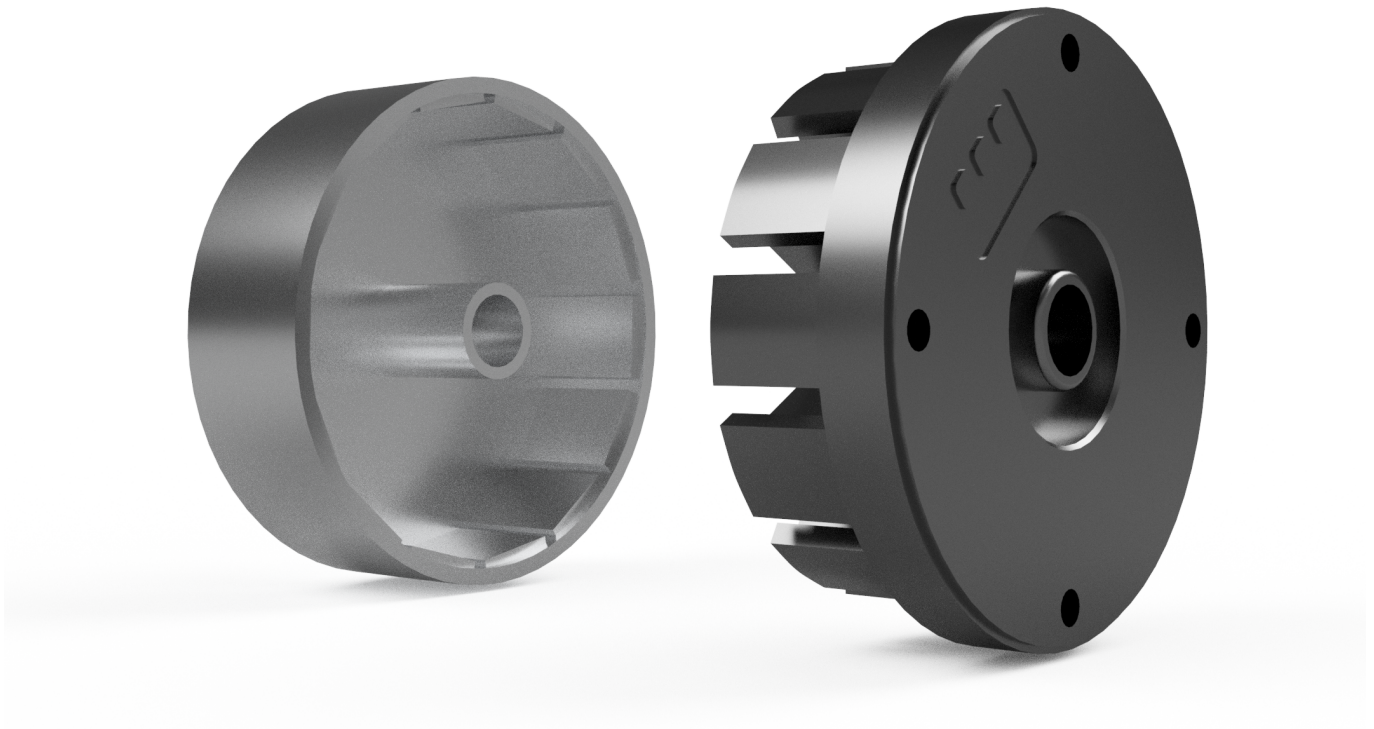


8TH SEMESTER MASTER

LOW-SPEED FIELD-ORIENTED CONTROL OF DIRECT-DRIVE PMSM USING SVPWM

Frode Maaløe-Jørgensen, Kasper Juhl Mortensen, Konstantinos Spiliotopoulos, Terkel Føns Dyring, Tobias
Pinstrup Knudsen, Zacharias Møller Jakobsen



Electro-Mechanical System Design, EMSD2-4.115
Materials and Production, Aalborg University

23rd of May 2024

**Materials and Production**

Electro-Mechanical System Design

Fibigerstræde 16

9220 Aalborg Øst

Title:

Low-Speed Field-Oriented Control of
Direct-Drive PMSM using SVPWM

Project:

8th Semester Master

Project period:

1st of February to 23rd of May 2024

Project group:

EMSD2-4.115

Email:

mp-24-emsd-2-4115@student.aau.dk

Supervisor:

Peter Omand Rasmussen

Pages:

60

Appendices:

10

Participants:

Frode Maaløe-Jørgensen

Kasper Juhl Mortensen

Konstantinos Spiliotopoulos

Terkel Føns Dyring

Tobias Pinstруп Knudsen

Zacharias Møller Jakobsen

Preface

This project is produced by 8th Semester Master, students of the group EMSD2-4.115, studying Electro-Mechanical System Design, at Aalborg University. It is written in the period 1st of February to 23rd of May 2024 under the supervision of Peter Omand Rasmussen. The semester's general theme is *System Design* and the project chosen is *Low-Speed Field-Oriented Control of Direct-Drive PMSM using SVPWM*.

Different descriptions for distinguishing between PMSM and BLDC motors exist, where the definition herein is based on whether the motor is controlled with trapezoidal or sinusoidal voltages. In the beginning chapters, BLDC and PMSM are grouped in a category called AC Synchronous Motors. Since the motor is controlled with FOC and SVPWM, the motor used in this project is called a PMSM.

During the making of this project, the following programs have been used:

Overleaf L ^A T _E X	For text processing and layout of this report.
MathWorks MATLAB and Simulink	For calculations, modeling, simulation, and data processing.
Adobe Illustrator and Inkscape	For diagrams and illustrations.
SolidWorks	For schematics of the Lab Setup.
CubeIDE, CubeMX, and CubeMonitor	For developing, debugging, and reading data.

Frode Maaløe-Jørgensen

Kasper Juhl Mortensen

Konstantinos Spiliotopoulos

Terkel Føns Dyring

Tobias Pinstrup Knudsen

Zacharias Møller Jakobsen

Aalborg University, 23rd of May 2024

Abstract

This project investigates the performance of a Permanent Magnet Synchronous Motor (PMSM) compared to the LEGO Powered Up Technic Large DC Motor based on an interest expressed by LEGO. The interest stems from the potential of the PMSM having a higher efficiency while keeping the same form factor, and being able to omit a gearbox, thereby reducing manufacturing steps. The CubeMars GL35 is selected for its low KV-value and 7 pole pairs, offering a suitable performance with high torque at low RPM. The motor is controlled with an STM32F411RE development board, X-NUCLEO-IHM07M1 inverter board, and an added incremental encoder, allowing for commutating with Space Vector Pulse Width Modulation (SVPWM) combined with Field-Oriented Control (FOC) executed through C code written to the development board. A 2-axis hall sensor is tested, however, given the setup, implementation is unsuccessful, leading to only the encoder being used.

An electrical and mechanical model is established for designing and tuning controllers for the motor. The electrical model includes the three stator phases, back EMF, and conversion into the d - q coordinate system. The mechanical model consists of the PMSM including its friction and inertia. Additionally, an inverter model for controlling the PWM is set up as an ideal inverter, meaning it operates without loss. Both of the models are validated after an initial tuning by comparing the models with test data from an open-loop test without any added inertia, and a closed-loop test with an added inertia disk to simulate a startup torque.

Based on the models, control loops, with FOC and SVPWM, are developed and implemented, tuned for high inertia to ensure a proper response for the wide range of applications in various LEGO sets. The first test is an added inertia disk, to investigate the transient response and startup torque of the PMSM motor.

Tests with a mass on a string are conducted to evaluate performance under different load torques and the energy usage hereof. Here it is concluded that the PMSM motor has a higher torque and higher efficiency.

The PMSM is compared to the LEGO DC motor in no-load testing. The transient response of the PMSM is faster and has a lower steady-state error due to high friction in the DC motor. Furthermore, the PMSM exhibits a smoother performance at 1 rad/s rotational speed due to lower friction, displaying the brushless and gearless advantages.

The results indicate that the PMSM, controlled via FOC and SVPWM, offers potential improvements in efficiency and direct-drive capabilities for LEGO applications compared to the original LEGO Powered Up Technic Large DC Motor.

Nomenclature

Symbol	Definition
B	Viscous friction
C	Capacitor <i>or</i> Controller
G	Transfer function
f	Frequency
J	Moment of Inertia
K	Constant
L	Inductance
M	Mass
i	Current
P	Plant <i>or</i> Proportional
R	Resistance
t	Time
x	State vector
y	Output vector
V	Voltage
u	Input vector
α	Discretization coefficient
e	Euler's Number
ω	Angular Speed
λ	Flux linkage
τ	Torque
ϕ_f	Permanent magnetic flux
$\mathcal{L}$	Laplace

Subscript

Subscript	Definition
cut	Cut-off point
n	Iteration number
O	Output
vd	Voltage divider
s	Sector
ls	Leakage
m	Magnetization
e	Electrical
r	Rotor
ds	d stator
ref	Reference
ij	Reference
r	Indices for matrix

Abbreviations

Abbreviation	Definition
AAU	Aalborg University
BD	Backward Difference
PP	Pole Pair
RPM	Rotations pr. minute
DC	Direct Current
PMSM	Permant Magnet Synchronous Motor
BLDCM	Brushless Direct Current Motor
FOC	Field Oriented Control
PWM	Pulse Width Modulation
HB	Half-Bridge
SVPWM	Space Vector Pulse Width Modulation
SPWM	Sinusoidal Pulse Width Modulation
ADC	Analog-to-Digital Converter
GPIO	General-purpose Input/Output
CCRX	Counter-compare register
UART	Universal Asynchronous Receiver/- Transmitter
ARR	Auto-reload Register
MOI	Moment of Inertia
RGA	Relative Gain Array
BCT	Bias Current Trimming
SPI	Serial Peripheral Interface
PR	Period
LP	Low-Pass
SISO	Single-Input Single-Output
MIMO	Multiple-Input Multiple-Output

Contents

1	Introduction	1
2	Problem analysis	2
2.1	Requirements	2
2.2	Motor parameters effect	3
2.2.1	KV-value	3
2.2.2	Number of pole pairs	3
2.3	Motor choice	3
2.4	Control of AC Synchronous motors	4
2.4.1	Commutation techniques	4
2.4.2	Sensor-based solutions	5
2.4.3	Sensorless solution	6
2.5	Deciding on commutation techniques and sensor type	6
2.5.1	Current Measurement	7
2.6	Control Considerations for LEGO Sets	7
3	Problem Statement	8
4	System Setup	9
4.1	Choice of equipment	10
4.2	General system overview	11
4.3	Inverter	11
4.4	Software implementation	13
4.4.1	PWM timer	14
4.4.2	PWM and control loop	14
4.4.3	Position measurement	15
4.4.4	Speed Calculation and Filtering	16
4.4.5	Current measurement	18
4.4.6	Optimization of C-code	20
5	Modeling the motor	22
5.1	PMSM model	22
5.1.1	Electrical model	22
5.1.2	Mechanical model	24
5.2	Inverter models	25
5.3	Model validation	25
5.3.1	Electrical model test	25
5.3.2	Current open-loop test	26

5.3.3	Speed open-loop current closed-loop test	27
6	Control Architecture	29
6.1	Field-Oriented Control	29
6.2	Space Vector PWM	30
7	Controller Design	33
7.1	S-plane model	33
7.1.1	Multiple-Input Multiple-Output	33
7.1.2	Linear model validation	34
7.1.3	Single-Input Single-Output System	36
7.2	Controller design	37
7.2.1	Controller requirements	38
7.2.2	Loop shaping	38
7.2.3	Decoupling	45
7.2.4	Discretization and anti-windup	46
8	Performance testing	49
8.1	Mass on string testing	50
8.1.1	Speed Ramp Test	50
8.1.2	Position Step Response	50
8.2	Inertia disk testing	51
8.2.1	Speed Step Response	51
8.2.2	Low-speed control	52
8.3	LEGO Motor Comparison	53
8.3.1	Transient Comparison	53
8.3.2	Low-Speed Control Comparison	53
9	Discussion	55
9.1	Filter effect on system	55
9.2	Further tuning of controllers	56
9.3	Filtering of current measurements	56
9.4	Z-channel saved to flash memory	56
9.5	Hall Sensor	56
9.6	LEGO Use Cases	57
10	Conclusion	58
	Bibliography	60
A	KV-Value	I
B	Resolver	II
C	Clark and Park transformation	III

D	Gear vs direct drive	IV
E	Sensorless solution	VI
F	Model validation plots	VII
G	Varying control on different inertia	IX
H	C-code	X

Introduction 1

more complex control LEGO produces a variety of motorized sets in the 'Powered UP' theme, which currently are actuated by small-scale brushed DC motors. To improve LEGO's lineup, it is decided to look into replacing the DC motors with AC Synchronous motors. DC motors generally have a lower efficiency compared to AC Synchronous motors (Robotics 2023). DC motors furthermore have a shorter lifespan or require more maintenance due to the carbon brushes being worn down from the frictional contact with the commutator, additionally making the DC motor noisier than the frictionless commutation of the AC Synchronous motors (Millett 2024). The AC Synchronous motors, however, require more complex control algorithms due to the need to control an inverter to create the alternating current. LEGO is interested in testing the performance of a direct-drive solution. This can cut the assembly cost of the gearboxes used currently, remove friction in the system, thereby reducing mechanical losses and audible noise, and allow for higher rotational speeds. The focus should be on the impact of the direct-drive solution in an AC Synchronous motor.

The DC motor used for comparison is the LEGO Powered Up Technic Large Motor which can be seen with and without its outer casing in Figure 1.1. To get the required speed and torque it utilizes a two-stage planetary gearbox with a total gearing ratio of 36:1 and a 2-axis hall sensor to get position and speed feedback. This motor is used in many different LEGO sets with different control requirements, everything from low-speed position control with $\pm 90^\circ$ rotation to speed control with its maximum speed of 315 RPM (Philohome 2024).

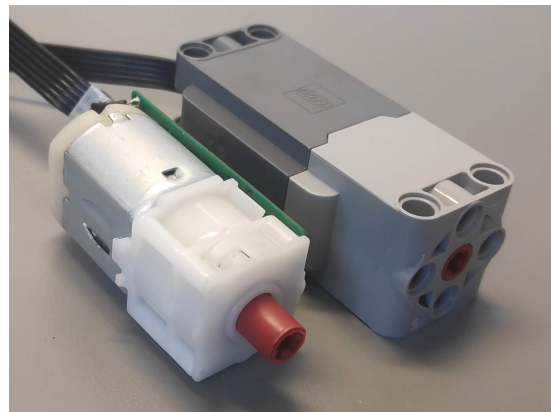


Figure 1.1. Internal and external view of the LEGO Powered Up Technic Large Motor which is to be replaced

In both cases, there can be a need for a significant startup torque to overcome friction and other external forces. For this report, the goal is to develop an AC Synchronous motor direct-drive setup, including the controls of the motor, and investigate if it is adequate as an alternative when comparing the performance to the LEGO Powered Up Technic Large Motor.

This gives the following initial problem statement:

Which suitable AC Synchronous motor and sensor solutions are there to replace the current drivetrain in the LEGO Powered Up Technic Large Motor?

Problem analysis 2

Through the problem analysis, the requirements for the system are determined and different commutation and sensor options are investigated. A general formulation for the given problem will be set up and used as a guideline throughout the report.

2.1 Requirements

The requirements are based on the LEGO Powered Up Technic Large Motor, however, given the AC Synchronous motors will be utilizing direct-drive, it does not have to be a one-to-one replacement. A similar or better performance without increasing the motor volume significantly is the goal, while keeping the lack of the gearing ratio 36:1 in mind.

To define how this is achieved, the following physical requirements are set up, based on contact with LEGO and test data from the LEGO Powered Up Technic Large Motor (Philohome 2024). Controller requirements will be presented in Section 7.2.1.

- Stall torque $\geq 160 \text{ Nmm}$: The stall torque should be similar to or higher than for the existing motor.
- Output speed $\leq 1000 \text{ RPM}$: The existing motor runs at 315 RPM, but LEGO is flexible with the rotational speed, given the removal of the gearbox and to make it easier accommodate the AC Synchronous motors. The speed should however be below 1000 RPM due to the risk associated with fast-rotating objects within the LEGO models.
- Output power $\geq 2.5 \text{ W}$: The possible output power should be greater than or equal to the maximum output power of the current motor.
- Max efficiency $> 45 \%$: The efficiency should be greater than for the existing motor.
- Voltage supply $\leq 24 \text{ V}$: To accommodate a wider voltage range, the allowed voltage supply is set higher, however, due to safety regulations for toys the supply voltage should be below 24V.
- Mass $\leq 150 \text{ g}$: The mass is not that important of a parameter. However, the following limit is set to not greatly increase the mass of the motor compared to the existing motor, while still allowing for a heavier motor.
- Volume $\leq 40,000 \text{ mm}^3$: The volume of the motor assembly should not be significantly greater than that of the existing motor. The shape of the motor can however be significantly different.

2.2 Motor parameters effect

Based on the requirements in Section 2.1 a motor has to be chosen. Besides the already listed requirements, the KV-value and number of pole pairs also have to be taken into account.

2.2.1 KV-value

The KV-value determines how fast the motor will rotate at a given supply voltage. A low KV-value for motor selection is therefore desired as the motor has to operate at a low RPM.

When doing larger-scale production this is a parameter that can be tailored to specifications if needed by changing the number of windings, giving different characteristics for a motor. A specific motor architecture can therefore be decided and designed for better performance if the selected motor is not sufficient by the end of the project. Further explained in Appendix A

2.2.2 Number of pole pairs

The number of pole pairs affects the rotational speed and the developed torque. With n pole pairs n electrical revolutions of the input voltage are required to rotate the output shaft one rotation. Furthermore, higher torque density is generally achieved with a higher number of pole pairs.

2.3 Motor choice

Based on the above requirements and considerations about motor parameters multiple motors have been found. The relevant parameters for the investigated motors can be found in Table 2.1.

Motor	KV-value [$\frac{\text{RPM}}{\text{V}}$]	Number of poles [—]	Stall torque [Nmm]	Rated speed [RPM]	Mass [g]	Volume [cm ³]	Rated voltage [V]	Max efficiency [%]
LEGO motor (Geared)	33.3	2	160	300	50	37	9	44
CubeMars GL35	100	14	460	900	90	28	16	78
FaulHaber 3216 BXT	530	14	115	6230	65	15	12	83
RS-PRO 216-3794	242	16	160	2910	80	28.3	12	-
RS-RPO 216-3787	233	8	75	2800	50	15	12	-
FaulHaber 2250 BX4	502	4	150	6000	105	19.8	12	76

Table 2.1. Comparison of potential motors. (CubeMars 2024), (FaulHaber-3216 2024), (RS-Pro-3794 2024), (RS-Pro-3787 2024), (FaulHaber-2250 2024)

Based on Table 2.1 the CubeMars GL35 and the RS-RPO 216-3794 are the only motors capable of delivering the required stall torque of 160 Nmm while also fulfilling the other requirements. Of those two the CubeMars GL35 is chosen since it has a lower KV-value. This makes it most relevant for this project since a wider voltage range can be used to operate it within the 0 – 1000 RPM spectrum. For serial production, the KV-value could be changed for the RS-RPO 216-3794 as explained in Section 2.2.1.

2.4 Control of AC Synchronous motors

Control of AC Synchronous motors with a DC supply is done through commutation methods, where the windings are energized at the right time using a three-phase inverter, thereby achieving the desired rotational speed. Three widely used commutation methods will be presented: Trapezoidal (also known as: Six-Step method), Sinusoidal Pulse Width Modulation (SPWM) (also known as Sine-Triangle PWM), and Space Vector Pulse Width Modulation (SVPWM). These methods can either be done sensor-based or sensorless, depending on the needed precision of the motor position and speed.

2.4.1 Commutation techniques

Trapezoidal, Sinusoidal PWM, and SVPWM commutation will be listed with a general description of how they function. All three can be done either position sensor-based or position sensorless.

Trapezoidal The most simple of the three is the trapezoidal, also known as six-step commutation, which energizes the windings in a simple on/off sequence. This is done in a six-step configuration, meaning every 60° deg of the electrical rotation. This yields a high achievable speed and torque, however, torque ripples are introduced at every commutation step.

Sinusoidal PWM (SPWM) The three motor windings are each energized with a current varying according to a sinusoidal wave, each phase shifted 120°. The sinusoidal commutation yields a smooth output curve, however, given the phase shift of the waves, the angle between the magnetic field of the stator and the rotor varies from the optimum 90°. As an effect, a torque ripple is introduced albeit smaller than for trapezoidal commutation. The maximum speed compared to trapezoidal is, however, lower.

Space Vector PWM (SVPWM) Space Vector PWM utilizes the stationary $\alpha - \beta$ coordinate system. Based on the activated transistors you get six distinguishable vectors in the $\alpha - \beta$ coordinate system and two zero vectors. Utilizing PWM makes it possible to switch between these vectors quickly in an ordered sequence giving the ability to create a voltage corresponding to any vector in the $\alpha - \beta$ coordinate system. This significantly increases the complexity of the commutation compared to

sinusoidal PWM and trapezoidal but in return gives lower torque ripple and a higher achievable voltage compared to SPWM.

2.4.2 Sensor-based solutions

When using a sensor-based solution, extra components are introduced to measure a given signal and thereby determine the position and speed. Different types of sensor-based solutions will be listed with a description of how they function.

Encoders

An encoder is a sensor measuring mechanical motion and converting it into a digital signal, thereby making it possible to measure the position and speed. It can be done both for rotational and linear motion, however the focus here will be on rotary encoders. An encoder can be in either an incremental or absolute configuration, where the incremental will get the data by counting the number of steps from a predetermined start point, contrary to an absolute configuration where each position has a unique characteristic.

Mechanical In an incremental mechanical encoder, two copper tracks provide positional information through contact brushes. In a quadrature encoder, these copper tracks correspond to two channels which are phase-shifted 90° . Based on the on-off sequence of the tracks, the rotational direction is found. In an absolute encoder, there are many tracks, providing a unique signal for each position.

Optical A light-emitting diode is used to shine light through a disk, where the light is detected by the photodetector on the other side. The disk is attached to the rotation shaft, so it has the same rotational motion. An optical encoder can be used both as an incremental and absolute, where the absolute configuration increases the complexity of the disk. Mechanical and optical encoders are often relatively large and expensive but can provide reliable high-resolution measurements.

Magnetic A hall effect sensor is used to measure the rotational motion through a changing rotational magnetic field which in turn induces a voltage over the sensor. This can either be done as incremental or absolute depending on the configuration of the magnets which are measured on.

Latch Works through bipolar operation meaning two opposite poles need to be applied to the Hall Latch for it to be activated and deactivated. Furthermore, the Hall Latch retains its state after the magnetic field is removed.

Switch Operation mode can be bipolar, unipolar, or omnipolar, where a sufficiently strong magnetic field, either of different polarities or the same, can activate/deactivate the Switch. The Hall Switch does not retain its state. Hall latches and switches provide a very low resolution which can only be used in six-step commutation, but are inexpensive.

2 Axis hall sensor A two-axis hall sensor can determine the absolute position of a two-pole magnet based on the magnetic field with high accuracy. This can be used to determine the position and speed of a shaft with a ring magnet placed on it. This is done by combining two hall latches placed orthogonal to each other.

A higher position resolution can be achieved by having more than two magnetic poles through a revolution. The disadvantage with more than two magnetic poles is that the absolute shaft position is not known during initialization, therefore, an incremental approach has to be used. These sensors can be in the form factor of a very small and inexpensive chip and can provide a higher resolution compared to latches and switches which only detect a certain sector of the rotation.

Capacitive Measures the capacitance of a disk by sending a high-frequency signal through the disk and measuring the signal after the disk. Given the capacitance from different areas of the disk, the position can be determined incrementally or absolutely.

Resolvers

When a sinusoidal current is applied to a transformer's primary winding, the induced current in the secondary winding is also sinusoidal. If the secondary winding is rotated relative to the primary, the amplitude of the induced sinusoidal current will change, where a resolver utilizes this changing amplitude. The resolver consists of a stator and a rotor, where the stator consists of an exciter winding, placed on the axis of the resolver, and two two-phase winding. The exciter winding induces a sinusoidal current into the rotor winding, which in turn induces a current in the two two-phase windings. Since these are placed at a 90° angle from each other, one winding will output a sinusoidal current and one a cosine. From the sinusoidal and cosine currents the exact position and speed can be determined, by analyzing the amplitude and frequency of these. As further explained in Appendix B.

2.4.3 Sensorless solution

There is also the possibility of doing a sensorless solution, further described in Appendix E, thereby reducing the number of components. The back emf of the motor is utilized to determine the speed and position of the rotor, however, due to the need to know the position of the rotor given the use case of a LEGO toy, achieving a satisfying result with a sensorless solution is deemed outside the scope of this report.

2.5 Deciding on commutation techniques and sensor type

Based on Section 2.4.1, the Space Vector PWM (SVPWM) is chosen since it offers the best efficiency and voltage output compared to both Trapezoidal and Sinusoidal methods. Using SVPWM, from here onwards the AC Synchronous motor will be classified as a Permanent Magnet Synchronous Motor (PMSM).

For measuring the position of the shaft, it is determined that it would be best to go with a 2-axis hall sensor. This is based on the fact that from a cost and manufacturing perspective, this solution is fairly cheap and easy to use on a PCB making it viable for a finished product that should comply with the requirements of volume and mass. This sensor can also acquire the absolute position of the rotor through a ring magnet with decent resolution, and possibly utilize the magnets along the outside of the rotor thereby increasing resolution without adding unnecessary parts. To validate the 2-axis hall sensor and compare the results for further calibration a mechanical encoder is also added to the setup.

2.5.1 Current Measurement

Current measurement is needed to implement FOC, which gives the highest output torque and improves the efficiency further described in Section 6.1. The two main ways to measure current are the voltage drop over a shunt resistor or utilizing a Hall effect sensor which measures the magnetic field.

The current measurement method is generally built into the inverter and is therefore determined along with the inverter board in Chapter 4.

2.6 Control Considerations for LEGO Sets

The use case of the motor control in LEGO sets has some requirements that are more important than others. Considering the steering rack of a LEGO car, positional accuracy and limiting overshoot are more important than having a very fast transient response, however, the settling time should be as low as possible while fulfilling the aforementioned requirements. Furthermore, the motor is also used to drive cars and operate the winch of cranes. Here a speed controller without overshoot and oscillations is the primary requirement of this controller since that would make the controlled object jerk back and forth. An additional consideration is the audible noise level since the LEGO sets are designed to be operated indoors without disturbing the user.

Problem Statement 3

In the problem analysis, the CubeMars GL35 is found to be the most suitable motor since it delivers the required torque at the desired speed with the possibility of using direct-drive. With regards to sensing the position of the shaft, it is determined that from a cost and manufacturing perspective, it would be best to go with a 2-axis hall sensor.

How can the CubeMars GL35 be controlled to outperform the LEGO Technic Large motor with regards to low-speed operation, efficiency, and position control using direct-drive?

System Setup 4

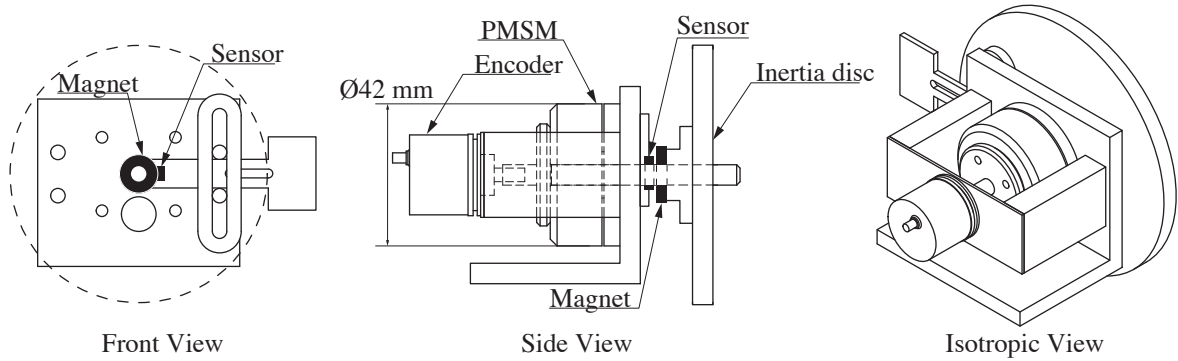


Figure 4.1. Test setup overview with inertia disk, hall sensor, and encoder.

To test the motor setup, verify the model, and determine if the motor is adequate for the stated problem, two test setups are made in the laboratory. Figure 4.1 shows the general setup, with the inertia disk attached. For both test setups, the motor is fixed to a mount attached to an aluminum baseplate, with the encoder attached with a metal bracket and connected to the motor with a rubber coupling. A 2-axis hall sensor measures on a ring magnet attached to the shaft, as can be seen on the front and side view in Figure 4.1.

Figure 4.1 shows the first setup with an inertia disk attached to the shaft. This test aims to see the motor's transient response and determine the power usage during the transient response. A steel disk with a diameter of 100 mm, a thickness of 6 mm, and weighing 0.37 kg is added onto the shaft increasing the moment of inertia with $460.7 \text{ kg} \cdot \text{mm}^2$.

Mass on a string

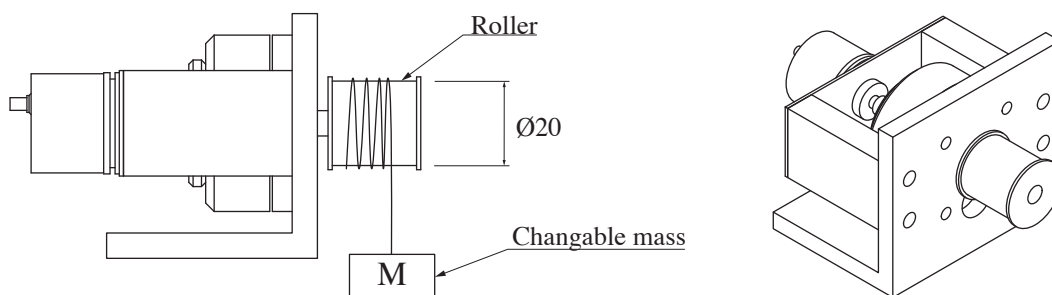


Figure 4.2. Simplified drawing of a test setup with a mass on a string

In the second test, the inertia disk is interchanged with a roller with a diameter of 20 mm, as shown in Figure 4.2, where a string, which is assumed inelastic with no elongation, is wrapped

around the wheel with a mass attached to the string. With the mass hanging freely, it is lifted for a distance of 1.5 m, with a ramp in speed, aiming to have constant acceleration and torque. This is used to determine the power usage, efficiency, and produce a performance curve of the motor. Additionally, the test setup can be used to analyze the disturbance rejection capability of the system, by doing a step in position against gravity.

4.1 Choice of equipment

Product name:	Category:	Rated Voltage:
CubeMars GL35	PMSM	16 V
STM32 F411RE	Microcontroller	[-]
Nucleo IHM07M1	Power module	8-48 V
Scancon SCA24	Incremental encoder	5-24 V
MPS TBMA730	2-axis hall sensor	3.3 V

Table 4.1. General Overview of Components.

As outlined in Chapter 3, the CubeMars GL35 PMSM, is chosen for further investigation. Due to its ease of use, data acquisition and motor control are done through an STM32 NUCLEO Development Board. Specifically, the STM32 NUCLEO-F411RE (microcontroller) is chosen, since this has adequate computational power and a dedicated port for encoders. The expansion board X-NUCLEO-IHM07M1 (Power Module) is required since it includes a three-phase half-bridge inverter, current measurement capabilities, and supports up to 48 V. The TBMA730-Q-LT-01A, a 2-axis Hall sensor, is chosen since it offers a 14-bit absolute position. The goal is to incorporate the 2-axis Hall sensor into a potential final product. To utilize the TBMA730 the Serial Peripheral Interface (SPI) protocol is applied, which is used for communicating with different peripheral devices. The sensor outputs a 16-bit signal measured on a ring magnet attached to the shaft to provide an absolute position. Furthermore, the SCA24 Incremental Encoder from Scancon is used to calibrate and verify the measurements from the 2-axis hall sensor.

4.2 General system overview

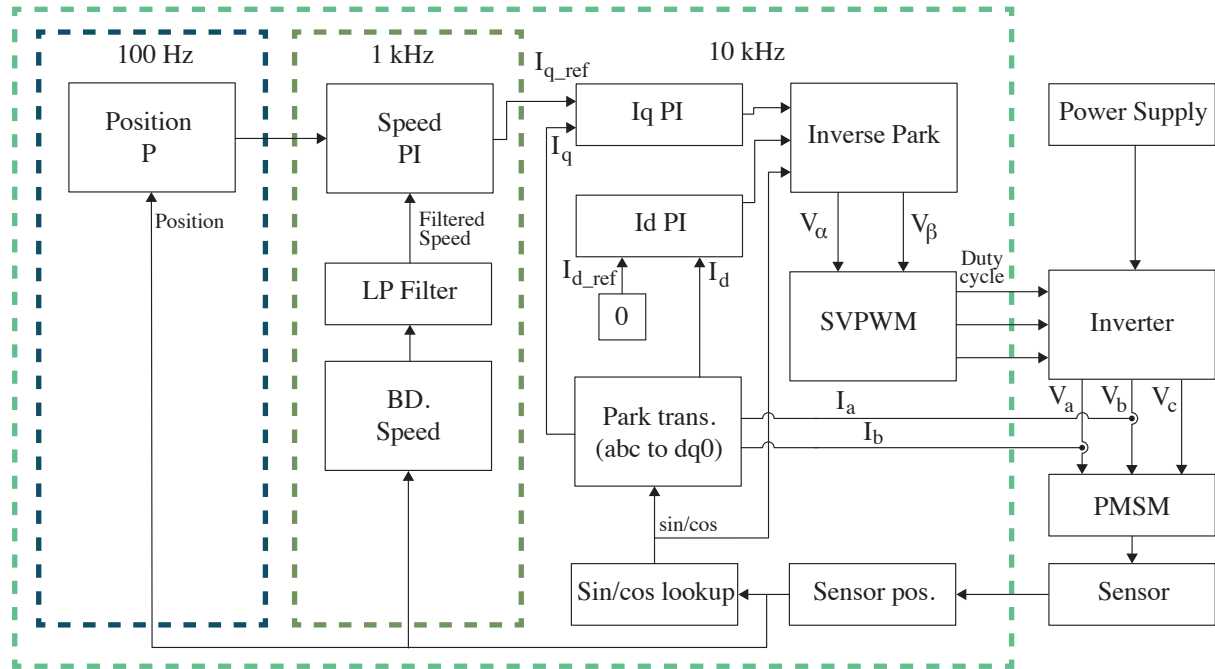


Figure 4.3. Overview of system implementation. (BD: Backward difference) LP: Low-pass

In Figure 4.3, a general overview of the system implementation can be seen, showing how the different parts of the system are connected. It is divided into two main sectors: the rightmost blocks outside of the dashed green line represent the physical setup and the blocks inside the outmost dashed green line represent the software system. The latter is further divided yielding three areas, each marked by either 100 Hz, 1 kHz, or 10 kHz, indicating the frequency of the blocks specified inside the dashed lines e.g. *SVPWM* updates with a frequency of 10 kHz and *Speed PI* updates with a frequency of 1 kHz. In the following sections, the different blocks will be further explained.

4.3 Inverter

The power module consists of the three half-bridges, gate logic, and the circuit required to measure the current in the three phases. The built-in gate logic means that there is a dead time of $1 \mu\text{s}$ in between the switching of transistors in the same half-bridge. The dead time is necessary to avoid a short-circuit between the positive and negative terminal when switching between the two transistors in the half-bridge. With the built-in dead time, the switching is controlled by inverted signals. When the invert signal is 1 the bottom transistor is closed and when the invert signal is 0 the top transistor is closed. This is done with a gate logic controller which also gives the ability to turn off each half-bridge individually using an enable signal. The transistor states for the different enable and invert signals can be seen in Figure 4.4. Here it can be noted that the enable signal is always 1 when the motor is operated.

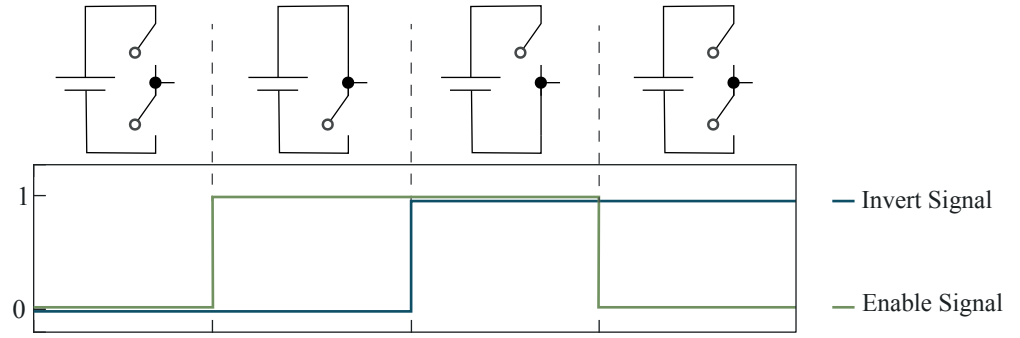


Figure 4.4. Illustration showing the effect of the enable and invert signals on the transistor states in a half-bridge.

The diagram of the inverter with the three half-bridges and their connection to the three phases can be seen in Figure 4.5. The connection with the shunt resistors used in current measurements can be seen.

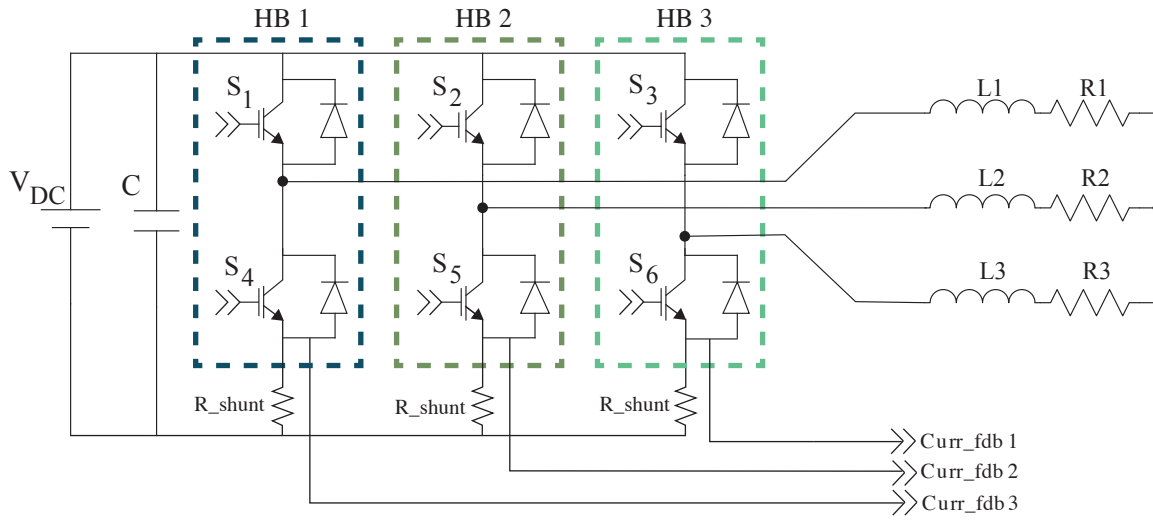


Figure 4.5. Illustration of simplified three-bridge inverter structure.

4.4 Software implementation

To control the motor, the software for the microcontroller has to read the sensors, do the calculations, and output the correct Pulse-Width Modulation (PWM) signals, and to do this, multiple timers, Analog-to-Digital Converter (ADC) channels, and digital channels are used. This is elaborated further in the next sections, and an example of the C Code structure can be seen in Algorithm 1. The C code can be seen in Appendix H.

Function *PWM interrupt function*

```

    Set GPIO pin 6A high
    Measure ADC values to determine current (done in the background)
    Measure angle
    i = i + 1
    if i = 10 then
        Calculate speed using backward difference
        Filter speed measurement
        i = 0
    Approximate sine and cosine for given angle using lookup table
    Convert ADC measurements to phase currents
    Use combined Park and Clark transform to convert currents to  $dq$ -system
    if Position-controller = Active then
        Calculate the error in position
        Calculate new reference velocity

    Function Velocity controller
        Calculate the error in velocity
        Calculate the reference  $q$ -axis current using velocity controller
    end
    Function Current controller
        Calculate the error in  $d$ -and  $q$ -axis current
        Calculate  $d$ -and  $q$ -axis voltage using current controller
    end
    Log the wanted data and send them over UART every 10000 iterations
    Function SVPWM calculations
        Saturate  $d$ -and  $q$ -axis voltage if the combined length is above  $V_{DC}/\sqrt{3}$ 
        Use Inverse park transform to convert  $dq$ -voltage to  $\alpha\beta$ -voltage
        Determine the sector of the reference  $dq$ -voltage
        Calculate the counts corresponding to  $T_0$ ,  $T_1$  and  $T_2$  in the given sector
        Update CCR to get the correct PWM waveform
    end
    Set GPIO pin 6A low
end

```

Algorithm 1: Pseudocode showing the overall structure of the 10kHz PWM loop.

4.4.1 PWM timer

PWM is used to control the phase voltage in the motor space vector, which is explained further in Section 6.2. Here, it is advantageous to use symmetrical PWM since this reduces the number of transistors that have to be switched. An up-down counter is used, which first counts up to its auto-reload register (ARR) value and then back down to 0. This can be seen in Figure 4.6.

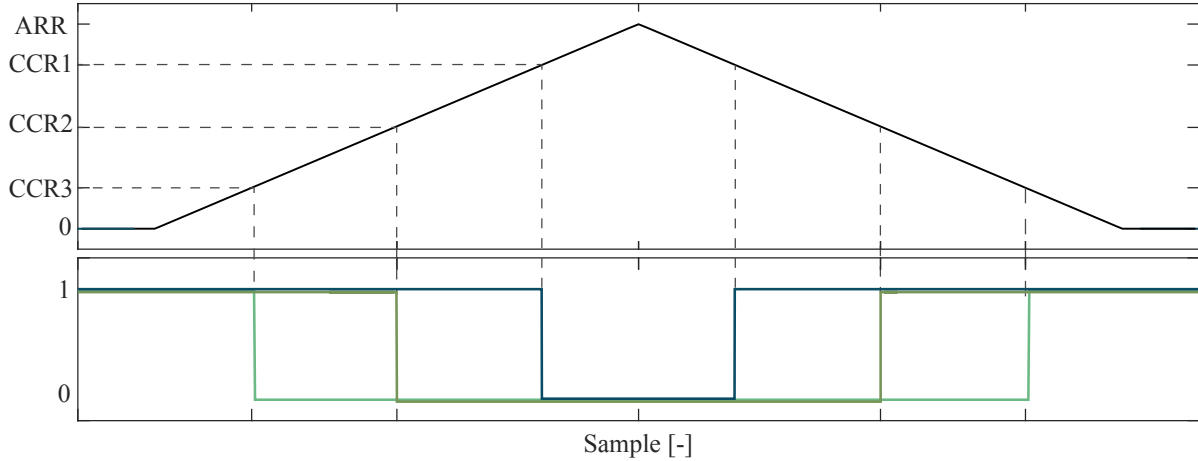


Figure 4.6. Figure showing the timer counting from 0 to the ARR value and back down. The CCR values control the corresponding PWM signals

Three counter-compare register (CCR) values are set up to generate an interrupt at a specific value of the the up-down timer. These values determine the PWM duty cycle of the invert pin of the three transistors. The invert pin controls which transistor is activated in each half-bridge. When the PWM signal which goes to the invert pin is 0, the top transistor is closed, and when the signal is 1 the bottom transistor is closed.

The desired ARR value can be determined based on the frequency of the up-down count combined with potential pre-scalers. This is done with Equation (4.1).

$$ARR = \frac{f_{clock}}{2 \cdot PR \cdot f_{timer}} \quad (4.1)$$

Where f_{clock} is the frequency of the clock cycle, PR is the counter period and f_{timer} is the clock for the timer.

4.4.2 PWM and control loop

To create a varying space vector, the duty cycles of the signals to the different half-bridges are changed. Since the desired space vector changes with rotor position and desired currents in FOC, the counter compare values should be changed with the same frequency as the current is sampled. To do this, a timer interrupt executes a subroutine at that frequency.

In this subroutine, the current controllers are executed every iteration since the current dynamics are the quickest. The speed calculation and controller are executed every 10 iterations (1 kHz) since it has significantly slower dynamics and therefore utilizes the theory of cascade control which is further elaborated in Section 7.2.2. Position control is done every 100 iterations of the loop (100 Hz) since the position dynamics are significantly slower than the speed dynamics.

4.4.3 Position measurement

To measure the position, both an incremental encoder and a 2-axis hall sensor are used. The encoder is used to get high-accuracy position data to validate models and furthermore, test the accuracy of the hall sensor. The rotary encoder is too large and expensive to be implemented in a potential final product. Therefore, a 2-axis hall sensor is also added to the setup to investigate the possibility of using it as a more cost-effective and compact position sensor.

The position measurements give a mechanical angle that does not correspond to the “electrical” angle used in the Park transformation. To determine the electrical angle from the mechanical angle, the offset between the two zero-points needs to be determined. This is done by activating the transistors corresponding to the V_1 -vector in Space vector pulse-width modulation (SVPWM), further explained in Section 6.2, moving the rotor to a position where the electrical angle is 0. The measured mechanical angle is then used as an offset.

The CubeMars motor has 7 pole pairs which means that 7 electrical revolutions are needed for each mechanical revolution. Here the electrical angle is found by modulating the mechanical angle, which in both cases is measured as an integer, by $1/7$ of the maximum angle. This means that the resolution of the electrical angle is $1/7$ of that of the mechanical angle.

The encoder has two channels with a 90° phase sending out signals, and measuring the encoder signal is done using a timer in encoder mode. Here, both rising and falling edges are counted meaning that with 3750 pulses pr. revolution 15000 distinct positions can be measured by increasing or decreasing the encoder count per rising and falling edge detected. To get an absolute position, which is needed for SVPWM, the Z channel of the encoder is utilized which produces a pulse for every mechanical rotation. This signal resets the encoder count to 0. Therefore, up to one rotation is needed to calibrate to the correct position measurement with the quadrature encoder.

The 2-axis hall sensor by default gives an absolute position since it measures the rotation of a magnetic field and converts it to a digital signal. This 16-bit signal, with a 14-bit resolution, can be read using the SPI communication interface. Due to the 2-axis hall sensor being mounted as a side-shaft mounting, as shown in Figure 4.1, the magnetic field angle and mechanical angle are no longer directly proportional. This leads to an error curve between the actual angular position and the measurement from the hall sensor. This error is shown in Figure 4.7, where the encoder shows a mostly linear rise in angular position and the hall sensor resembles more of a sine curve.

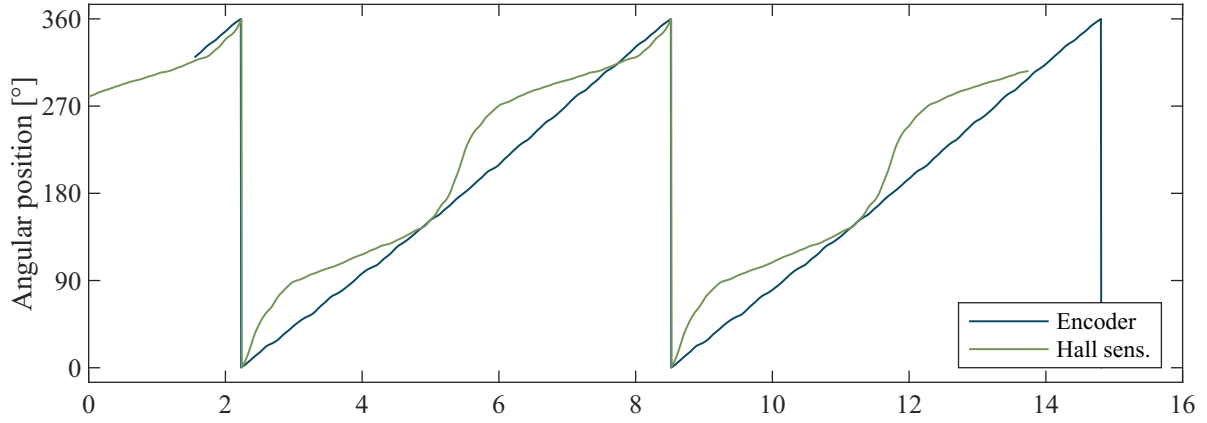


Figure 4.7. Angular position measurement for both the encoder and hall sensor

The MPS TBMA730 2-axis hall sensor can mitigate this by adjusting the register for the *Bias Current Trimming* (BCT) in the on-chip non-volatile memory through a write register operation. The BCT value adjusts how the 2-axis hall sensor weighs the magnetic field along the x and y -axis and is determined from experimental data. However, through multiple tests, it has not been possible to achieve a satisfying linear result with the BCT for the angular position measurement. Furthermore, it has been unsuccessful in achieving a useful result from measuring on the permanent magnets on the rotor. Therefore, the encoder will be used throughout the controller development.

4.4.4 Speed Calculation and Filtering

The rotational speed of the motor is calculated with backward difference from the encoder count at 1 kHz. This introduces noise since the sensor has a finite resolution which is being differentiated. Directly implemented into the speed controller, this noise causes spikes in the i_q reference used in the current controller, leading to significant audible vibration in the motor. To mitigate the noise, a low-pass filter is applied to the speed calculation. A low-pass transfer function discretized using Tustin's method was shown to provide nearly identical results compared to an exponential weighted moving average filter. Therefore the latter is chosen due to its clearer difference equation as seen in Equations (4.2) and (4.3) and it being easier to implement and adjust.

$$\alpha = e^{-\omega_{cut} \cdot T} \quad (4.2)$$

$$u_n = \alpha e_n + (1 - \alpha)u_{n-1} \quad (4.3)$$

Where α is the discretization coefficient, ω_{cut} is the cutoff frequency, T is the sampling time, e is Euler's Number, e_n is the input, and u_n is the output, where the subscript -1 indicates the previous step.

The filtering can be manipulated by changing the cutoff frequency ω_{cut} , where lower cutoff frequencies will lead to a smoother result but will cause a delay in the transient response. To analyze the effect of the chosen cutoff frequency, a Fourier Transform of the motor startup test is shown in Figure 4.8. Here, the steady-state vibrations can be seen as a spike at 144 Hz.

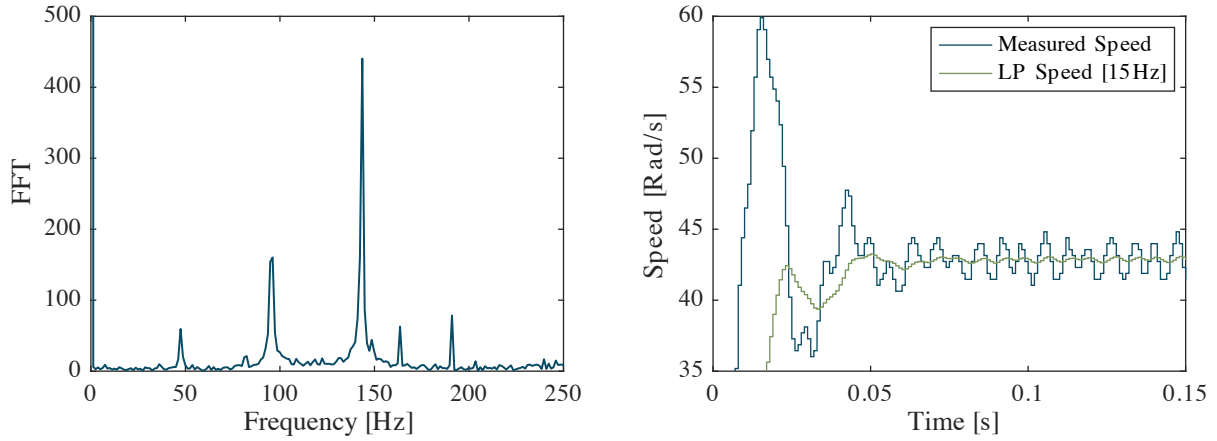


Figure 4.8. Left: Fast Fourier Transform of raw speed test data. The steady-state vibrations are seen as a large spike at 144 Hz. Right: Raw speed calculation and LP-filtered speed with a cutoff frequency of 15 Hz.

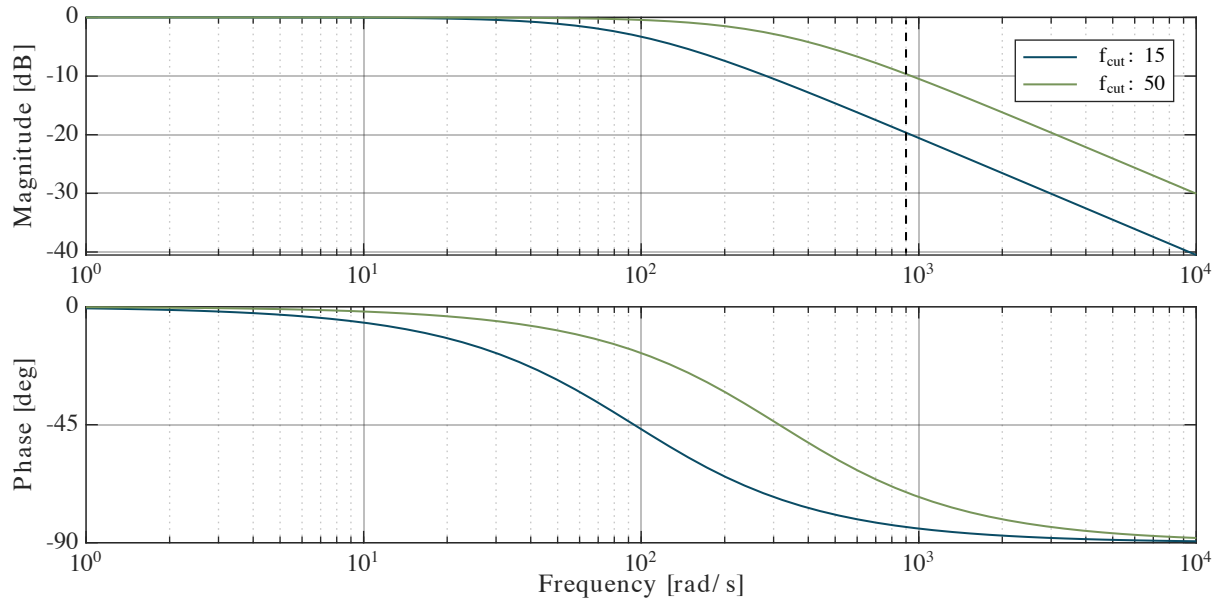


Figure 4.9. Frequency response plot of the low pass filter with cutoff frequencies of 15 and 50 Hz. The vibration frequency shown in Figure 4.8 is marked by a vertical line at the frequency corresponding to 144 Hz.

Encoder count reset handling

After each encoder rotation, the count is reset through an external interrupt from the encoder Z-channel, which causes a large spike in the speed calculated with finite difference. To remove this spike, an if-statement is set up to detect the positional difference between the two latest samples, which can be seen below. If the difference is larger than half a rotation in either direction, 360 degrees is added or subtracted from the calculation depending on the rotational direction, thereby removing the spike. An example of the C Code implementation can be seen in Algorithm 2.

Function *speed-calculation*

```

    phi_diff = phi - phi_old
    if phi_diff > 180 then
        | speed = (phi_diff - 360)/(0.001)
    else if phi_diff < -180 then
        | speed = (phi_diff + 360)/(0.001) ;
    else
        | speed = (phi_diff)/(0.001) ;
    end
end

```

end

Algorithm 2: Pseudocode of the if-statement to handle speed calculations when the counter resets.

4.4.5 Current measurement

The three shunt resistors placed between the bottom transistors and ground, as shown in Figure 4.5, are used to calculate the current by measuring their voltage drop. Toggling the transistor creates current ripples which are not desired to be measured. Therefore the current should be measured in the middle of the period where the bottom transistor in the half-bridge is closed, meaning the PWM inverter signal created by the timer should be high at the given point. This effect can be seen in the right plot of Figure 4.10.

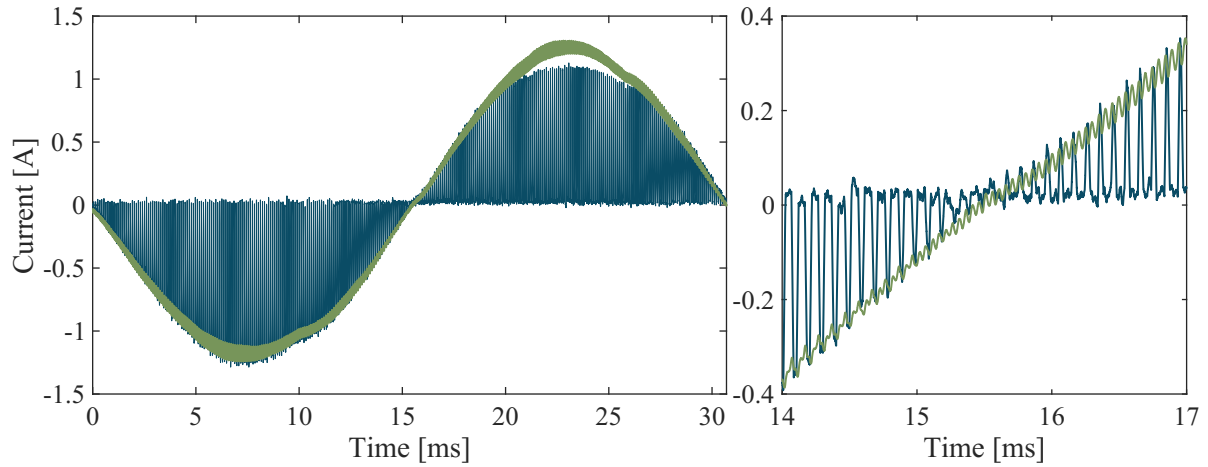


Figure 4.10. Current measurement with a current probe (green), and a voltage measured directly across a shunt resistor, recalculated to a current (blue). The ADC is timed to do a measurement at the center of the voltage peaks.

This further gives the average current through the shunt resistor since it changes linearly over time when the transistor is closed, as shown in Figure 4.11.

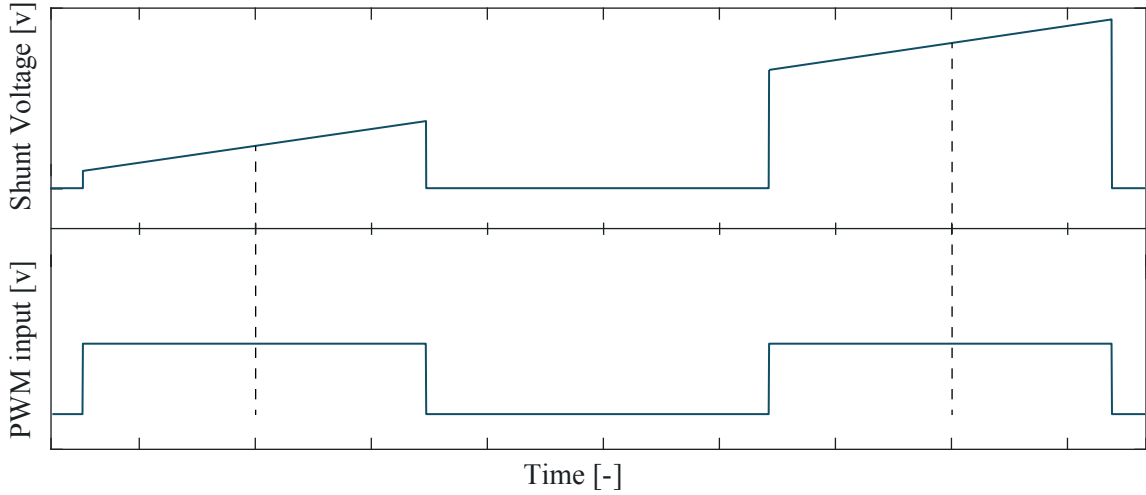


Figure 4.11. Timing of current measurements.

An ADC measures the amplified and offset voltage drop over the shunt resistor. A voltage divider is used to offset the voltage to measure both positive and negative currents as the built-in ADC can only measure positive voltages. Furthermore, an operational amplifier is used to amplify the voltage to make use of the ADC's entire voltage range. The circuit with the voltage divider and operational amplifier can be seen in Figure 4.12.

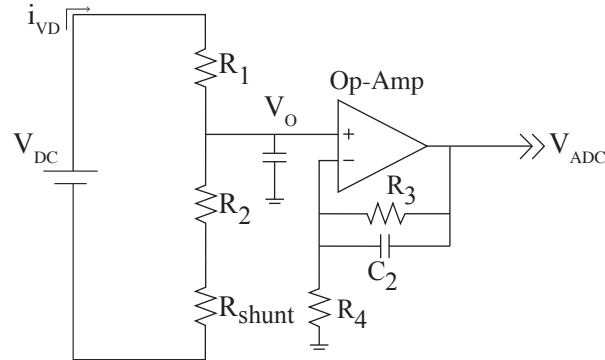


Figure 4.12. Amplification and offset circuit with voltage divider and op-amp (Arrow 2024)

The ADC measurement gives a 12-bit integer value from 0 – 4095 corresponding to 0–3.3 V and outputting it as a value between 0-1. Equation (4.4) is used to convert the raw ADC measurements to the physical voltage.

$$V_{ADC} = ADC_{raw} \frac{3.3}{4095} \quad (4.4)$$

V_{ADC} is the voltage read from the ADC and ADC_{raw} is the raw integer value. The voltage input to the op-amp can be found using Ohms law and Kirchoff's voltage law with the assumption that no current goes into the op-amp. Furthermore, it is assumed that all of the voltage drop over R_{shunt} comes from the current in the phase. Based on these assumptions the voltage can be determined from Equations (4.5) and (4.6).

$$V_{DC} = R_1 \cdot i_{VD} + R_2 \cdot i_{VD} + V_{shunt} \Rightarrow i_{VD} = \frac{V_{DC} - V_{shunt}}{R_1 + R_2} \quad (4.5)$$

$$V_O = R_2 \cdot i_{VD} + V_{shunt} \Rightarrow V_O = \frac{V_{shunt} \cdot R_1 + V_{DC} \cdot R_2}{R_1 + R_2} \quad (4.6)$$

Here V_{DC} is the input voltage, R_1 and R_2 are the resistors in the offset circuit, i_{VD} is the current through the voltage divider, V_{shunt} is the voltage drop over R_{shunt} and V_O is the voltage out of the amp. The op-amp acts as a non-inverting amplifier in this configuration, where the gain ratio is given as Equation (4.7).

$$V_{ADC} = V_O \cdot \left(1 + \frac{R_3}{R_4}\right) \quad (4.7)$$

R_3 is the resistor connected to the inverting input of the op-amp, and R_4 is the resistor connected to the ground. The relationship between the ADC measurement and the voltage drop over the shunt resistor can be established as Equation (4.8).

$$V_{shunt} = \left(V_{ADC} - \frac{V_{DC} \cdot R_2 \cdot (R_3 + R_4)}{(R_1 + R_2) \cdot R_4}\right) \cdot \frac{(R_1 + R_2) \cdot R_4}{R_1 \cdot (R_3 + R_4)} = (V_{ADC} - 1.56 \text{ V}) \cdot 0.655 \quad (4.8)$$

The results in Equation (4.8) have been verified by comparing it to an oscilloscope measurement of the voltage drop over the shunt resistor. Ohm's law is used to convert the voltage drop over the shunt resistor to a current as shown in Equation (4.9).

$$i_{shunt} = \frac{V_{shunt}}{R_{shunt}} = (V_{ADC} - 1.56 \text{ V}) \cdot \frac{0.655}{0.33} \quad (4.9)$$

The capacitors in the circuit in Figure 4.12 are used as a lowpass filter to limit noise disturbances in the reading. The current through the parallel circuit of R_3 and C_2 can be described as Equation (4.10).

$$i = \frac{dV}{dt} \cdot C_2 + \frac{V}{R_3} \quad (4.10)$$

This can be converted into the Laplace domain and rewritten as a transfer function, shown in Equation (4.11).

$$i = V \left(s \cdot C_2 + \frac{1}{R_3}\right) \Rightarrow \frac{V}{i} = \frac{R_3}{R_3 \cdot C_2 \cdot s + 1} \quad (4.11)$$

This yields a lowpass filter as a function of R_3 and C_2 with a cutoff frequency of $\frac{1}{R_3 \cdot C_2}$.

4.4.6 Optimization of C-code

Parts of the code are run at 10 kHz meaning that the calculations have to be done within a 100 μ s time window, however, due to resource constraints on the F411RE development board, the code has to be optimized to be able to run at this frequency.

To see the computation time a GPIO output pin is set high at the start of the 10 kHz loop and low at the end. This way the time period can be visualized on an oscilloscope and the impact of the optimization efforts can be measured. The most expensive calculations are the sine and cosine calculations, which are used in the FOC method. To lower the computational cost of these calculations, sine and cosine lookup tables are used, providing pre-calculated results for a finite amount of angle intervals. Additionally, quarter wave symmetry is utilized to only need a fourth of the table size. Instead of calling the lookup tables for the same angle several times per

loop, the sine and cosine lookup values for the given angle are stored in two variables that are only updated once per loop. However, the Park Transform includes four terms which include a 120° phase shift, meaning that the stored lookup value can no longer be used directly. Instead of doing four new lookup operations, trigonometric identities can be exploited to still only use the two sine and cosine lookup variables for the given angle:

$$\cos\left(\theta + \frac{2\pi}{3}\right) = -\frac{\cos(\theta)}{2} - \frac{\sqrt{3}}{2}\sin(\theta) \quad (4.12)$$

Another way processing time is cut is by replacing mathematical operations with constants whenever possible, such as square roots, divisions, products, etc. This makes the code run faster but trades off readability.

Modeling the motor 5

To predict the behavior of the PMSM combined with the required motor control, models for the different parts of the systems are set up and implemented in Matlab such that a simulation of the whole system can be made. This chapter will describe the parts modeled such that further control strategies can be investigated.

5.1 PMSM model

The motor model consists of two parts: an electrical and a mechanical model. The electrical model includes the electrical circuit and magnetic flux, while the mechanical model includes the load on the motor and moment of inertia. Both parts will be expanded in the following.

5.1.1 Electrical model

A simplified sketch of the PMSM and its electrical circuit can be seen in Figure 5.1. Here as , bs and cs are the three stator phases connected in a star connection to a common neutral in the middle. The common neutral is not connected to the ground, however, the neutral point comes from the vector sum of the voltage and currents in the phases being equal to zero, as an effect of the 120° displacement between the phases as can be seen in Figure 5.1b.

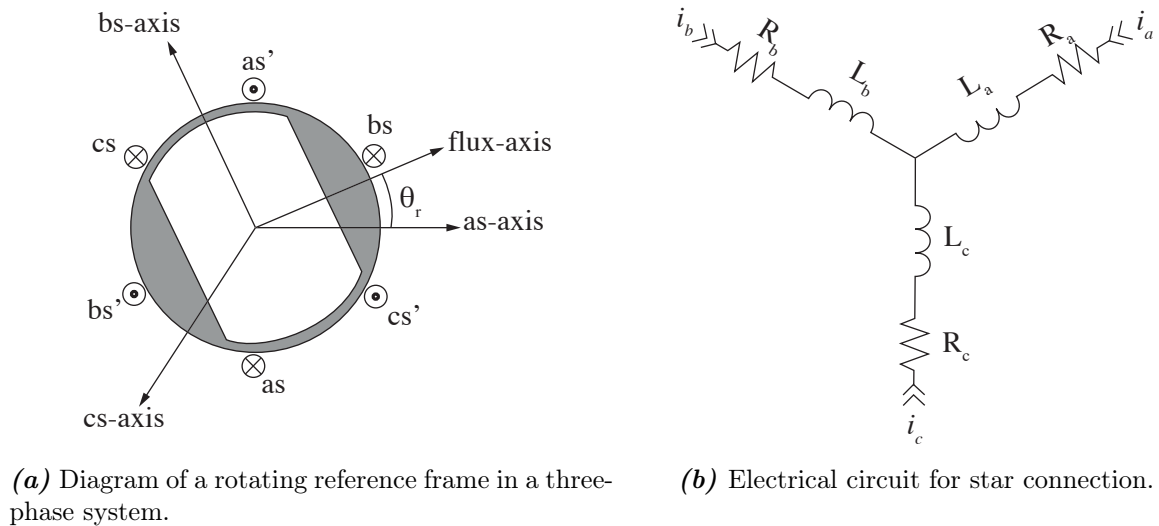


Figure 5.1. Illustrations of a PMSM.

The resistance over the three phases is assumed equal leading to a stator resistance $R_s = R_a = R_b = R_c$. The voltage equation for the three phases can be seen in Equation (5.1) where the

vectors all are components of the 3 different phases i.e. all are 3x1 $\mathbf{x}_{abcs} = [x_{as} \ x_{bs} \ x_{cs}]^T$ vectors and the resistance is a 3x3 matrix (Kim 2017).

$$\mathbf{V}_{abcs} = \mathbf{R}_s \cdot \mathbf{i}_{abcs} + \frac{d\lambda_{abcs}}{dt} \quad (5.1)$$

The flux linkages λ_{abcs} can be split into two parts as shown in Equation (5.2). One is the mutual flux linkage between the coils $\lambda_{abcs(s)}$ and the other is the flux linkage between the permanent magnet and the coils $\lambda_{abcs(f)}$.

$$\lambda_{abcs} = \lambda_{abcs(s)} + \lambda_{abcs(f)} \quad (5.2)$$

The flux linkage $\lambda_{abcs(s)}$ between the phase inductors is independent of the rotor position because the rotor's permanent magnets have a very high magnetic reluctance similar to that of air.

The flux linkage $\lambda_{abcs(f)}$ from the permanent magnets is however position-dependent. This is due to the permanent magnets rotating as the rotor rotates, causing a changing magnetic field, and inducing the back EMF in the voltage equation. This gives the following voltage Equation (5.3). Here L_{ls} is the leakage inductance and L_m is the magnetization inductance.

$$\begin{bmatrix} V_{as} \\ V_{bs} \\ V_{cs} \end{bmatrix} = \begin{bmatrix} R_s & 0 & 0 \\ 0 & R_s & 0 \\ 0 & 0 & R_s \end{bmatrix} \begin{bmatrix} i_{as} \\ i_{bs} \\ i_{cs} \end{bmatrix} + \begin{bmatrix} L_{ls} + L_m & -\frac{1}{2}L_m & -\frac{1}{2}L_m \\ -\frac{1}{2}L_m & L_{ls} + L_m & -\frac{1}{2}L_m \\ -\frac{1}{2}L_m & -\frac{1}{2}L_m & L_{ls} + L_m \end{bmatrix} \frac{d}{dt} \begin{bmatrix} i_{as} \\ i_{bs} \\ i_{cs} \end{bmatrix} + \omega_r PP \begin{bmatrix} \cos(\theta_r) \\ \cos(\theta_r - \frac{2}{3}\pi) \\ \cos(\theta_r - \frac{4}{3}\pi) \end{bmatrix} \phi_f \quad (5.3)$$

Applying that in a three-phase system $i_{as} + i_{bs} + i_{cs} = 0 \Leftrightarrow i_{as} = -(i_{bs} + i_{cs})$ to V_{as} in Equation (5.3) yields Equation (5.4) for V_{as} .

$$V_{as} = R_s \cdot i_{as} + (L_{ls} + L_m) \frac{di_{as}}{dt} - \frac{1}{2}L_m \left(\frac{di_{bs}}{dt} + \frac{di_{cs}}{dt} \right) = R_s \cdot i_{as} + \left(L_{ls} + \frac{3}{2}L_m \right) \frac{di_{as}}{dt} \quad (5.4)$$

Here ω_e is the electrical rotation. Applying this to V_{bs} and V_{cs} as well leads to the inductance matrix being reduced, which gives Equation (5.5).

$$\begin{bmatrix} V_{as} \\ V_{bs} \\ V_{cs} \end{bmatrix} = \begin{bmatrix} R_s & 0 & 0 \\ 0 & R_s & 0 \\ 0 & 0 & R_s \end{bmatrix} \begin{bmatrix} i_{as} \\ i_{bs} \\ i_{cs} \end{bmatrix} + \begin{bmatrix} L_{ls} + \frac{3}{2}L_m & 0 & 0 \\ 0 & L_{ls} + \frac{3}{2}L_m & 0 \\ 0 & 0 & L_{ls} + \frac{3}{2}L_m \end{bmatrix} \frac{d}{dt} \begin{bmatrix} i_{as} \\ i_{bs} \\ i_{cs} \end{bmatrix} + \omega_r PP \begin{bmatrix} \cos(\theta_r) \\ \cos(\theta_r - \frac{2}{3}\pi) \\ \cos(\theta_r - \frac{4}{3}\pi) \end{bmatrix} \phi_f \quad (5.5)$$

Here ϕ_f is the permanent magnetic flux. The motor modeling is converted to the rotating d - q coordinate system, yielding significantly simpler equations with DC-values at steady-state.

Furthermore, controlling the d - q currents is a prerequisite for the implementation of Field Oriented Control (FOC), which will be further expanded in Section 6.1. In the d - q coordinate system, the voltage equation for V_d can be seen in Equation (5.6) and in Equation (5.7) for V_q . Here $L_d = L_q = L_{ls} + \frac{3}{2}L_m$.

$$V_d = R_s \cdot i_d + \frac{d(i_d \cdot L_d + \phi_f)}{dt} - \omega_r \cdot PP \cdot L_q \cdot i_q \quad (5.6)$$

$$V_q = R_s \cdot i_q + \frac{di_q \cdot L_q}{dt} + \omega_r \cdot PP \cdot L_d \cdot i_d + \omega_r \cdot \phi_f \quad (5.7)$$

The d - q current derivatives are isolated from Equation (5.6) and (5.7) into Equation (5.8) and (5.9). This is done since integration is more numerically stable than differentiation and it is the form the equations need to take to be formulated in state-space form.

$$\frac{di_d}{dt} = \frac{V_d}{L_d} - \frac{R_s \cdot i_d}{L_d} + \frac{\omega_r \cdot PP \cdot L_q \cdot i_q}{L_d} \quad (5.8)$$

$$\frac{di_q}{dt} = \frac{V_q}{L_q} - \frac{R_s \cdot i_q}{L_q} - \omega_r \cdot PP \cdot \left(\frac{L_d \cdot i_d}{L_q} + \frac{\phi_f}{L_q} \right) \quad (5.9)$$

Here PP are the pole pairs.

5.1.2 Mechanical model

The mechanical system is modeled by Newton's second law for rotation, which is shown in Equation (5.10).

$$J \cdot \dot{\omega} = \tau_e - B \cdot \omega - \tau_{friction} - \tau_{load} \quad (5.10)$$

Here J is the moment of inertia and $\dot{\omega}$ is the angular acceleration. $\tau_e = \frac{3P}{4} \cdot \phi_f \cdot i_q(t)$ is the torque output from the PMSM motor as a function of the q -axis current. This torque is derived in Section 6.1.

The friction is modeled with viscous friction B and static coulomb friction $\tau_{friction}$. The viscous friction is linearly dependent on the motor velocity and is therefore implemented directly. The static friction is however nonlinear, meaning it ideally would be modeled such that the friction force can increase without any change in angle θ . This friction is modeled as a hyperbolic tangent function with a steep increase from 0 to ± 1 which is a scaling of the friction. This is then multiplied by the constant static friction coefficient, yielding an approximated physical behavior while maintaining numerical stability.

The load torque τ_{load} mimics an external load applied to the system, which somewhat does the same in the simulation as the coulomb friction, giving a constant load to the model. The load is however set up such that a time-varying load can be simulated to get the dynamics of the rest of the model. Another difference is that while coulomb friction is nonlinear and will drop out when linearized for control design, external torque will be described as a disturbance in state-space/transfer function models.

Equivalent moment of inertia

To calculate the equivalent moment of inertia (MOI) of the setup the MOI of the different parts has to be determined. SolidWorks is used to determine the inertia disk and the shaft and for the encoder, the MOI is found in the data sheet. CubeMars does not state the MOI of the rotor and is therefore calculated with the assumption that the outer rotor is a thin-walled hollow cylinder. From this, the MOI can be calculated as $J_{rotor} = m_{rotor} \cdot r_{rotor}^2$.

The total MOI can then be found by adding the MOI of the different parts together since they all rotate about the same axis with the same speed. The different MOIs are presented in Table 5.1 where it can be seen that without the inertia disk and reel for the string, the rotor is by far the largest contributor to the MOI.

Part	[kg · mm ²]
Rotor	12.5
Shaft	2.3
Encoder	0.1
Bolts for rotor mount	0.3
Inertia disk	460.7
Reel for string	0.6
Total without inertia disk or roll for string	14.88
Total with reel for string	15.48
Total with inertia disk	475.6

Table 5.1. Moment of inertia of the different parts.

5.2 Inverter models

The transistors in the inverter are modelled as ideal, meaning the voltage/current dynamics can be ignored. Ideal switches also mean that the models and simulations are working without switching losses and current build-up, simplifying the system to focus on motor dynamics with ideal PWM switching (Yu 1999).

5.3 Model validation

The presented models are both used to describe the behavior of the physical system and to further simulate system responses with control strategies implemented. To ensure that the mathematical models are accurately depicting the physics, tests are performed and compared to simulated results to see how alike the responses are. Firstly an open-loop test is performed mainly to verify the electrical model, and then a test where the current loop is closed and the mechanical response is verified.

5.3.1 Electrical model test

To validate the electrical model a voltage step input of 11.6 V is done through phases A and B, using the transistors in the inverter. Here the voltages and currents are measured and the response is plotted in Figure 5.2. The electrical model is a first-order system, where the resistance is tuned to get the correct DC-gain. Here the resistance is found as:

$$2 \cdot R_s = \frac{V_{DC}}{i_{DC}} = 3.5 + 2 \cdot 0.75 + 0.33 + 0.3$$

Most of the resistance observed can be derived from the different components: $R_{motor} = 3.5 \Omega$, $R_{dson} = 0.75 \Omega$, $R_{shunt} = 0.33 \Omega$. The last 0.3Ω is needed to fit the model, however, it is not related to a specific component. Afterwards, the inductance is tuned to get the correct transient response since it affects the system's time constant. This gives an inductance of $L_m = 1.6 \text{ mH}$ with the assumption that the leakage inductance L_{ls} is small.

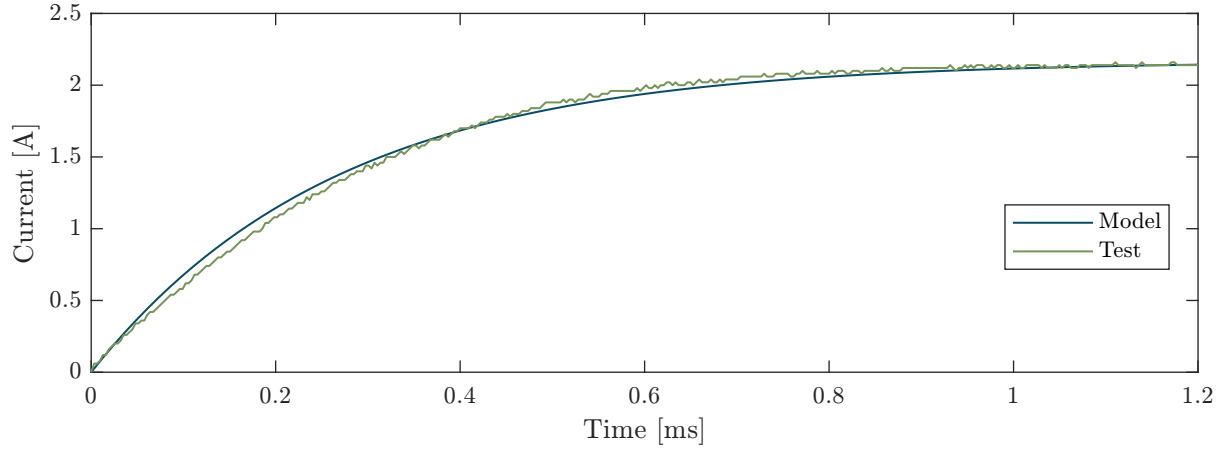


Figure 5.2. Current response from a voltage step input of 11.6 V

5.3.2 Current open-loop test

A reference voltage and angular position are supplied for the motor's space vector to follow. The test is performed with no additional load on the motor, and only the inertia of the rotor and the shaft. The applied space vector is transformed into the d - q axis coordinate system as inputs to the motor model. The corresponding currents are measured and compared with the simulation, where parameter tuning is performed to make the model resemble the test more. The resistance and inductance are based on Section 5.3.1. After the tuning, with both tests, are presented in Table 5.2.

Parameter			After tuning
Resistance	R_s	$[\Omega]$	$(3.5+0.75 \cdot 2+0.33+0.3)/2$
Inductance	L_d	$[H]$	$0.0024/2$
Torque constant	K_t	$[Nm/A]$	0.1
Moment of inertia	J	$[kg \cdot m^2]$	$16.25 \cdot 10^{-6}$
Vicious friction	B	$[N \cdot s]$	$1 \cdot 10^{-4}$
Coulomb friction	$\tau_{friction}$	$[Nm]$	$8.5 \cdot 10^{-5}$

Table 5.2. Parameters for Simulink. Note that these values are measured phase to phase in series, so resistance and inductance are divided by 2 to get the value for each phase.

Applying the tuned parameters yields the current response shown in Figure 5.3.

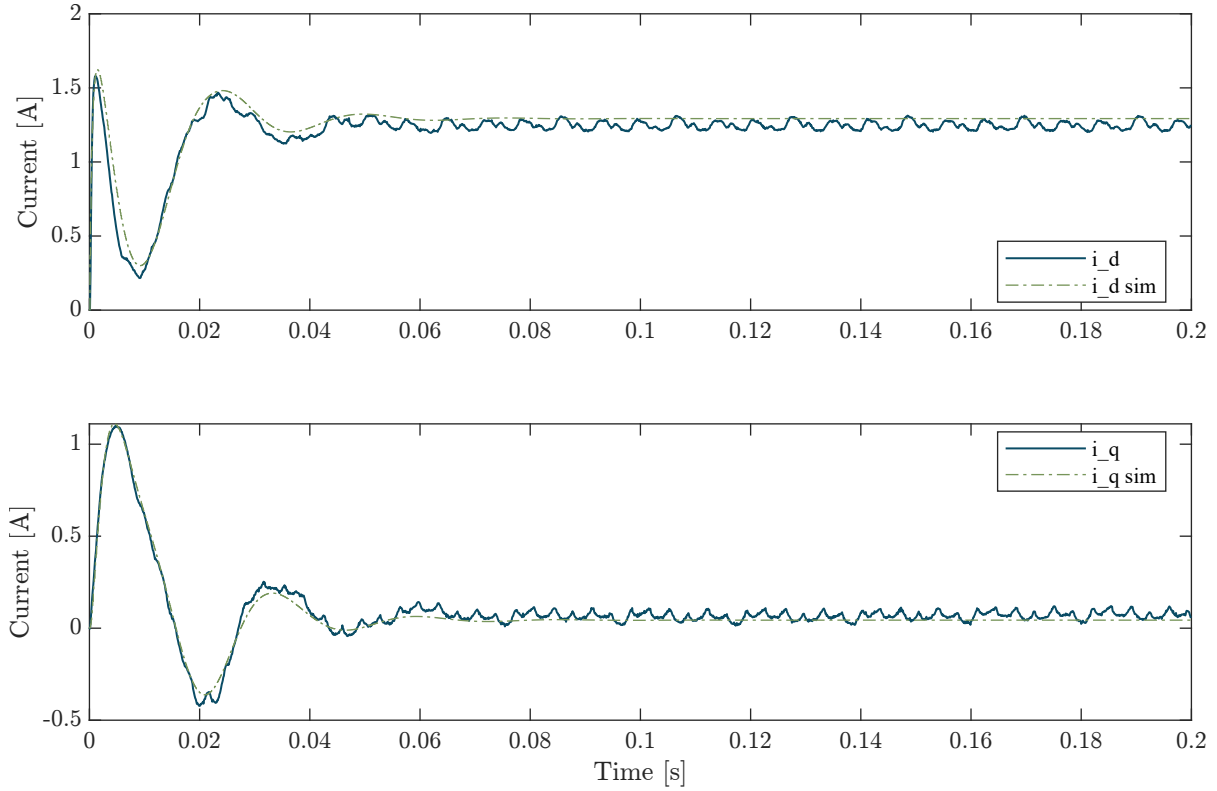


Figure 5.3. Current response of open-loop test

From Figure 5.3 both the transient response and steady-state values of the simulation describe the same behavior as the physical test with variations that could be explained by measurement noise. Therefore the electric model in open loop is deemed a sufficient representation of the setup and can be used further for controller design.

This test is performed without PWM commutation logic or switching of gate signals, to simulate the transistor on/off states. The same test is performed with PWM commutation and shows little variation beyond more noise in the signal which might be due to the filter size that is chosen to mimic the effect of inductances in the motor. The motor velocity is investigated during this test and shows a correlation between the model and the test data. The plots for the PWM test and velocity are shown in Appendix F.

5.3.3 Speed open-loop current closed-loop test

To further verify the mechanical model, the feedback current loop is closed, and a step input is applied to the q-axis current. With current control, the dynamics of the electric response can be neglected and the mechanical response is tuned through friction. As seen in the right side of Figure 5.4 the q-axis current reaches the step value of 1 nearly instantly. In the left side of Figure 5.4 the speed of the test and simulation is shown to have a good correlation after tuning parameters are fitted. For this test, an inertia disk is mounted on the shaft to get a larger transient period of the test. The inertia of the system with inertia disk is $J = 475.25 \cdot 10^{-6}$ but the inertia of the motor rotor itself is as stated in Table 5.2 along with the parameters

that showed similarity between simulation and test data. The only noteworthy difference is the closed-loop currents which do not reach the same steady-state values. The simulated response holds the saturated q-axis current longer which is likely due to model imperfections. Lowering frictions or increasing resistance gets the currents closer to the test data values but overall makes the speed and currents from the previous test deviate more. Therefore these parameter values are considered to give a good approximation of the physical system and will be used to get performance ideas for different tests and controls.

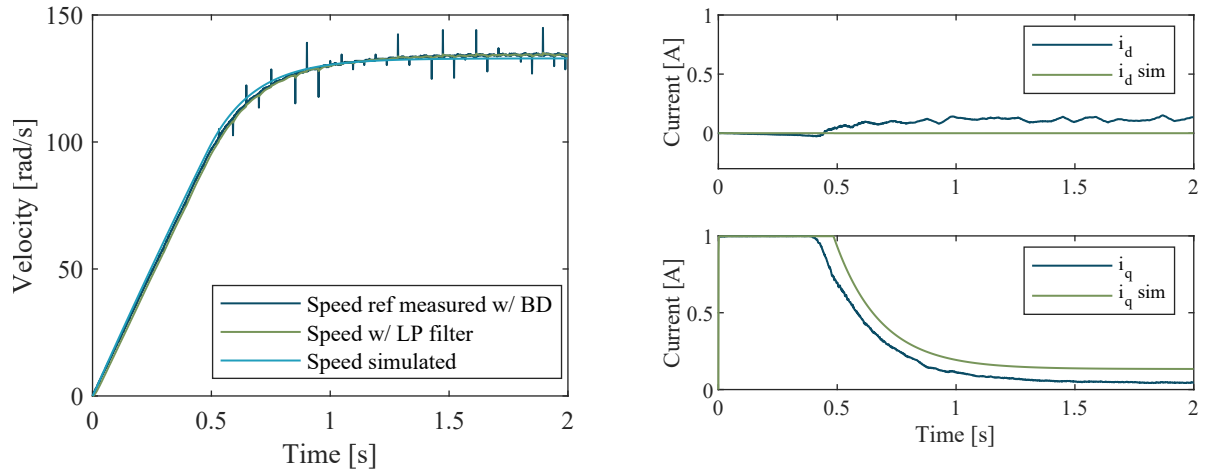


Figure 5.4. Left plot: Speed output of q-axis current unit step. Right: Closed-loop current from step.

Control Architecture 6

With the models validated, different control architectures can be investigated. First Field-Oriented Control (FOC) and its advantages are explained, followed by Space Vector Pulse-Width Modulation (SVPWM) given this is necessary for implementing FOC.

6.1 Field-Oriented Control

The idea behind FOC is to control the magnetic field's orientation generated by the motor coils of the stator. Here the q -axis currents generate the torque while the d -axis currents generate the losses, which will further be explained in the following. The derivation for this comes from the power equation of PMSMs, as shown in Equation (6.1).

$$P = \frac{3}{2}(V_d \cdot i_d + V_q \cdot i_q) \quad (6.1)$$

Substituting in voltage equations and rearranging gives an expanded equation for the power in Equation (6.2).

$$P = \frac{3}{2} \left(R_s(i_d^2 + i_q^2) + i_d \cdot \frac{d\lambda_d}{dt} + i_q \cdot \frac{d\lambda_q}{dt} + \omega_r \cdot \phi_f \cdot i_q + \omega_r(L_d - L_q)i_d \cdot i_q \right) \quad (6.2)$$

Here the part related to the resistance is stator copper losses, the derivatives are variations of magnetic energy and the remainder is mechanical power developed by the currents. For a surface-mounted PMSM, there is no variation of magnetic energy, meaning it is equal along the surface, and these terms drop out.

By dividing with the angular speed, the torque equation can be described by Equation (6.3).

$$\tau_e = \frac{3P}{4} (\phi_f \cdot i_q + (L_d - L_q)i_d \cdot i_q) \quad (6.3)$$

Here the inductances are related to reluctance torque and for a surface-mounted PMSM, the phase inductance is approximately equal $L_d \approx L_q$ meaning there is no reluctance torque. This leaves only the q -axis current multiplied by the constant to develop the torque of the motor. This means that the d -axis current only generates losses, which means it should be kept to zero.

To control the d - q currents, the correct d - q voltages have to be “created” using the inverter. This is done using Space Vector PWM (SVPWM) which dynamically determines the gate logic based on the desired α - β voltages which are determined from the rotation of the rotor and the d - q voltages using an inverse park-transform. SVPWM is further explained in Section 6.2.

6.2 Space Vector PWM

Using the SVPWM technique, the desired 3-phase voltages are transformed into a space vector representation in an α - β reference frame. The generated voltage reference vector rotates with the electric velocity. This reference vector is used to determine the PWM signals to create the desired 3-phase output voltage. The transistors in the inverter can make 8 distinct voltage vectors. The transistor states and corresponding phase voltages and space-vector voltage can be seen in Table 6.1.

Switch states			Phase voltages			Space vector voltage		
S_1	S_2	S_3	V_a	V_b	V_c	V_n (sectors)		
0	0	0	0	0	0	$V_0 = 0$	$\angle$	0°
1	0	0	$2/3V_{DC}$	$-1/3V_{DC}$	$-1/3V_{DC}$	$V_1 = 2/3V_{DC}$	$\angle$	0°
1	1	0	$1/3V_{DC}$	$1/3V_{DC}$	$-2/3V_{DC}$	$V_2 = 2/3V_{DC}$	$\angle$	60°
0	1	0	$-1/3V_{DC}$	$2/3V_{DC}$	$-1/3V_{DC}$	$V_3 = 2/3V_{DC}$	$\angle$	120°
0	1	1	$-2/3V_{DC}$	$1/3V_{DC}$	$1/3V_{DC}$	$V_4 = 2/3V_{DC}$	$\angle$	180°
0	0	1	$2/3V_{DC}$	$-1/3V_{DC}$	$1/3V_{DC}$	$V_5 = 2/3V_{DC}$	$\angle$	240°
1	0	1	$1/3V_{DC}$	$-2/3V_{DC}$	$1/3V_{DC}$	$V_6 = 2/3V_{DC}$	$\angle$	300°
1	1	1	0	0	0	$V_7 = 0$	$\angle$	0°

Table 6.1. Switching states for SVPWM

Here V_0 and V_7 are zero voltage vectors used to regulate the magnitude of the output to match the reference, and the rest are active voltage vectors outputting a certain phase voltage. These phase voltages can be found at steady-state applying Ohm's and Kirchoff's laws. The electrical connections for Vector 1 can be seen in Figure 6.1a, where the voltage drop over V_b and V_c are negative since they go against the direction defined as positive. $V_b + V_c = V_{DC}$ and since the current going through R_b and R_c is half the current going through R_a due to Kirchoff's current law, the voltage drop over R_a is $V_a = 2 \cdot (-V_b)$. This gives the values shown in Table 6.1.

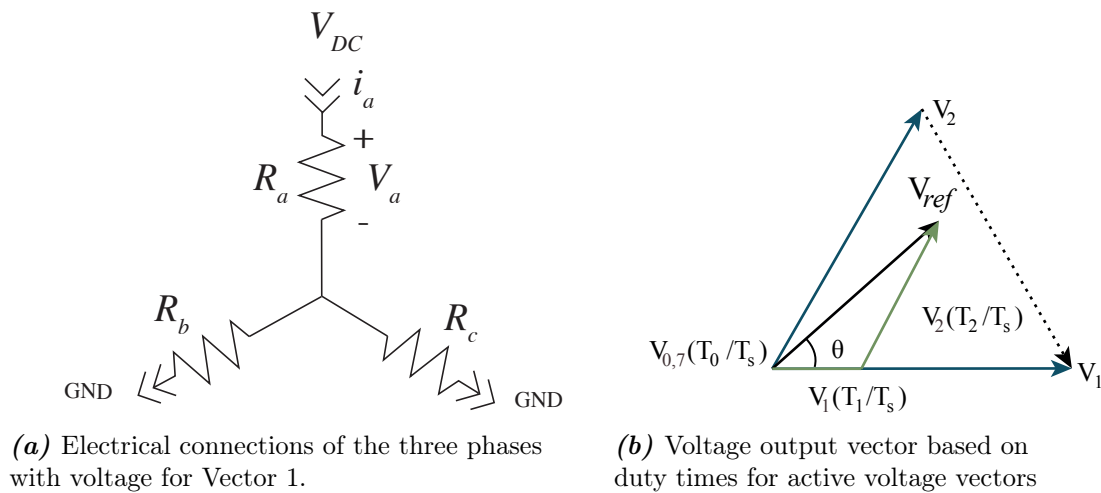


Figure 6.1

The reference vector can be described as a linear combination of the nearest two voltage vectors as seen in Equation (6.4), where x is the sector number. For sector 6 the second vector V_{x+1} is

vector 1 since a full rotation has been made. The linear combination is illustrated in Figure 6.1b for a reference vector in sector 1.

$$\mathbf{V}_{ref} = \frac{1}{T_S} (T_1 \cdot \mathbf{V}_x + T_2 \cdot \mathbf{V}_{x+1}) \quad (6.4)$$

Here T_S is half the time of the symmetrical PWM waveform. Since there has to be a specified voltage vector at all times the zero-vectors are used to fill in the time whenever the reference voltage is below the maximum of $V_{DC}/\sqrt{3}$, which leads to $T_S = T_0 + T_1 + T_2$.

Equation (6.4) can be written as a matrix-vector product and isolated for T_1 and T_2 , yielding Equation (6.5). Here T_1 is the time the first vector in the sector is activated and T_2 is the time the second vector is activated going counterclockwise.

$$\mathbf{V}_{ref} = \frac{1}{T_S} \cdot [\mathbf{V}_x \quad \mathbf{V}_{x+1}] \begin{bmatrix} T_1 \\ T_2 \end{bmatrix} \Rightarrow \begin{bmatrix} T_1 \\ T_2 \end{bmatrix} = [\mathbf{V}_x \quad \mathbf{V}_{x+1}]^{-1} \mathbf{V}_{ref} \cdot T_S \quad (6.5)$$

The transistor states for the 6 active voltage vectors can be seen in Figure 6.2 along the voltage vectors' location in the d - q coordinate system. The circle in Figure 6.2 shows the maximum voltage vector that can be produced without overmodulation.

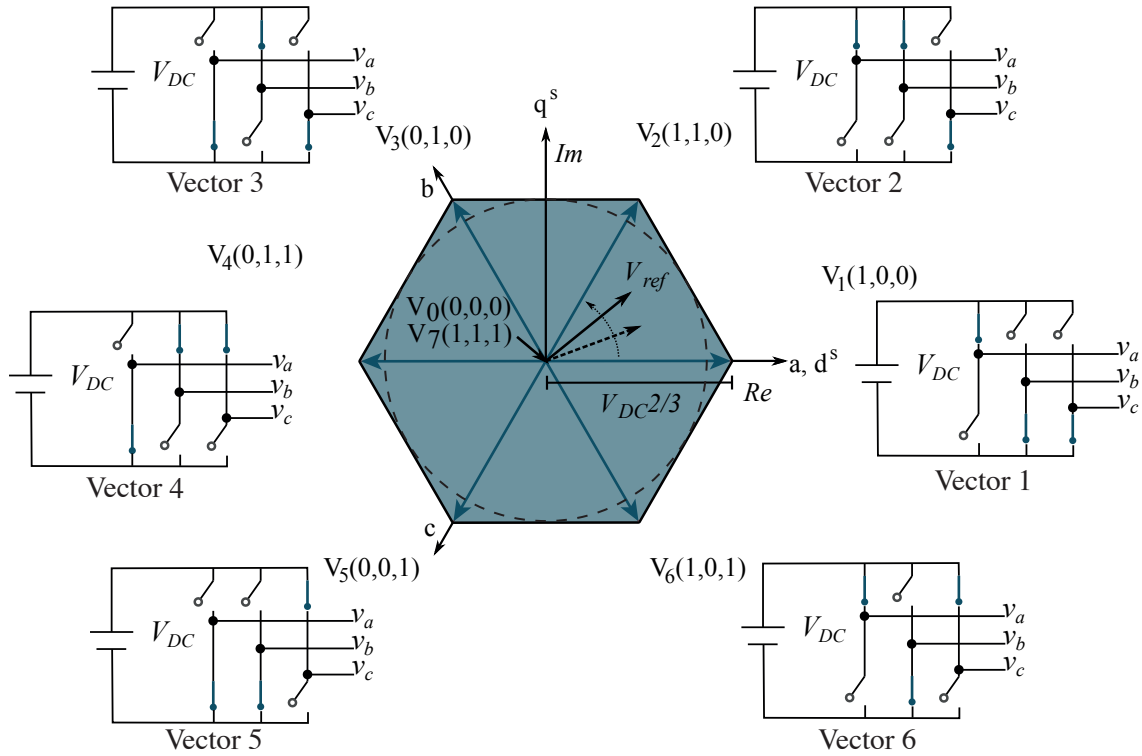


Figure 6.2. Schematic of SVPWM switching states.

The amplitude of this voltage vector is $V_{DC}/\sqrt{3}$. To reduce the switching losses symmetrical SVPWM is utilized, where only one transistor is changed at a time thereby reducing switching losses. To do this, the switching has to be done in the correct order for each sector. The switching order for the 6 sectors can be seen in Figure 6.3.

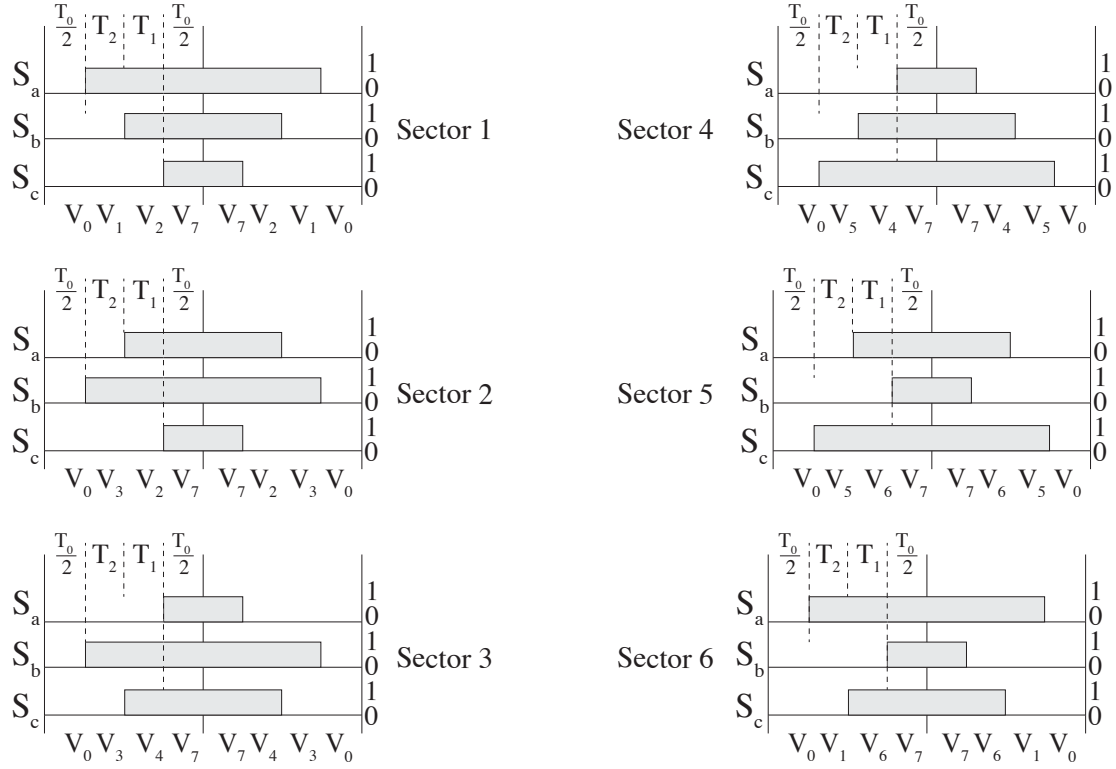


Figure 6.3. Schematic of the symmetrical PWM waveform in each sector.

Controller Design 7

The models are converted into the S-plane to aid in the design of controllers for the system, first as a MIMO system and then converted into a SISO system.

This leads to controllers being designed based on the SISO system, where FOC is implemented to achieve the maximum torque, which is when the angle between the rotor and stator magnetic flux is at 90° .

7.1 S-plane model

The same models can be formulated in the s-plane for time-invariant systems with the needed dynamic equations established.

7.1.1 Multiple-Input Multiple-Output

As the electric AC machine is a multiple input, multiple output (MIMO) system a state-space model is developed based on the d - q axis voltage equations and the mechanical system, which gives the state-space model as seen in Equation (7.2). Here the non-linear term of the voltage equations in Equations (5.8) and (5.9) which are $(\omega_r \cdot PP \cdot i_q \cdot L_q)/L_d$ and $(\omega_r \cdot PP \cdot i_d \cdot L_d)/L_q$ is linearised around a constant speed ω_{r0} . A constant speed is chosen over a constant current since the current dynamics are faster and therefore more important.

To linearise the mechanical model, shown in Equation (5.10), the coulomb friction $\tau_{friction}$ and external load τ_{load} are removed.

$$\dot{x} = \mathbf{A}x + \mathbf{B}u \quad ; \quad y = \mathbf{C}x + \mathbf{D}u \quad (7.1)$$

$\Updownarrow$

$$\begin{bmatrix} \dot{i}_d \\ \dot{i}_q \\ \dot{\omega}_r \end{bmatrix} = \begin{bmatrix} -\frac{R_s}{L_d} & \frac{\omega_{r0} \cdot PP \cdot L_q}{L_d} & 0 \\ -\frac{\omega_{r0} \cdot PP \cdot L_d}{L_q} & -\frac{R_s}{L_q} & \frac{-PP \cdot \phi_f}{L_q} \\ 0 & (\frac{3P}{4} \phi_f)/J & -\frac{B}{J} \end{bmatrix} \begin{bmatrix} i_d \\ i_q \\ \omega_r \end{bmatrix} + \begin{bmatrix} \frac{1}{L_d} & 0 \\ 0 & \frac{1}{L_q} \\ 0 & 0 \end{bmatrix} \begin{bmatrix} V_d \\ V_q \end{bmatrix} \quad ; \quad \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} i_d \\ i_q \\ \omega \end{bmatrix} \quad (7.2)$$

Feedforward matrix

The inputs to the system are the d - q axis voltages and the states that are desired to control are the d - q axis currents as well as the velocity of the motor. The state-space model, in Equation (7.2) is a minimal system that can be modified so that position is a state as well, however, position does not change the dynamics of the equations and can be found by integrating the velocity state.

With this model, several different analyses can be conducted including responses of the linear system that can be compared to the nonlinear simulations.

7.1.2 Linear model validation

With the non-linear models verified, the same equations but in a linear form are run through the same test to check the performance after losing model certainty when linearizing. This is done because the linear equations will be used to controller design later on, and therefore the linear system should behave like the non-linear model and test data even when information is lost due to linearization. In practice the state-space open loop transfer function is stepped with the same as the non-linear simulations.

Another parameter to tune is the linearization constants which come from products of variables when linearizing. It is chosen to linearise about a speed as the current system is assumed faster and will therefore be more critical when doing linear analysis of the systems. The tuned linearization constant for speed is $45 \frac{\text{rad}}{\text{s}}$ and gives the open loop currents as seen in Figure 7.1 which shows the same response as the non-linear simulation.

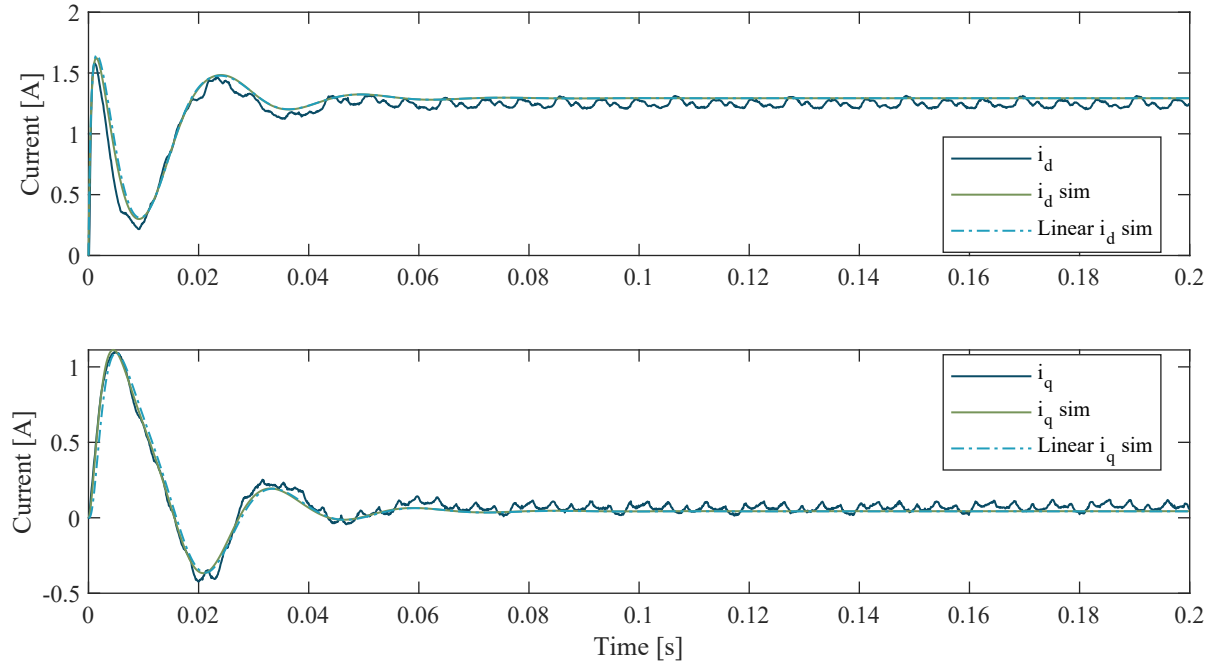


Figure 7.1. Linear current validation

Then the closed current loop test is run on the state-space model which shows the following response in Figure 7.2. Here some variations from the non-linear systems is observed. Still, the steady-state values are the same with the overall dynamics of the linear response being much faster which is reasonable as friction is non-linear and therefore not included when linearizing and the linearization speed also gives unnatural behaviours. Still, overall it is determined that the linear model approximates the physical system and linear control tools can be utilized in combination with non-linear simulations.

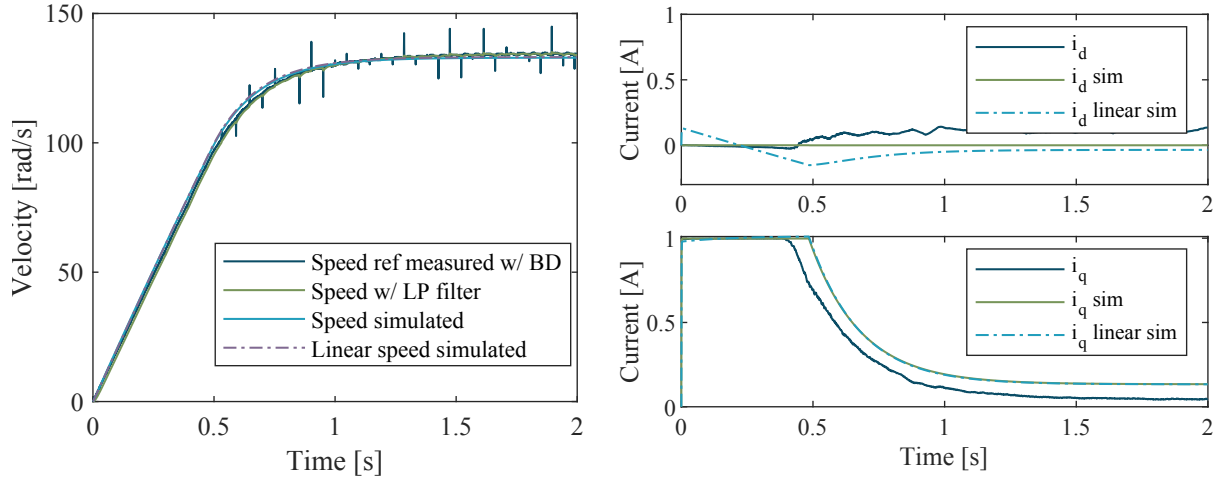


Figure 7.2. Plots of left) Linear speed output of q-axis current unit step and right) linear closed-loop current from step

Transfer function matrix

From the state space model, transfer functions can be obtained relating each input to each output. The formulas for obtaining the open loop transfer function from the Laplace transformed state space model are presented in Equation (7.3).

$$\begin{aligned}
 \dot{x}(t) &= \mathbf{A}x(t) + \mathbf{B}u(t) \quad ; \quad y(t) = \mathbf{C}x(t) + \mathbf{D}u(t) \\
 sX(s) &= \mathbf{A}X(s) + \mathbf{B}U(s) \quad ; \quad Y(s) = \mathbf{C}X(s) + \mathbf{D}U(s) \\
 (s\mathbf{I} - \mathbf{A})X(s) &= \mathbf{B}U(s) \quad ; \quad Y(s) = \mathbf{C}X(s) + \mathbf{D}U(s) \\
 X(s) &= (s\mathbf{I} - \mathbf{A})^{-1}\mathbf{B}U(s) \quad ; \quad Y(s) = \mathbf{C}X(s) + \mathbf{D}U(s)
 \end{aligned} \tag{7.3}$$

Combining the last two lines in Equation (7.3) while setting $\mathbf{D} = 0$, since there is no feedthrough, yields Equation (7.4).

$$\mathbf{G}_p = \mathbf{C}(s\mathbf{I} - \mathbf{A})^{-1}\mathbf{B} + \mathbf{D} \quad ; \quad Y(s) = \mathbf{G}_p(s)U(s) \tag{7.4}$$

Here $\mathbf{G}_p$ is the transfer function matrix.

Relative Gain Array

The Relative Gain Array (RGA) analysis is done to determine how coupled a multivariable system is and in what frequency range the coupling is most significant. Here the inverse of the transfer function matrix is needed, which can only be done for a square matrix. The RGA analysis is done with the output matrix $\mathbf{C}$ where the i_d and i_q outputs are investigated. This is done since the electrical dynamics are significantly faster than the mechanical dynamics. RGA values are found from the system plant as shown in Equation (7.5).

$$H = RGA(\mathbf{G}_p) = \mathbf{G}_p \times (\mathbf{G}_p^{-1})^T = \begin{bmatrix} \lambda_{11} & \lambda_{12} \\ \lambda_{21} & \lambda_{22} \end{bmatrix} = \begin{bmatrix} \lambda_{11} & 1 - \lambda_{11} \\ 1 - \lambda_{11} & \lambda_{11} \end{bmatrix} \quad ; \quad \lambda_{11} = \frac{1}{1 - \frac{G_{12}G_{21}}{G_{11}G_{22}}} \tag{7.5}$$

Here $\times$ denotes the elementwise multiplication and the H matrix entries are the relative gains between input-output pairs that give information about the coupling at different frequencies

viewed as SISO systems. The λ_{ij} is found from the ratio of input/output pair gain in the open loop over input/output pair gain in the closed loop. G_{ij} indicates the matrix indices for the matrix G . The open loop is achieved when all other inputs are kept at zero, and the closed loop is achieved when all outputs are zero meaning a perfectly controlled system. The rows (and columns) of H will always sum to 1 i.e. at frequencies where $\lambda_{11} \neq 1$, meaning there exists some coupling.

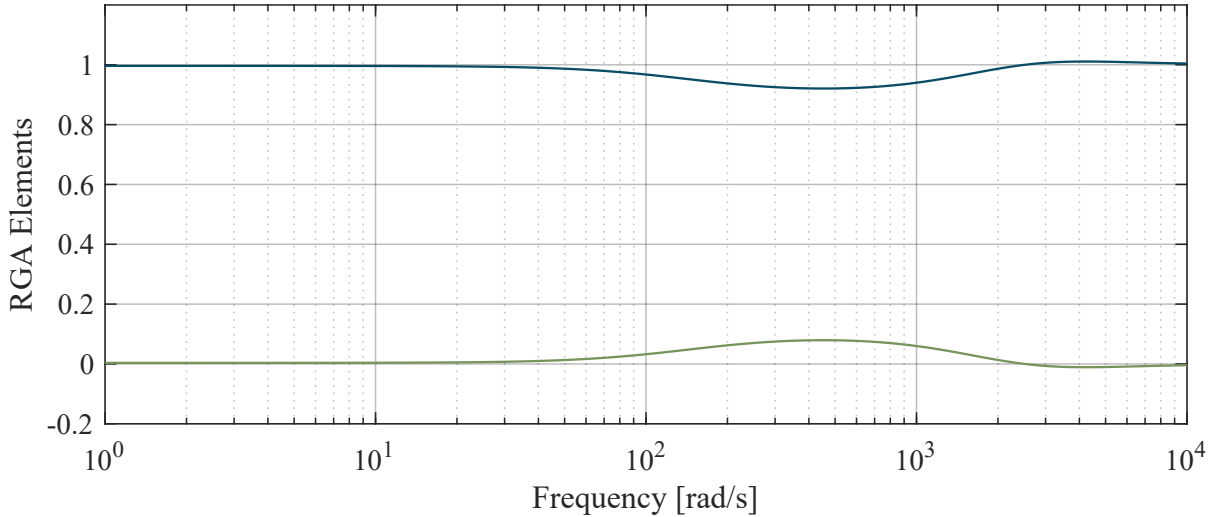


Figure 7.3. RGA analysis plot

As it is seen in Figure 7.3, the elements of the RGA matrix only have a slight deviation from 1 and 0 throughout the frequency span meaning that there exists very little to almost no coupling between the different input/output pairings in the linear state space model. Through further investigation, it was found that the linearisation speed has a significant impact on the coupling according to the RGA analysis where a higher speed means more coupling. The RGA analysis is therefore done at 100rad/s since that is at the top end of the motor's operating speed.

With the system only being slightly coupled it is regarded as two SISO-systems initially. It is therefore assumed that only the d -axis voltage affects the d -axis current while the rest is a disturbance.

7.1.3 Single-Input Single-Output System

With the coupling of the system determined to be minimal, the system is simplified to two SISO systems where classical control can be used. The system consists of four different control loops as seen in Figure 7.4.

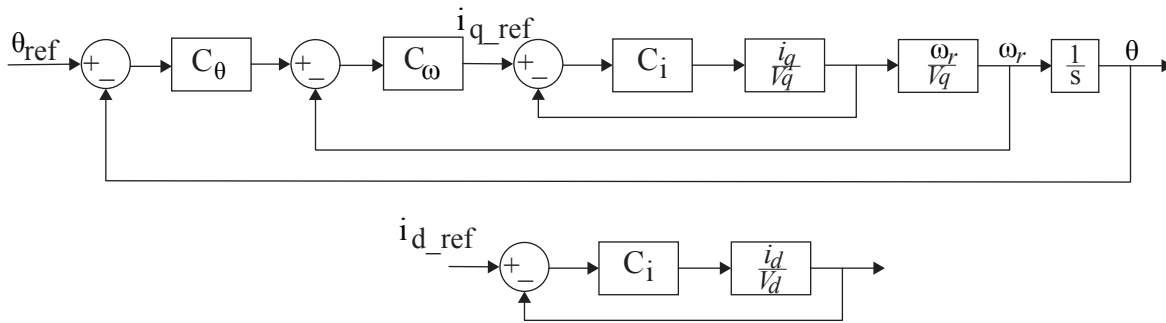


Figure 7.4. Overall control structure with position, speed, and current controllers.

For this structure, SISO controllers will be designed for the transfer function of the two plants $\frac{i_q}{V_q}$ and $\frac{\omega_r}{V_q}$.

Electric system

The electrical system consists of a transfer function for the d - and q -axis systems respectively which can be seen in Equation (7.6).

$$\frac{i_d}{V_d} = \frac{1/R_s}{s \cdot L_d/R_s + 1} \quad ; \quad \frac{i_q}{V_a} = \frac{1/R_s}{s \cdot L_q/R_s + 1} \quad (7.6)$$

Since $L_d = L_q$ for a PMSM with surface-mounted magnets, the two transfer functions are identical first-order systems with a time-constant of $\tau_e = L_d/R_s$

A PI controller is the obvious choice to control these systems since it would remove the steady-state error that would otherwise be seen due to disturbances. For FOC of a surface-mounted PMSM, the d -axis current has no impact on the developed torque so the reference value is zero and the current loop is closed to this reference not regarding any position/velocity control.

Mechanical system

The mechanical transfer function, in Equation (7.7) is acquired from the state-space model in Equation (7.2).

$$\frac{\omega}{i_a} = \frac{3P \cdot \phi_f}{4(s \cdot J + B)} = \frac{3P \cdot \phi_f / 4B}{s \cdot J/B + 1} \quad (7.7)$$

This is a first-order system with a time-constant of $\tau_m = J/B$. Again a PI-controller is the obvious choice to control this system.

7.2 Controller design

With the SISO transfer functions determined classical control theory can be applied to design the controllers to fulfill the requirements. This is done using loop shaping to achieve the desired response in the frequency domain. The designed controllers are verified throughout the section to see that the desired response is achieved.

With the SISO controllers designed the effect of using full-state feedback to decouple the system is investigated. This leads to the controllers being discretized and anti-windup gains determined for the two PI controllers.

7.2.1 Controller requirements

The controller considerations in Section 2.6 need to be converted to some quantifiable controller requirements. Here it is stated that the speed and position controller should show minimal oscillations. This is quantified through low allowable overshoot, but also by how many times the overshoot is allowed to surpass the reference value back and forth which is similar to defining a settling time but as no rise time is defined a fitting settling time is hard to determine. Large oscillations in current control will propagate to oscillations in developed torque and hence velocity and position. Most importantly, no oscillation is allowed on position control. Speed and current control can allow small oscillations which surpass the reference value a limited amount of times. Excessive oscillations will cause jerking motion and affect user experience. For all the controllers it is required that minimal steady-state error is allowed as this would impact the usability of the motor. In no load, and therefore no disturbance, conditions the steady-state error is expected to be removed completely. This is quantified to the following controlled system requirements in Table 7.1.

	Overshoot	n reference surpassing
Position control	0%	0
Speed control	2.5%	1
Current control	5%	1
	Steady-state error	Rise- and settling time
All controllers	minimal	minimal

Table 7.1. Table of controller requirements. n references is number of allowed times for signal to surpass reference value.

Specified bandwidths are not defined here as this will be highly dependent on the different plants and if not tuned correctly will have a high influence on other systems also causing jerking motion where e.g. position is tuned too aggressively giving a high initial velocity. When the speed then exceeds its reference value a negative controller input is created causing undesired oscillations of the position during the transient period of the response.

7.2.2 Loop shaping

With linear transfer functions for the systems established, loop shaping is done utilizing frequency response of the open loop transfer functions to manipulate the system response to fulfill the requirements.

Cascade control

As seen in Figure 7.4 the q -axis system is composed of multiple feedback loops. If cascade control is to be utilized the inner loop is required to be ten times faster than the outer loop so that its transient response can be considered negligible. This means that the bandwidth of the inner closed loop should be ten times higher than the bandwidth of the outer loop. If it is not possible to make the inner loop ten times faster the closed loop dynamics of it has to be taken into account when designing the controller for the outer loop.

For the velocity controller, the inner loop is the current controller and for the speed controller the combined velocity and speed controller is the inner loop.

Current controller design

With the transfer function being the same for both the d - and q -axis current the controller designed here will be applied on both. The frequency response of the electric model is shown in Figure 7.5 and the transfer function can be seen in Equation (7.8).

$$\frac{1}{L_{qs} \cdot s + R_s} \leftrightarrow \frac{1}{0.0012 \cdot s + 2.865} \quad (7.8)$$

This is a first-order system with a break frequency of about 2000 rad/s which is desirably fast. The undesired characteristics of this transfer function are that as a first-order system, it is subject to steady-state error as there is a DC gain of $1/2.865$, and at the low-frequency range of the bode diagram, seen in Figure 7.5, there is a gain below 1. A high gain at low-frequency is desirable for disturbance rejection.

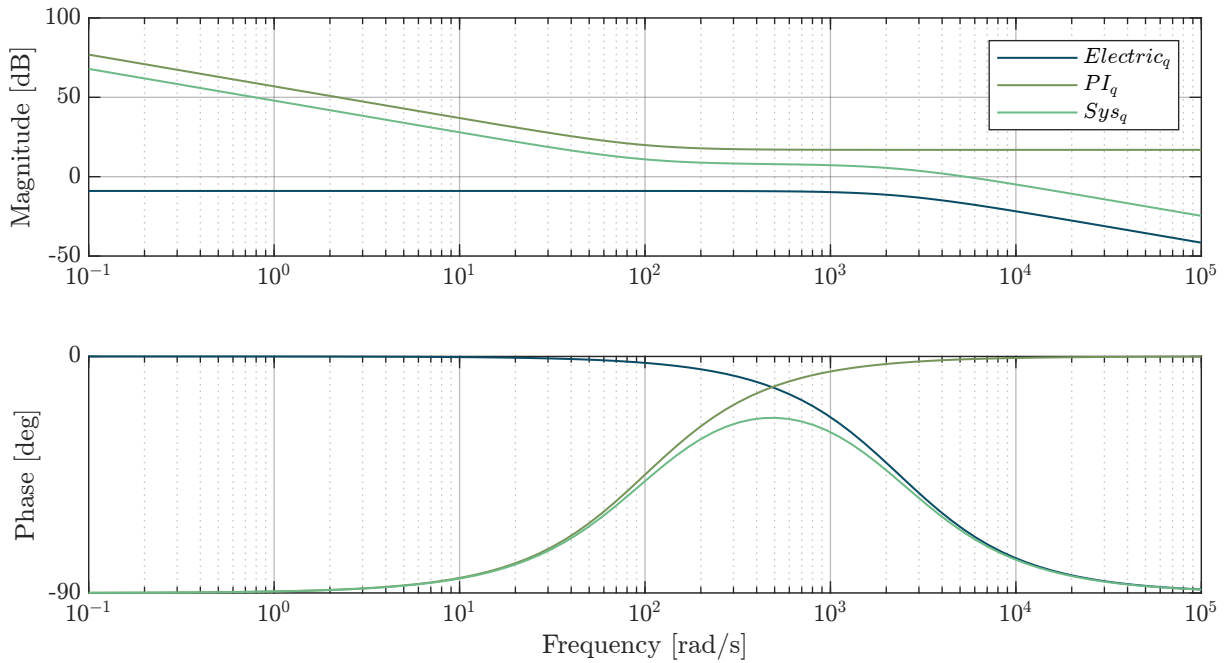


Figure 7.5. Bode diagram of the q -axis electric model

Using a PI-controller a free integrator is placed, changing the magnitude slope from 0 to $-20 \frac{\text{dB}}{\text{dec}}$ until the break frequency of the controller integrator term adding low-frequency gain and increasing the system order to eliminate steady-state error. The general structure of the PI control is shown in Equation (7.9).

$$PI = K_p + \frac{K_i}{s} \leftrightarrow PI = \frac{K_p \cdot s + K_i}{s} \leftrightarrow K_i \cdot \frac{s \cdot K_p/K_i + 1}{s} \quad (7.9)$$

Following the literature (Kim 2017, pp. 255-264), the initial procedure for PI-controller design of DC quantities in the synchronous frame is to determine a desired control bandwidth ω_c which is used to determine the values in Equation (7.10).

$$K_p = \omega_c \cdot L_{dq} ; K_i = \omega_c \cdot R_s \quad (7.10)$$

This will give a PI-controller with a time constant equal to that of the electric first-order system. By combining the electrical plant and the PI controller the open-loop system will look like a free integrator with a gain to raise the bandwidth to that of ω_c . As this method did not give the desired transient response as the bandwidth frequency was chosen arbitrarily, the control is further tuned with the same loop shape in mind.

Generally, the integral gain will be much higher than the proportional gain as electric resistance is higher than inductance. In practice, it is discovered that the high integral gain gives integrator windup problems, why a higher proportional gain and lower integral gain is desired compared to what the analytical tuning dictates. This decreases the break frequency of the PI controller. This change was made as the analytical tuning had overshoot due to windup because of incorrect tuning of the anti-windup. This deviates from the analytical approach but gives a more desirable current response when simulating. The final current control values are presented in Equation (7.11).

$$K_p = 7 ; K_i = 700 \quad (7.11)$$

Underneath in Figure 7.6 the controlled electric system is stepped. The left plot is closed-loop current controller output in voltage and the right is closed-loop electric plant output in ampere. The transfer functions are $\frac{C}{1+C \cdot P}$ and $\frac{C \cdot P}{1+C \cdot P}$ respectively with C being the controller and P the plant where the green plots have unity control of gain one. The electric systems are stepped with 2.85 A

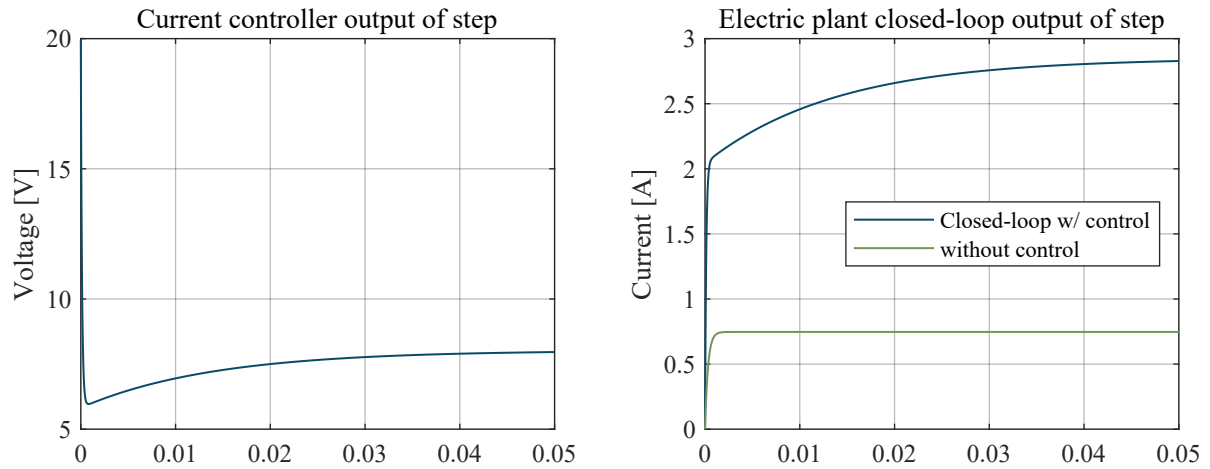


Figure 7.6. Electric system control signal and plant output current

With voltage in the system set to saturate at the limit of $V_{DV}/\sqrt{3} = 9.32$ V it is evident that the control signal will saturate if giving high input values. This is not bad as the plant output shows stable dynamics so the controllers use the whole range of allowable voltages. From the right plot, it is clear that the controlled system has improved performance as steady-state errors are corrected. However, the different controller gains are visible. Initially, a high proportional gain gives a high instant response after which the integrator starts correcting the remaining error as it is accumulated. With the largest possible step done within the physical limits of the system, some saturation is not a problem. This means that a faster response can be expected for smaller inputs to the controller which is desirable.

Speed controller design

With a motor utilized for multiple different applications, a choice has to be made regarding which moment of inertia the speed controller is tuned for. When the moment of inertia is large, the time constant increases, lowering the bandwidth compared to the system without the added inertia. This can be seen in the frequency response in Figure 7.7.

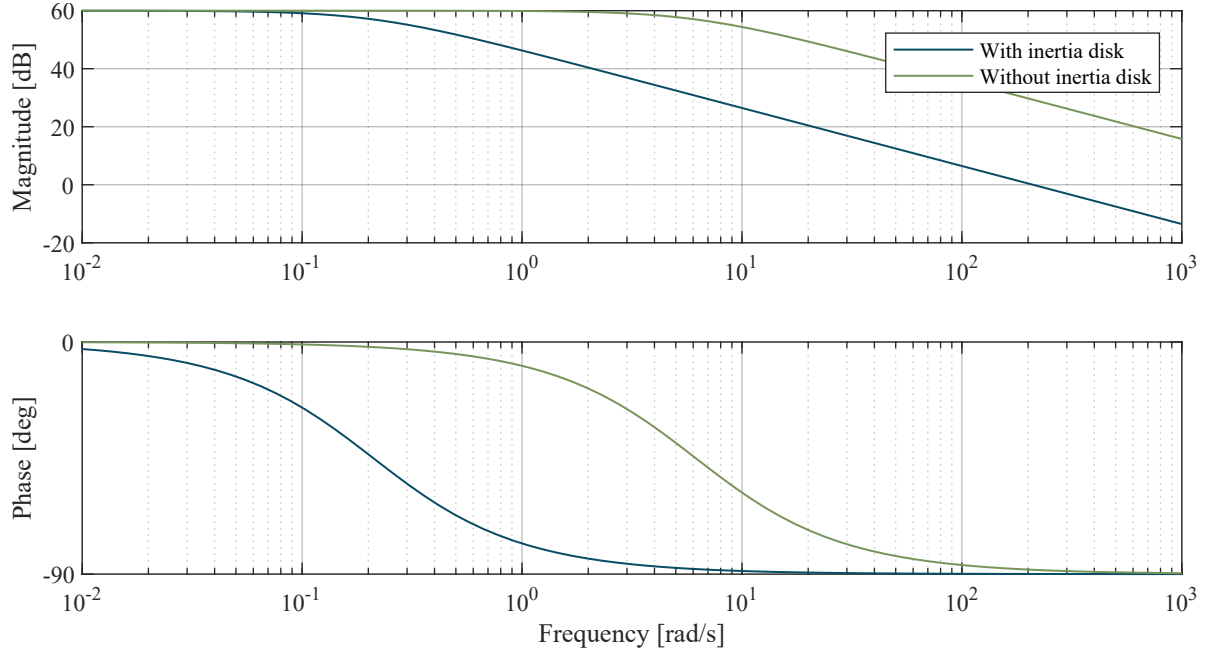


Figure 7.7. Frequency response of the mechanical systems with and without inertia disk

When tuning a PI controller for a first-order system you would normally place the PI controller cut-off frequency at the cut-off frequency of the first-order system. If this is done for the mechanical system without the inertia disk an under-dampened system would be created by adding more inertia as the combined phase lag of the system and controller would go towards -180° . This leads to an undesired oscillatory response.

Tuning the PI controller for the system with the maximum inertia removes this problem giving an over-dampened response when inertia is removed. For this reason, when only a single controller is used for a range of products, it is better to tune the controller for the highest expected moment of inertia than for the lowest. Test of this have been made and is seen in Appendix G where either undesired overshoot and oscillations or a slow responding system is seen for the speed plots.

The mechanical transfer function can be seen in Equation (7.12) where the moment of inertia with the inertia disk is used.

$$\frac{3 \cdot PP \cdot \phi_f}{4(J \cdot s + B)} \leftrightarrow \frac{0.4}{6.5 \cdot 10^{-5} \cdot s + 0.000212} \quad (7.12)$$

This is also a first-order system and therefore has the same undesired characteristics as the electric system which is why a PI controller is used.

The time constant of the PI controller is placed at the same place as the time constant for the mechanical transfer function to make the combined system mimic a free integrator with a gain.

This can be seen in Figure 7.8 Applying this controller to the system without the inertia disk will add some phase above -90° . Henceforth simulations will be done with the inertia including the inertia disk unless stated otherwise. (7.13).

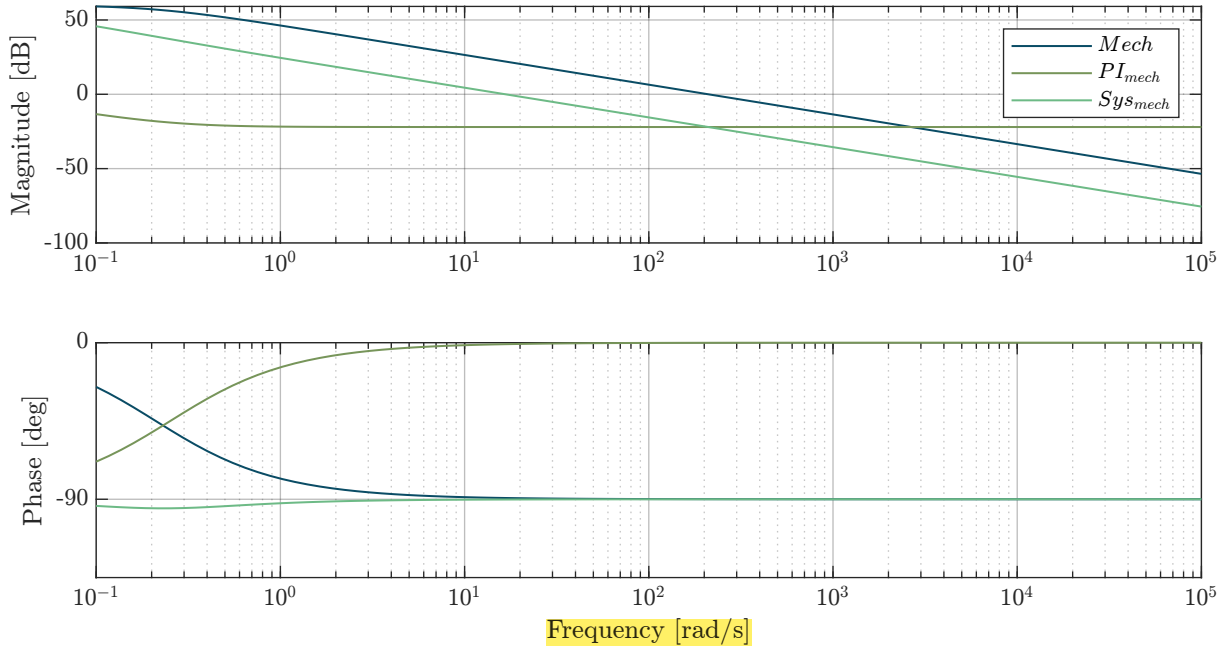


Figure 7.8. Bode diagram of the motor mechanical model

After tuning the PI-controller for the speed the gains are as shown in Equation

$$K_p = 0.0792 ; K_i = 0.02 \quad (7.13)$$

With the mechanical controller determined a step input is tested in Figure 7.9. The left plot is closed-loop controller outputs, whereas the right plot is closed-loop plant outputs with and without control. The speed is stepped with $100 \frac{\text{rad}}{\text{s}}$.

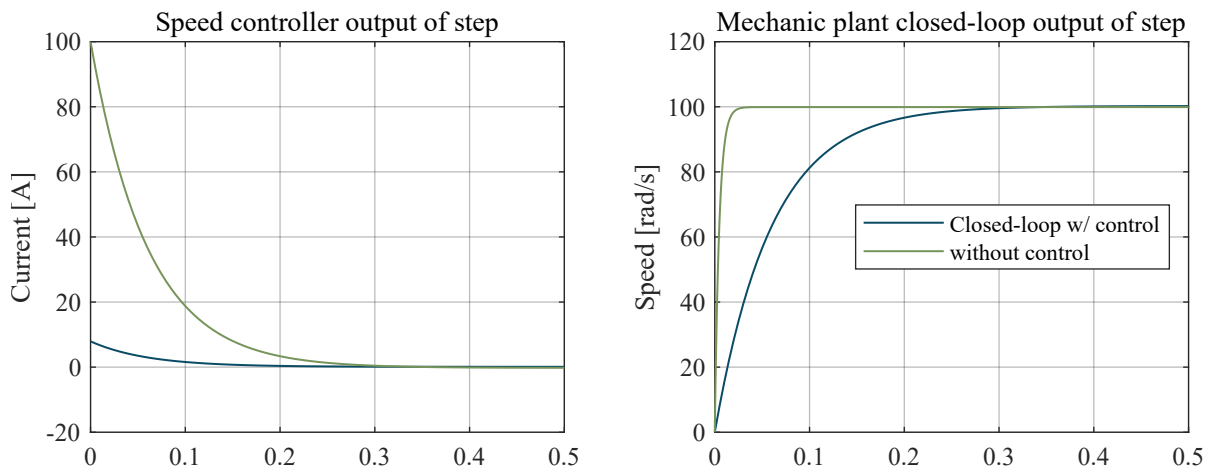


Figure 7.9. Mechanical system control signal and plant output speed

The current saturation is set to 2.85 A, due to limitations in the inverter board. This means that the current output of the controller will saturate with the designed controller with a step

of $100 \frac{\text{rad}}{\text{s}}$ as seen in Figure 7.9. This could be removed by lowering the gain of the controller to avoid saturation. This would however lead to a significantly slower response when small steps are applied which is not desirable. This amount of saturation is therefore deemed acceptable. The control output without control is also plotted showing a current of about 100 A which will saturate a lot which means that the shown output response is not achievable. The controller is therefore applied.

From the left plot in Figure 7.9 it can be seen that closed-loop system with controls reaches the desired value without any steady state error since the system has a free integrator. Furthermore, it can be seen that the system has no overshoot as desired.

Validation of current and speed controller

The step responses shows the linear response to inputs which is a simplification of the actual system. The response of the non-linear system is compared with the linear system in Figure 7.10. Here a step of $100 \frac{\text{rad}}{\text{s}}$ is applied to both the linear and non-linear models showcasing the non-linearity of saturation limits, frictions and coupling in the system. Until now linear step responses is shown for linear transfer functions. The non-linear simulations is combining linear transfer functions with the non-linear plants to showcase the combined effects. From the current plot, it is seen that the step exceeds saturation instantly, making the stepped voltage spike to above saturation and give a speed output far faster than the simulated response. The saturation stops the controller output from giving too high values to the next part of the system, meaning that smaller current reference is input to the current controller which overall makes a slower system. The overshoot seen in the simulation comes from integrator-windup of the current controller. This can be seen from the voltage being saturated from 0.2 to 0.3 seconds while the measured current is higher than the reference value.

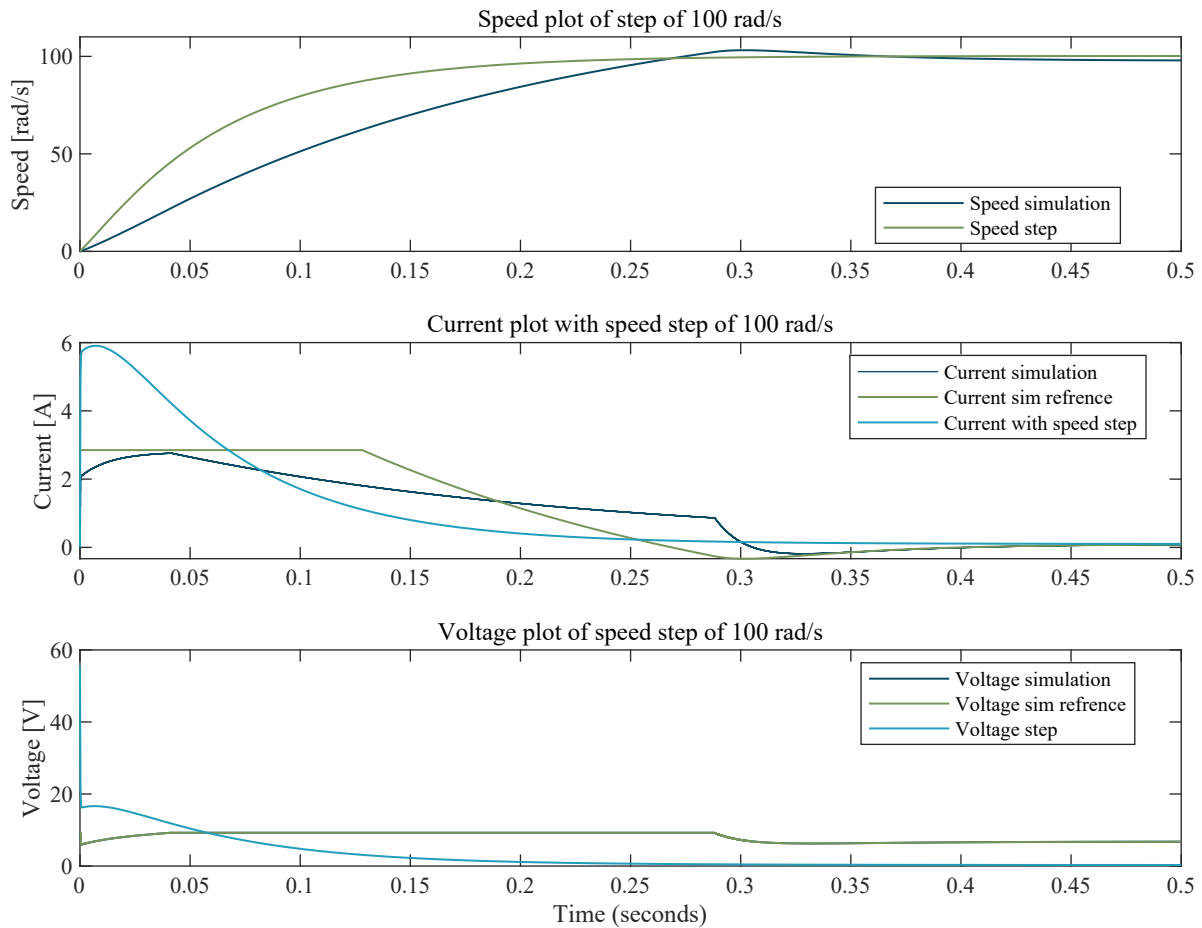


Figure 7.10. Comparison of simulated and stepped response

Position control

A desired option for the usability of the motor is to control the angular position instead of speed. The transfer function for the position is a free integrator as the speed is already a system state. A pure gain control is used as no steady-state error should be present. The limitation here is how much gain can be added before the system response is compromised due to cascade control not being applicable. In Figure 7.11 the bode diagram of the integrator as well as the closed-loop current and speed controller. Here the inertia of the motor without the inertia disk is used.

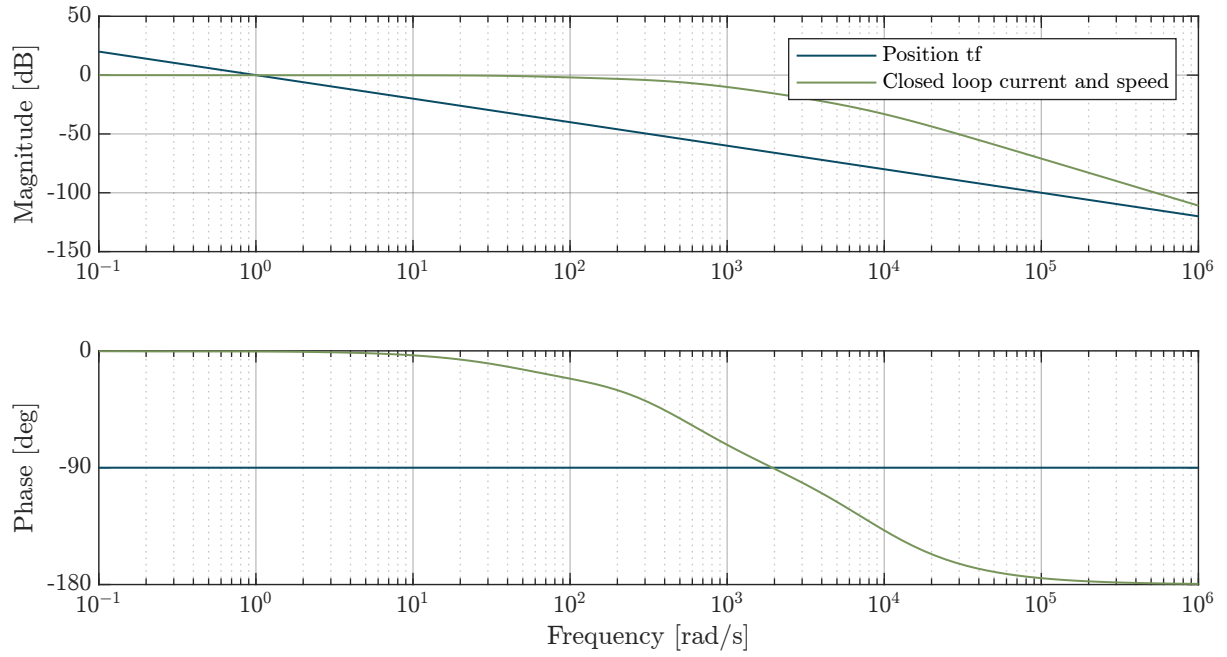


Figure 7.11. Bode diagram of the position transfer function and closed-loop transfer function of electric and speed systems

The approximated bandwidth of the closed-loop current-speed system is $180 \frac{\text{rad}}{\text{s}}$ meaning a gain of up to 18 can be used for position control of the system without the inertia disk. If the inertia disk is on, the allowed gain is lowered to 2. Gains higher than this will cause some oscillation in the system.

7.2.3 Decoupling

With the RGA analysis in 7.1.1 showing slight coupling in the system, especially at high linearization speeds the implementation of decoupling is investigated. Decoupling consists of taking the coupled current and velocity from the electric equations and adding these components to the control voltage signal. The decoupling terms are as shown in Equation (7.14).

$$V_d^* = -\omega_e \cdot L_q \cdot i_q \quad ; \quad V_q^* = \omega_e (L_d \cdot i_d + \phi_f) \quad (7.14)$$

The decoupling here is done with full-state feedback since all states are measured. This means that the state values from the previous iteration are used to improve the control signal. In the ideal world, this would completely eliminate the coupled part from which leads to the system being the same as the first-order system in Equation (7.8). The difference is that the coupling here is removed using full-state feedback instead of treating it as a disturbance. This leads to the same PI-controller being used with and without decoupling.

The effect of decoupling

To investigate the effect of applying decoupling using full-state feedback a step-responses is done to $50 \frac{\text{rad}}{\text{s}}$ and $100 \frac{\text{rad}}{\text{s}}$ respectively. The step is done at a relatively high speed since the coupling is linearly dependent on the velocity. Figure 7.12 shows the speed and current response with

and without decoupling. From this, it can be seen that the decoupling does not affect either the current or position response. This was also somewhat expected since there was only a small coupling in the RGA analysis. If the motor was operating at higher speeds the coupling would have been more significant. Due to this decoupling will not be applied to any further controllers.

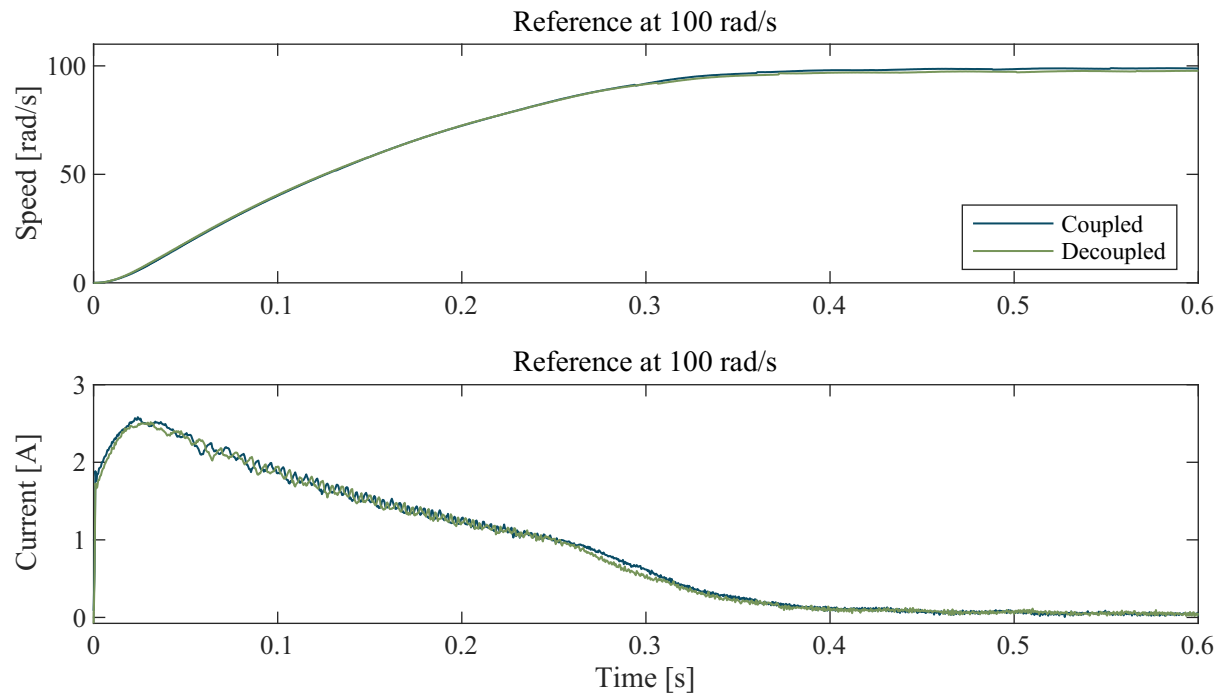


Figure 7.12. Comparison of Coupled and Decoupled system.

7.2.4 Discretization and anti-windup

The controllers have to be discretized to be implemented on the microprocessor. This is done using backward difference and the block diagram of this can be seen in Figure 7.13. Here anti-integrator windup is also implemented, since both the voltage and the current, which are controlled, have physical limitations in the setup. To not exceed them, saturations are implemented, however, for controllers with an integral part, this can lead to integrator windup where the controller output keeps increasing due to physical limits in the controlled value.

The anti-integrator windup feedback is scaled with a constant k_{AW} which can be tuned to get the desired characteristic. A low value provides a small feedback and thereby only decreases the windup slightly. A too-large value will lead to spiky oscillations where the system will oscillate between being saturated and non-saturated. The anti-windup gain should be below the integrator gain K_i . (Karl J. Åström 1995, pp. 83–86)

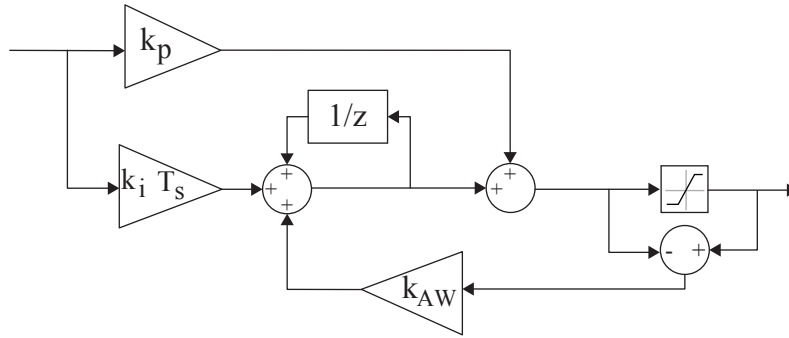


Figure 7.13. Block diagram of the discrete controller with anti-integrator windup implemented.

The anti-windup gains for the two controllers are found iteratively by taking offset in the K_i values for the controllers and then tuned from there to achieve a low windup while still removing most of the oscillations around the saturation point.

Anti-windup gain

The anti-windup gain affects how much integrator windup is removed with the feedback. For the current controller, it is desired to keep a little windup to reduce the feedthrough of noise. The saturated and non-saturated controller output can be seen in Figure 7.14. Here it can be seen that an anti-windup gain of 0.001 creates a significantly larger windup than the others. Furthermore, it can be seen that an anti-windup gain of 1 leads to noise feedthrough which is not desirable. This leads to an anti-integrator windup $K_{AW} = 0.1$ for the current controllers.

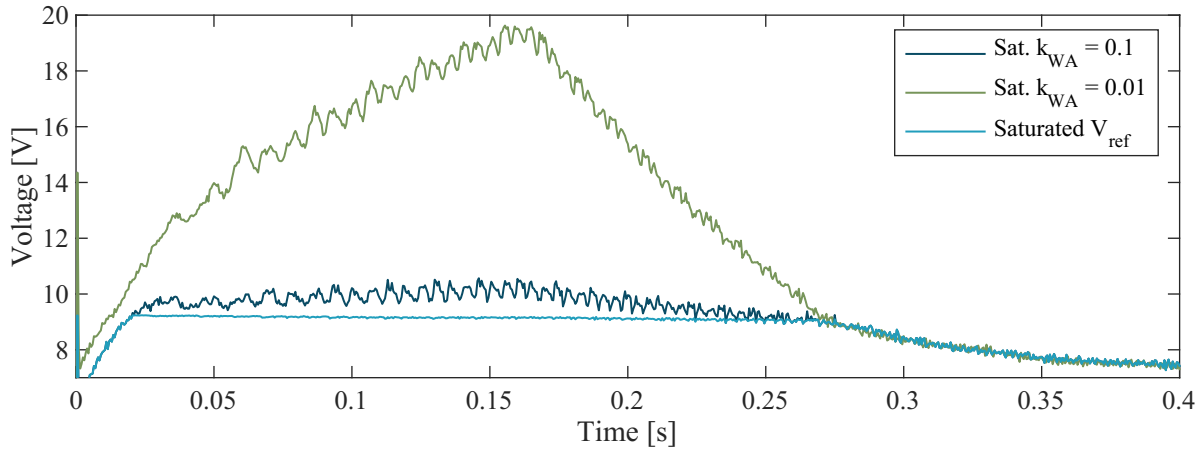


Figure 7.14. Plot showing the effect of changing the anti-windup gain of the current controller during saturation.

The anti-windup gain for the speed controllers has been seen to have a large influence on the transient response of the system especially with the inertia disk installed. With the anti-windup gain applied to the saturation of the entire controller output, it significantly affects how fast the initial spike from the proportional part of the controller is reduced which affects the transient response. This can be seen in Figure 7.15. Due to this an anti-windup gain of 0.001 is chosen to get the anti-windup effect while not significantly disturbing the transient response of the system.

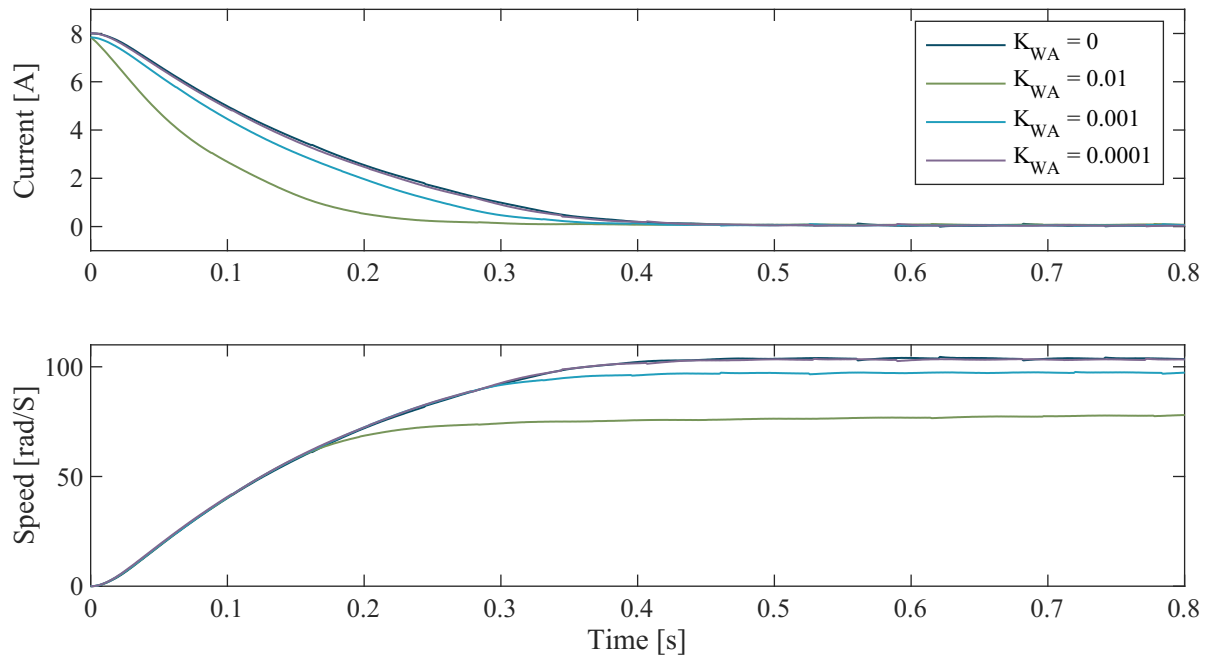


Figure 7.15. Figure showing how the effect of the speed controllers anti-windup gain on the transient response of the system.

Performance testing 8

As described in Chapter 4, two test setups are made, one with a mass on a string for load torque testing and position step response, and an inertia disk for transient testing. The table below describes the tests and their purpose.

Mass on string:		
Efficiency (Speed Ramp)	0.5 kg	A ramp speed test, with a fixed mass applying a load torque to the motor, where current and voltage measurements on the oscilloscope are logged. From this, a performance curve can be produced, showing the efficiency of the motor.
Position Step Response	0.5 kg	By stepping up 40 rad, the gravitational torque acting on the motor hinders the controller in the step response. In this way, the disturbance rejection capability of the position control is tested.
Inertia Disk:		
Speed Step Response	475 kg · mm ²	Two tests are performed to showcase the maximum performance of the motor. Here a step of 50 rad/s and 100 rad/s is used.
Low-Speed Control	475 kg · mm ²	The PMSM is tested at its lowest viable speeds with and without the inertia disk to test the low-speed performance of the control architecture.
PMSM & LEGO motor Comparison:		
Transient Comparison	No load	The transient response of the two motors is compared through a step response in position to investigate the settling time and steady-state error.
Low Speed Comparison	No load	The DC motor is tested to compare the motors at around the lowest viable speed.

8.1 Mass on string testing

Tests are done with a mass on a string attached to a reel on the axle to test load torque use cases.

8.1.1 Speed Ramp Test

The efficiency is calculated as the ratio between output power to input power. Whereas the input current and voltage are measured at the power supply, the output is calculated as the required torque of the motor and measured angular speed. Hence the efficiency represents the total system including switching losses of the half-bridges. This leaves the test as a conservative estimate of the efficiency as electrical losses could be found by phase voltage and current outputs from the inverter, and torque could be measured through a torque transducer instead of having a calculated required torque. As the speed reaches a steady state value with the given load, the viable data is limited to 100 rad/s achieving a peak efficiency of about 55% with this mass. According to the supplier's efficiency curve, the efficiency peaks at about 135 rad/s, so a test with a lower mass, therefore not hitting a steady-state speed should show a higher peak efficiency (CubeMars 2024).

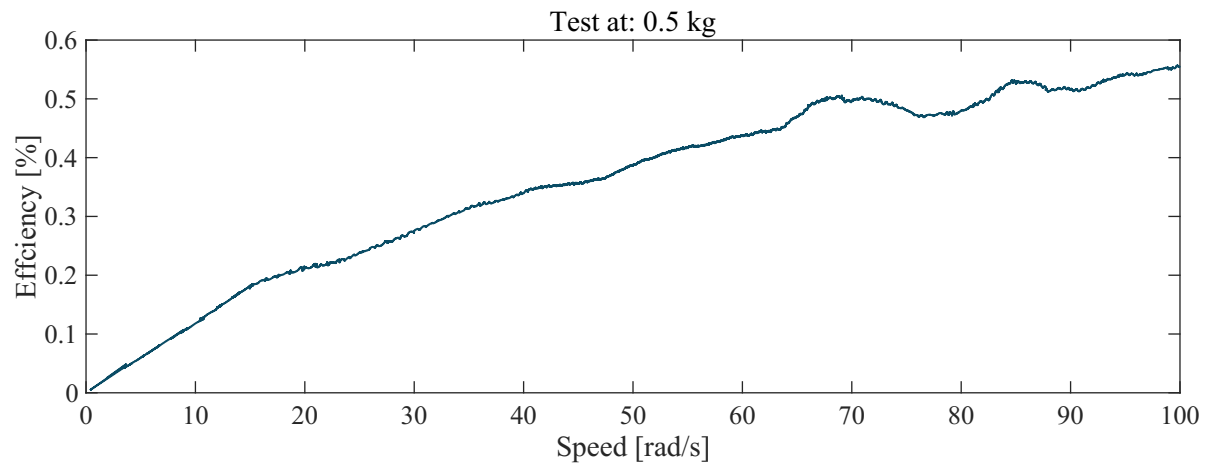


Figure 8.1. Efficiency curve calculated from the output power divided by the input power. 0.5 kg mass lifted from a string following a ramp in speed.

8.1.2 Position Step Response

The position is stepped with a counteracting load torque of 0.049 Nm. As this is a disturbance to the system the plot of the step input in Figure 8.2 does not reach the reference value within the time frame as the controller, for such disturbances, is challenged in reaching the reference value. The position is only controlled with a proportional gain as the free integrator itself should remove any steady-state error. However, when applying the disturbance torque as input disturbance to the mechanical model, this counteracting torque subtracts the total developed torque slowing down the system. Only a slight integrator effect is seen correcting the error, which gives reasoning for trying to implement PI control on the position as well as disturbance rejection as of now is not sufficient for this disturbance.

A disturbance of this magnitude looks good when viewed in real life, but the data shows that the disturbance is too high for this controller, so further tuning or testing with less mass could be conducted to see the capabilities of this controller. As this test does not fulfill the requirement of steady-state error being zero the rest of the performance characteristics is not investigated.

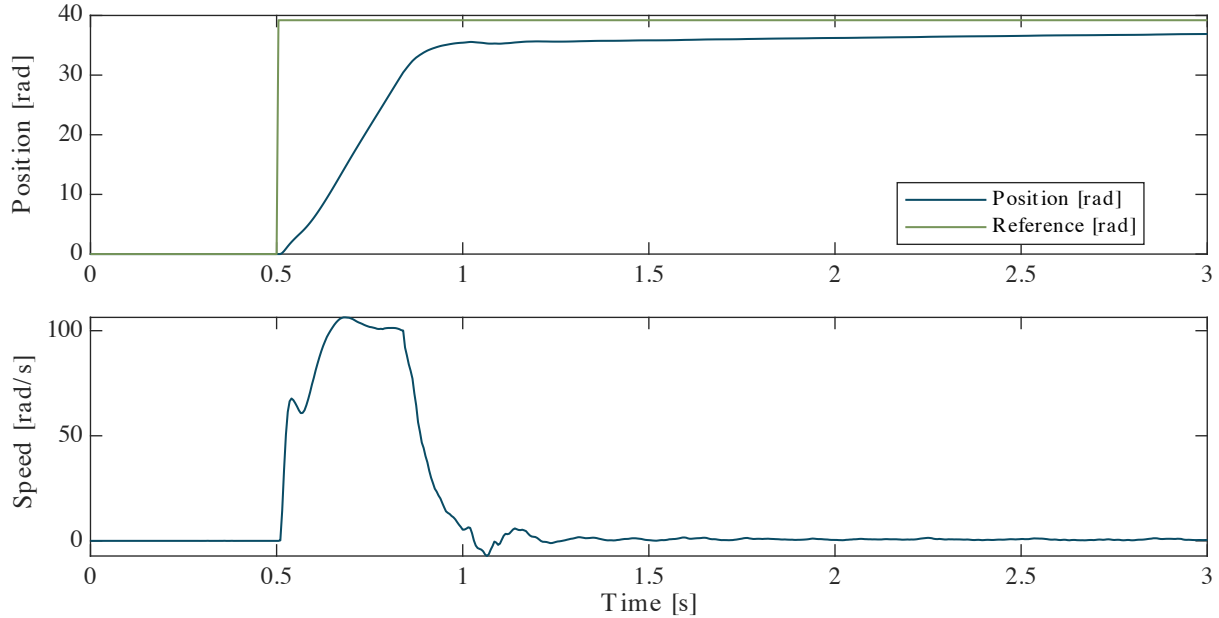


Figure 8.2. Upwards position step with a mass of 0.5 kg corresponding to a 0.049 Nm load torque with the 10 mm radius roller.

8.2 Inertia disk testing

Along with the tests used in the tuning of the speed controller in Section 7.2.2, the inertia disk is used in the following test.

8.2.1 Speed Step Response

The motor with the inertia disk is tested to simulate a stationary system with some mass that has to speed up to a desired reference, overcoming the system start torque. Here a speed reference of 100 rad/s and 50 rad/s is used. From Figure 8.3 it is seen that the 50 rad/s goes to the reference value within 0.25 s whilst the 100 rad/s step does not reach the reference value within 0.5 s due to the current saturation of the motor seen between $t = 0.05$ s to 0.27 s. This is caused by the increased speed leading to a higher back EMF. Therefore the maximum current delivered is inverse proportional to the speed of the motor.

It is also noticed that a small steady-state error is seen from the 100 rad/s speed reference. This is likely caused by high back EMF limiting the drawn current as the current follows the reference. More system analysis and controller tuning could improve understanding and performance of the controller outputs in order to get desired responses.

The 50 rad/s step is complying with all set requirements and it is presumed that steps up until this value will also experience a smooth and adequately fast transient speed response with no

steady-state error. The current for this step overshoots a little but is only surpassing the reference value once with minimal oscillations which might also be due to noisy signals.

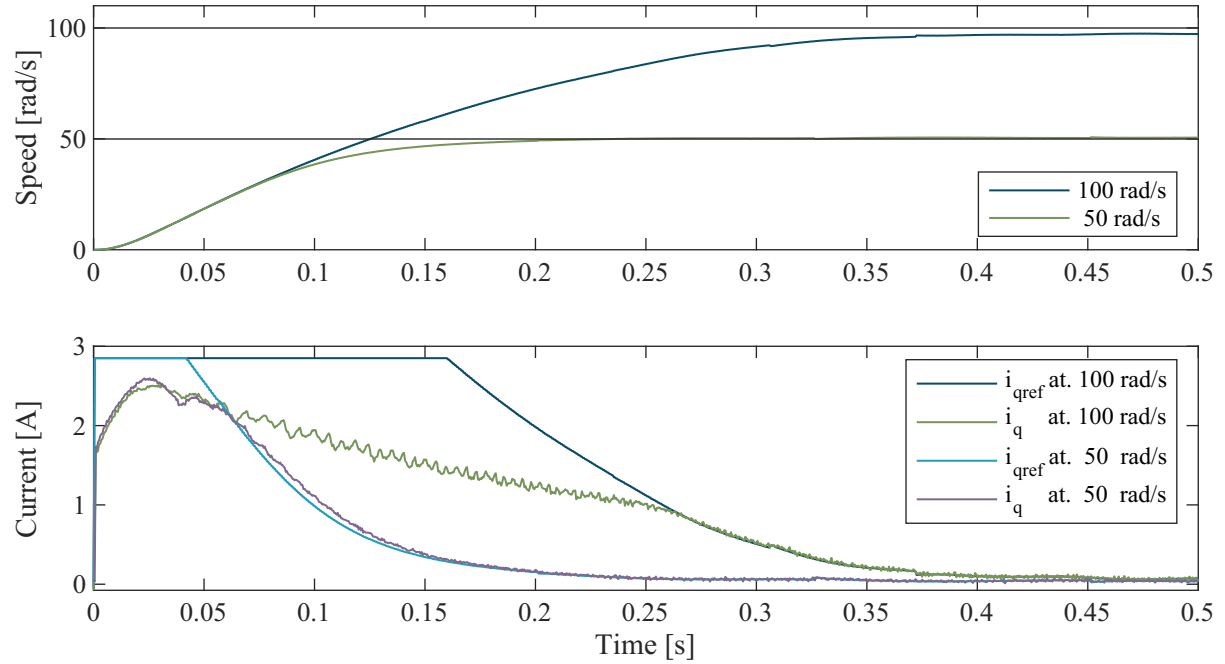


Figure 8.3. Speed step to 100 rad/s and 50 rad/s with tuned controller.

8.2.2 Low-speed control

A large inertia can be seen in Figure 8.4 to be a help in maintaining a low steady-state speed. The inertia disk allows for the motor to rotate far more smoothly at 1 rad/s than it can without the disk at 2 rad/s. The i_q current with the disk is seen to fluctuate far less since the output from the speed controller is more damped.

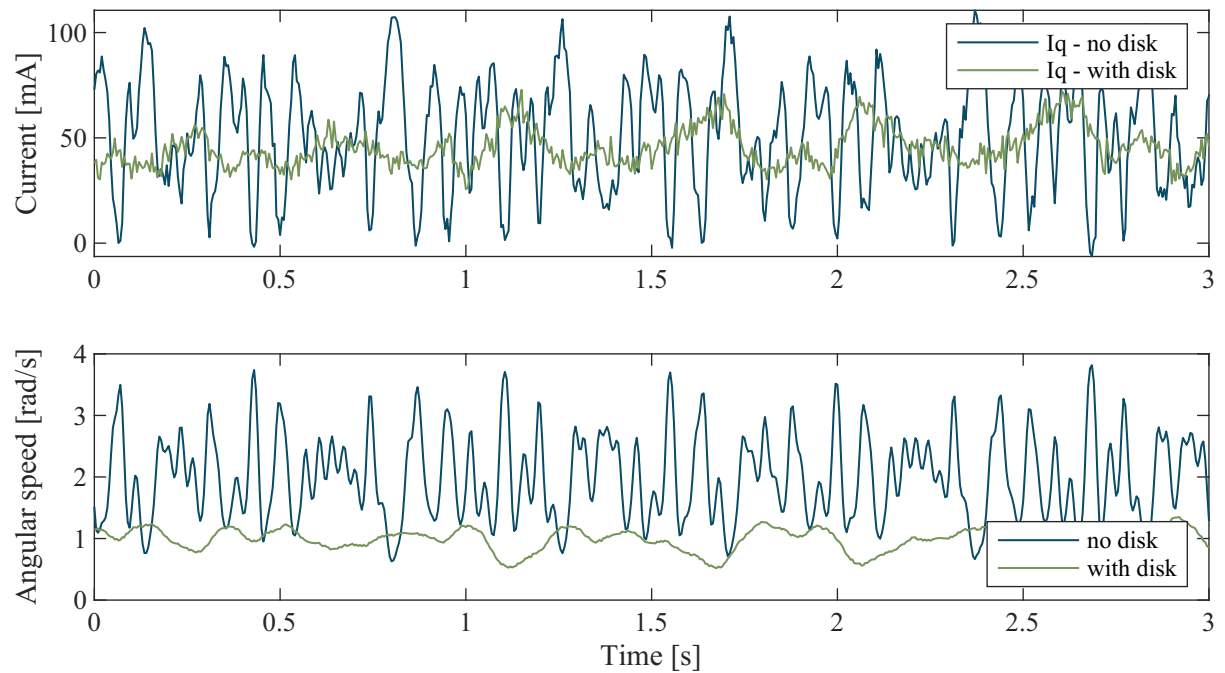


Figure 8.4. Comparison of q - d currents and speeds in a low-speed test, with and without the inertia disk.

8.3 LEGO Motor Comparison

The LEGO motor is controlled and powered by the LEGO Technic Large Hub for the tests, which has built-in speed and position controllers for the DC motor. The DC motor has significantly higher friction, due to being a brushed motor and the 36:1 gear reduction. For these reasons, these motors are not directly comparable but the results still showcase the strengths of PMSMs, direct-drive, and SVPWM.

8.3.1 Transient Comparison

The LEGO motor has a steady-state error due to its large static friction. The PMSM has a settling time of 0.237 seconds compared to 0.279 seconds for the LEGO motor. It is also seen that the LEGO motor has a steady-state error of 2° .

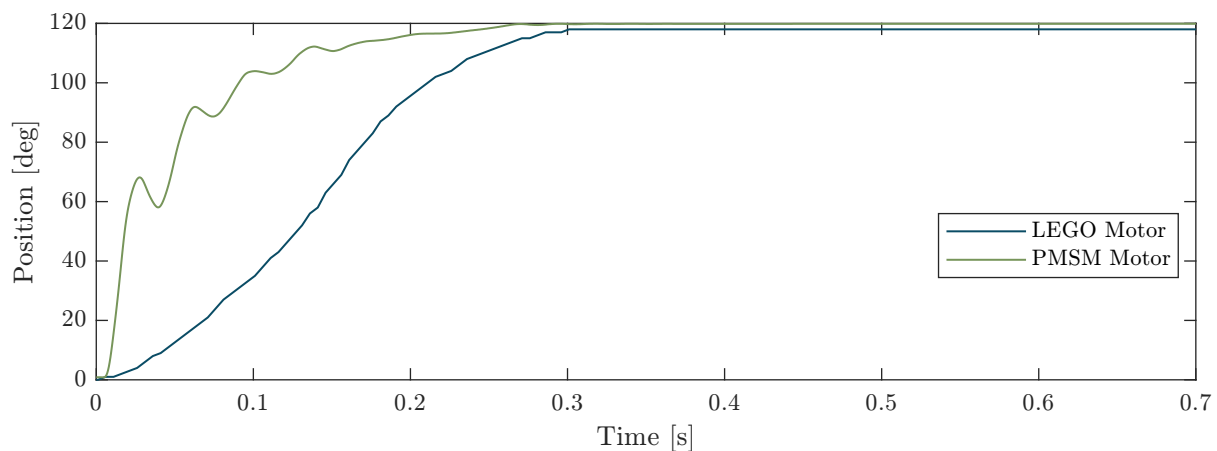


Figure 8.5. Comparison of no-load step response to 120 degrees. The LEGO DC motor has a steady-state error due to high friction in the assembly.

Overall the PMSM outperforms the LEGO motor in terms of rise time and settling time while fulfilling the requirement that no overshoot or steady state error is present. However, the system shows undesired oscillations caused by some cascade control not being applicable for the used control values. More tuning is needed to dampen out the oscillations while keeping as much of the transient performance as possible.

8.3.2 Low-Speed Control Comparison

In Figure 8.6 it is seen that the limiting factor of the LEGO motor is its high friction, which causes the motor to stop entirely at times. Here, the advantage of the direct-drive solution having lower friction outweighs the disadvantage of not having a gear reduction suited for the low-speed. At the points where the LEGO motor runs without hitches, it is running slightly more smoothly than the PMSM.

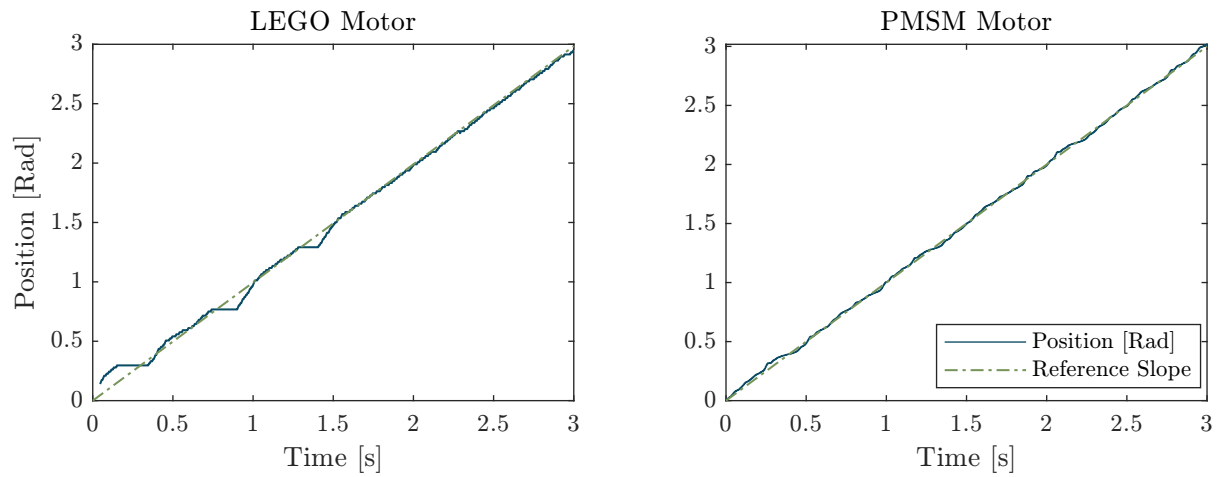


Figure 8.6. Both motors run at 1 Rad/s. This is around the limit of where the motors can run smoothly. The high friction in the LEGO motor causes the motor to stop entirely at points until the PI integrator winds up.

9.1 Filter effect on system

During testing, it was observed that the velocity response often was less damped than expected based on the tuning in the bode diagrams and step responses as shown in Figure 9.1. This is likely due to the speed low-pass filter having a cutoff frequency of $\approx 100 \text{ rad/s}$. This means that the phase shift of the low-pass filter affects the system making it less damped as is seen in the speed data in Figure 9.1. This is mostly the case when there is little inertia added to the system since the inertia affects the bandwidth significantly. During testing of the position controller in Section 8.3.1 oscillations were seen during the step response. This was expected to be due to the filtering of the speed measurements. To test this a position-step response was done with the non-filtered speed measurements. The comparison from this can be seen in Figure 9.1. Here the large oscillations during the step are removed and a faster transient response is achieved. This however comes with a trade-off of vibration during the step and at steady-state due to the BD calculation, which can be seen in the position measurements. This further leads to a lot of audible noise.

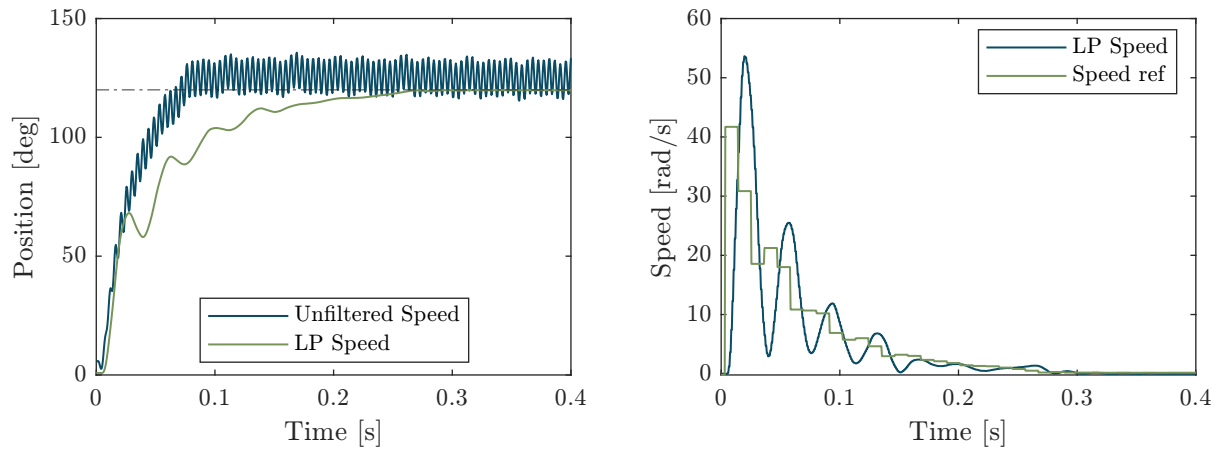


Figure 9.1. LP-filtered speed compared to the unfiltered speed response. The LP filter causes large oscillations of the speed around the reference, which slows down the transient position response.

For further work, the effect of the low-pass filter could be implemented in the design process. This might also lead to a return of the filter that potentially gives a better tradeoff between noise attenuation and transient performance.

9.2 Further tuning of controllers

After the test of the controllers, some new considerations have been made with regard to the controllers. With the noise reduced significantly with the low-pass filter, the effect of a lead term on the speed controller could be investigated. This should give the ability to increase the bandwidth of the system without making it under-dampened. Here, the effect of the low-pass filter has to be taken into account so that they do not cancel each other out.

The current PI controller is designed with a cutoff frequency significantly below that of the electrical system. This was done since at the time of implementation the designed anti-windup was not properly tuned leading to significant windup. With a better understanding of the anti-windup loop the cutoff frequency of the controller could be increased to the cutoff frequency of the electrical system. This would lead to increased disturbance rejection when the controller output is not saturated.

9.3 Filtering of current measurements

Although no test has been done to quantify this, it is obvious to a bystander that the PMSM motor produces far less audible mechanical noise than the LEGO DC motor. This is due to the reduced friction from the PMSM motor being brushless and direct-drive. However, there is a slight audible, high-frequency noise which is likely due to the motor controller reacting to high-frequency noise in the current measurements. This is most noticeable when the motor is at a standstill. Applying a low-pass filter to the current measurements could potentially reduce this noise, however, its effect on the transient performance should be taken into account.

9.4 Z-channel saved to flash memory

The encoder used is an incremental encoder which only finds its absolute position when the Z-channel is activated which happens once every iteration. Due to this, the motor has to rotate up to a revolution before the FOC is useable since it depends on the absolute position. This could somewhat be mitigated by storing the position in the flash. This would give the correct position at startup as long as the motor has not been rotated.

The best solution, however, would be to get the two-axis hall sensor or another absolute position measurement to work since the correct position is known from startup.

9.5 Hall Sensor

The 2-axis hall sensor MPS TBMA730 was initially implemented to be used as the position feedback for the controllers, however, as stated in Section 4.4.3 this was unsuccessful leading to the encoder being utilized instead. The reason for the 2-axis hall sensor not working likely comes from the positioning of the sensor compared to the magnet, where a better alignment would improve the error and most likely make it possible to achieve a linear angular position curve when utilizing BCT.

9.6 LEGO Use Cases

Using direct-drive has been shown to have advantages and disadvantages. Combined with a high pole pair and a low KV value, for the PMSM motor, it seems to be a viable solution. In cases where even higher torque is needed at low speeds, or to increase the efficiency at low speeds, a gear reduction assembly could be “outsourced” to the customers by letting them build it from LEGO parts. In this way, money can still be saved from the production side, while LEGO customers have a fun project to build.

The lower audible noise from the PMSM motor is likely to lead to an overall improved user experience of the LEGO set. It would reduce audible noise in the households that they operate in, leading to a more comfortable experience for people around it.

There is a wide range of applications in LEGO sets for the same motor. These applications come with different equivalent MOIs, meaning different system bandwidths. This leads to a couple of options: You could have one tuning of the controllers which is used for a variety of system bandwidths, or, the controllers could be tuned specifically for each LEGO-set use case, leading to optimal dynamic responses. In the first case, it is better to tune the controllers for the lowest expected bandwidth, as this leads to stability in sets with higher bandwidths at the cost of transient performance.

Conclusion 10

A PMSM is tested to compare its performance to the LEGO Powered Up Technic Large DC motor which LEGO currently sells. The hope is that a PMSM will be more efficient and open up the possibility of doing direct-drive instead of using the large 1:36 gear reduction that is used currently. This could save money on assembly costs of the gearbox and remove the friction caused by the gearing and commutator brushes. The CubeMars GL35 PMSM is chosen due to having a low KV value and 7 pole-pairs, making it suitable for low-speed position control while fulfilling the requirements. The motor is controlled through an STM32F411RE development board alongside an X-NUCLEO-IHM07M1 expansion board.

Space Vector PWM (SVPWM) combined with Field-Oriented Control (FOC) is chosen as the commutation method due to its high efficiency and low torque ripple. This method however requires more advanced math and implementation.

An electrical and mechanical system is modeled, for designing and tuning controllers. The electrical model includes the three stator phases, back EMF, and conversion into the reference frame transformations used in FOC. The mechanical model consists of the PMSM including its friction and inertia. Additionally, an ideal inverter model is set up. These models are validated by comparing them to data from open loop steps in speed, with and without added inertia from a rotating steel disk.

Four controllers are tuned based on the validated model. Two d - q current PI-controllers in a 10 kHz loop, a speed PI-controller in a 1 kHz loop, and a position P-controller in a 100 Hz loop. The speed is calculated with backward difference from the position signal of the sensor, which introduces noise to the result. Therefore, a low-pass filter is applied to the speed to reduce noise and torque ripples.

These control loops, FOC, and SVPWM are implemented in C code which can be compiled to the development board.

Considering the use case within a wide range of LEGO sets, the variations in the equivalent moment of inertia can be seen as disturbances in the system. The controllers are tuned for a high moment of inertia, with an added inertia disk, as this makes sure that the controllers work with low bandwidth systems. Additional tests are done with a mass on a string to do steady-state testing with different load torques.

The test results show that a direct-drive PMSM can be a viable choice, even for low-speed and low-speed position control. The PMSM provides better results rotating at 1 rad/s compared to the DC motor despite its 36:1 gear reduction. This is largely due to a vastly reduced friction from being direct-drive, avoiding friction in the gearbox and from the commutation brushes.

In terms of positional control, the PMSM has a settling time of 0.237 seconds in a 120° step, compared to 0.279 seconds for the DC motor, and is more capable of eliminating steady-state error, again due to reduced friction. Finally, the PMSM was tested to show a peak efficiency of 55%, compared to the 45% of the DC motor with gearbox, as shown in Appendix D.

Bibliography

- Arrow (2024). “Top 10 fundamental operational amplifier circuits”. In: URL: <https://www.arrow.com/en/research-and-events/articles/fundamentals-of-op-amp-circuits>.
- CubeMars (2024). “CubeMars GL35 product page”. In: URL: <https://www.cubemars.com/goods-1130-GL35.html>.
- FaulHaber-2250 (2024). “FaulHaber 2250 BX4”. In: URL: <https://www.faulhaber.com/en/products/series/2250bx4/>.
- FaulHaber-3216 (2024). “FaulHaber 3216 BXT”. In: URL: <https://www.faulhaber.com/en/products/series/3216bxth/>.
- Karl J. Åström, Tore Hägglund (1995). *PID-Controllers: Theory, Design and Tuning*. Vol. 2. Research Triangle Park. ISBN: 1-55617-516-7.
- Kim, Sang-Hoon (2017). *Electric Motor Control*. Vol. 1. Elsevier. ISBN: 978-0-12-812138-2.
- Millett, Pete (2024). “AC vs DC vs BLDC Motor Efficiency Comparison”. In: URL: <https://www.tytorobotics.com/blogs/articles/ac-vs-dc-motor-efficiency>.
- Philohome (2024). “LEGO 9V Technic Motors Comparison”. In: URL: <https://www.philohome.com/motors/motorcomp.htm>.
- RS-Pro-3787 (2024). “RS-Pro-216-3787”. In: URL: <https://dk.rs-online.com/web/p/dc-motorer-jaevnstromsmotorer/2163787>.
- RS-Pro-3794 (2024). “RS-Pro-216-3794”. In: URL: <https://dk.rs-online.com/web/p/dc-motorer-jaevnstromsmotorer/2163794>.
- Robotics, Tyto (2023). “Brushless Vs Brushed DC Motors: When and Why to Choose One Over the Other”. In: URL: <https://www.monolithicpower.com/en/brushless-vs-brushed-dc-motors>.
- Yu, Zhenyu (1999). “Space-Vector PWM With TMS320C24x/F24x Using Hardware and Software Determined Switching Patterns”. In: URL: https://www.ti.com/lit/an/spra524/spra524.pdf?ts=1712049395388&ref_url=https%253A%252F%252Fwww.google.com%252F.

KV-Value A

The KV-value determines how fast the motor will rotate at a given supply voltage using 6-stage commutation. Here, the switching frequency and pattern are solely dependent on the rotor position and rotational direction. A low KV-value for motor selection for this paper is therefore desired as the motor has to operate at low RPM.

When doing larger-scale production this is a parameter that can be tailored to specifications if needed by changing the amount of winding's, giving different characteristics for a motor. A specific motor architecture can therefore be decided if the selected motor is not sufficient by the end of the project. An operating motor produces a back EMF defined as the derivative of the flux linkages.

$$e = \frac{d\psi}{dt} = \frac{d\psi}{d\theta} \frac{d\theta}{dt} = \frac{d\psi}{d\theta} \cdot \omega = N \cdot A \cdot \frac{dB}{d\theta} \cdot \omega \quad (\text{A.1})$$

With $\psi = N \cdot B \cdot A$ and only flux density B is a function dependent on the position. If this is assumed constant over a revolution of the motor, the only change in design is the number of windings, N. Putting in some arbitrary numbers shows that the number of windings influences the back emf at the same rotational velocity.

$$\begin{cases} e(1000 \text{ RPM}) = 1 \text{ V}; & \text{for } N = 1 \\ e(1000 \text{ RPM}) = 2 \text{ V}; & \text{for } N = 2 \end{cases}$$

The resistance of a motor winding is $R = \frac{\rho \cdot l}{A}$ where ρ is the resistivity, l is the length of the wire, and A the cross-sectional area of the wire. Assuming the wire slot is filled with wire and has a constant resistivity of one, the following calculations can be made.

$$\left\{ \begin{array}{l} R = \frac{1}{1} = 1 \Omega; \quad I = \frac{e}{R} = \frac{1}{1} = 1; \quad P = R \cdot I^2 = 1 \cdot 1^2 = 1; \quad \text{for } N = 1, l = 1 \cdot N, A = 1 \frac{1}{N} \\ R = \frac{2}{\frac{1}{2}} = 4 \Omega; \quad I = \frac{e}{R} = \frac{2}{4} = \frac{1}{2}; \quad P = R \cdot I^2 = 4 \cdot \left(\frac{1}{2}\right)^2 = 1; \quad \text{for } N = 2, l = 1 \cdot N, A = 1 \frac{1}{N} \end{array} \right\} \quad (\text{A.2})$$

As evident, the power doesn't change for the different configurations, but due to the number of windings, the back EMF voltage is changed. A specific motor architecture can therefore be decided if the selected motor is not sufficient by the end of the project.

Resolver B

When a sinusoidal current is applied to a transformer's primary winding, the induced current in the secondary winding is also sinusoidal. If the secondary winding is rotated relative to the primary, the amplitude of the induced sinusoidal current will change, as described below:

- 0° Rotation: Amplitude is at its maximum.
- 90° Rotation: Amplitude decreases to zero.
- $90^\circ - 180^\circ$ Rotation: A phase-shifted sinusoidal current is induced, reaching its maximum negative amplitude at 180° .
- $180^\circ - 270^\circ$ Rotation: Amplitude decreases back to zero.
- $270^\circ - 360^\circ$ Rotation: Amplitude increases again, reaching its initial maximum at 360° , in phase with the original current.

A resolver utilizes this changing amplitude. The resolver consists of a stator and a rotor, where the stator consists of an exciter winding, placed on the axis of the resolver, and two two-phase winding, each placed 90° from each other on the stator. The exciter winding excites a sinusoidal voltage into the rotor winding, which in turn induces a voltage in the two two-phase windings. Since these are placed at a 90° angle from each other, one winding will output a sinusoidal voltage and one a cosine. From the sinusoidal and cosine voltages, the exact position and speed can be determined, by analyzing the amplitude and frequency of these.

Clark and Park transformation C

The point of doing transformations e.g. for supply voltage is to simplify the needed equations and control of the system. Supplying a motor with 3-phase AC voltages from the inverter, all spaced with a phase of 120° gives 3 voltage equations described in the natural coordinate system. By doing a Clark transformation this is simplified to 2 voltages at 90° phase apart. This orthogonality already makes the system of a BLDC motor easier to work with. The transformation is done as such

$$\begin{bmatrix} V_\alpha \\ V_\beta \end{bmatrix} = \frac{2}{3} \begin{bmatrix} 1 & -\frac{1}{2} & -\frac{1}{2} \\ 0 & \frac{\sqrt{3}}{2} & -\frac{\sqrt{3}}{2} \end{bmatrix} \begin{bmatrix} V_a \\ V_b \\ V_c \end{bmatrix} \quad (\text{C.1})$$

When in this stationary frame the 2 components can further be simplified by doing a rotation matrix as well, having the frame follow the mechanical rotation of the motor. The equations for this is.

$$\begin{bmatrix} V_d \\ V_q \end{bmatrix} = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} V_\alpha \\ V_\beta \end{bmatrix} \quad (\text{C.2})$$

By doing so the frame always follows the mechanical system which enables FOC control as the currents in the rotating frame can be controlled directly to give the highest torque output.

Gear vs direct drive D

When deciding whether to go with a geared or direct drive solution, multiple parameters should be taken into account. Primarily the efficiency and cost of the entire drive system.

To calculate the total efficiency of the motor data from PhilOhome is used (Philohome 2024). Here the current, voltage and rotational speed are measured at different applied torque loads. The input power can be found from the voltage and the current as $P_{in} = U \cdot I$. The output power can be found from the applied torque and the output speed as $P_{out} = \tau_{AP} \cdot \omega_{out}$. The total efficiency $\eta = \frac{P_{out}}{P_{in}}$.

The efficiency of the drivetrain can be split into two primary categories which are the efficiency of the gearbox and the efficiency of the motor. Most of the losses in the motor are due to copper losses which are dependent on the square of the current. With the internal resistance and the current measured, the copper losses can be calculated. Based on this the efficiency of the motor can be determined. The rest of the losses are then assumed to be in the gearbox. The efficiency as a function of output torque for the LEGO Powered Up Technic Large Motor can be seen in Figure D.1. Here it can be seen that the motor is most efficient at low torque where the current is small, whereas the gearbox is most efficient at high torque outputs. This is likely due to the gearbox having constant static friction. This leads to the most efficient drivetrain being dependent on the load case of the motor. Determining the different load cases for the motor is, however, outside the scope of this project.

As mentioned in chapter 1 the direct drive is proposed as it also benefits from lower assembly cost and overall lower mechanical complexity of the drivetrain while also removing the backlash existing in gearboxes. Therefore a direct drive solution is investigated since the primary focus of this project is on controlling the motor. This raises the challenges of studying how accurate low-speed/high-torque control can be achieved while not compromising efficiency.

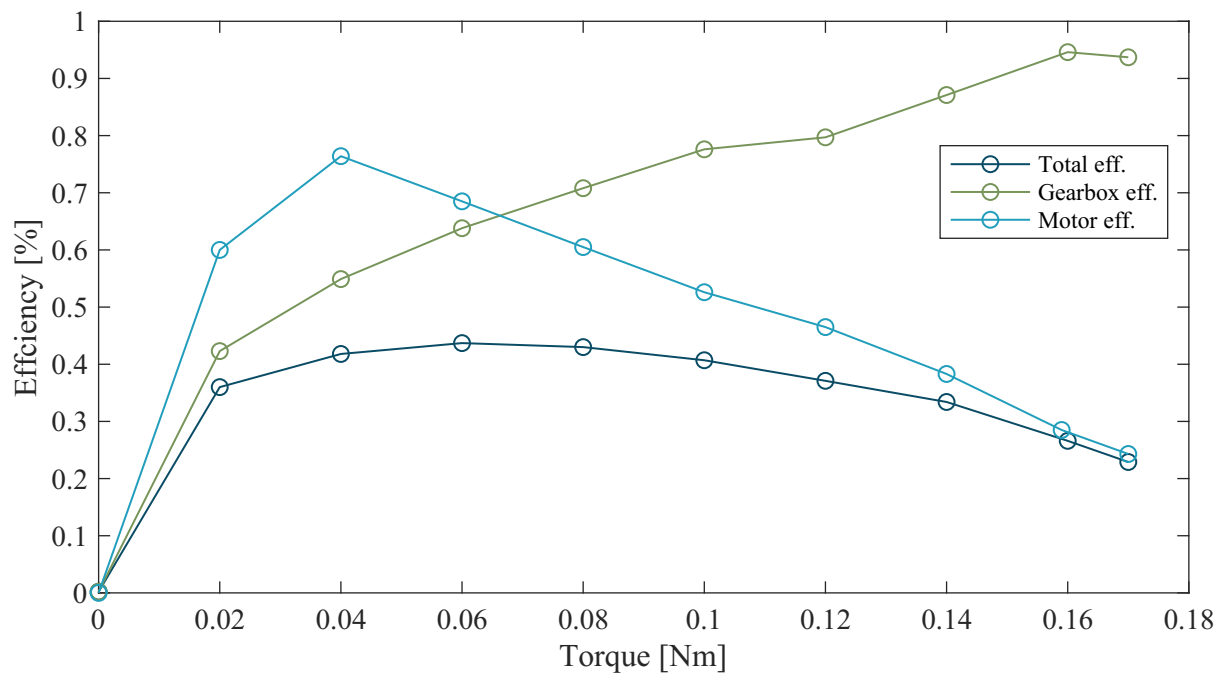


Figure D.1. The efficiency of the LEGO Powered Up Technic Large Motor motor(Philohome 2024).

Sensorless solution E

When using a sensorless solution, the position and speed of the motor are determined from observable values from the motor meaning there are no introduced extra components. This leads to less precise information, however, it also leads to a less complicated system and therefore cheaper.

The back EMF can be used to estimate both the speed and the position of the rotor. The speed can be estimated by detecting the frequency of the back EMF and the position can be estimated by detecting the zero-crossing point, meaning when the voltage goes from positive to negative over a winding. The disadvantage of using back EMF is that the position of the rotor is unknown when the motor is not running. Furthermore, the generated back EMF is low at low speeds, possibly leading to not enough information to base the commutation on. To overcome not knowing the position either “Align and go” or Initial Position Detection(IPD) can be utilized. “Align and go” applies a fixed current thereby aligning the rotor to a known position. This can however lead to unwanted motion of the rotor and backward rotation as well. IPD uses inductance measurements over the windings to determine the position of the rotor. While this ensures that the rotor does not start by moving or rotating backward, it requires additional electronics.

Model validation plots F

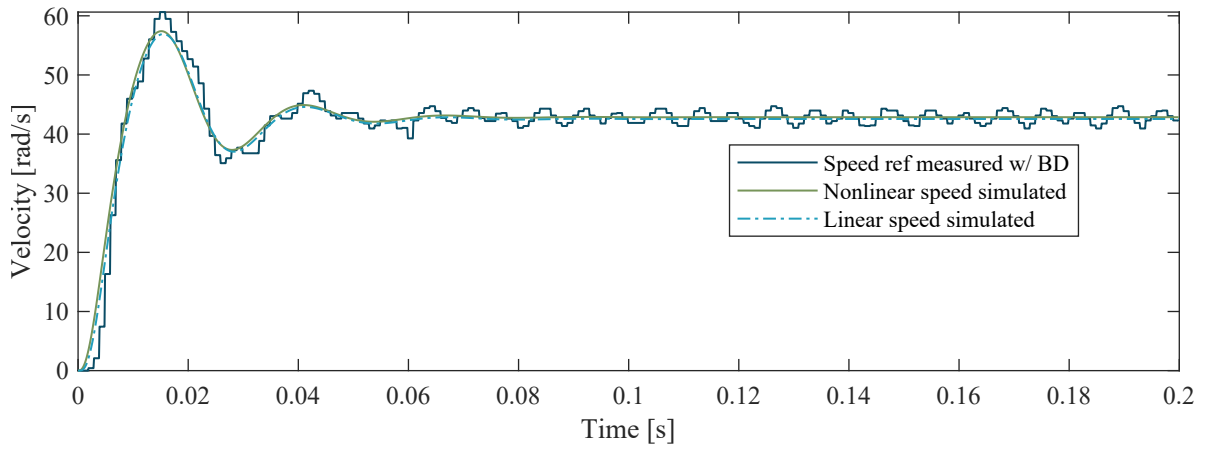


Figure F.1. Linear speed validation of open-loop current test

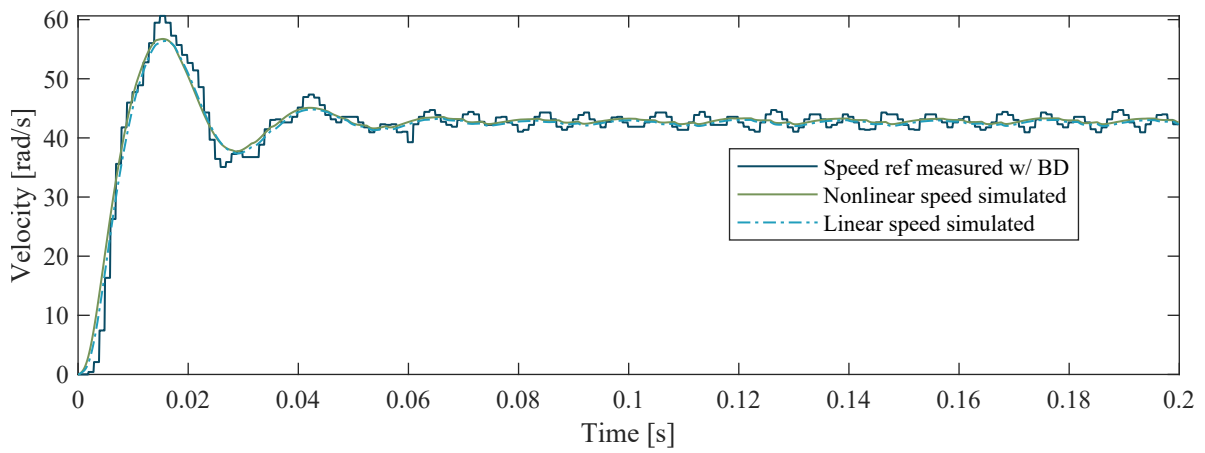


Figure F.2. Speed validation of open-loop current test with PWM modulation

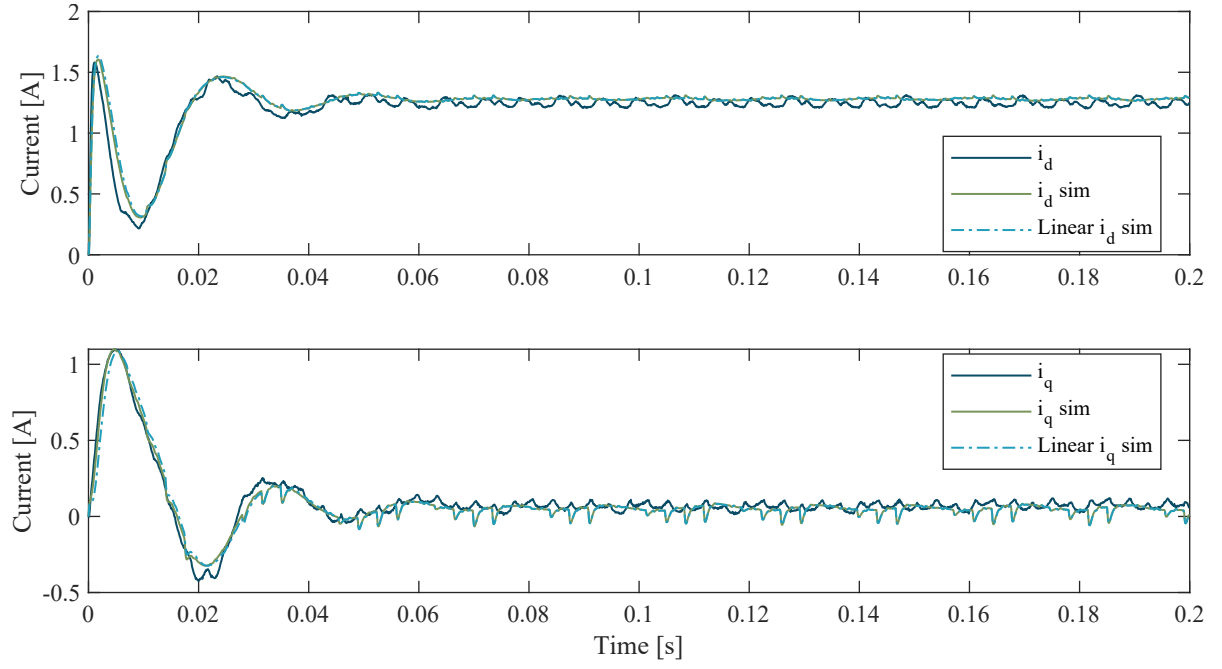


Figure F.3. Current validation of open-loop current test with PWM modulation

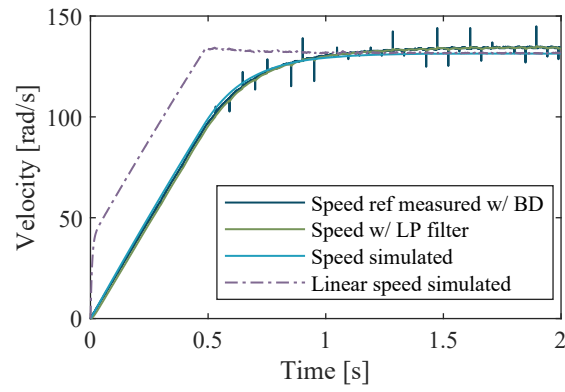


Figure F.4. Speed validation of closed-loop current test with PWM modulation

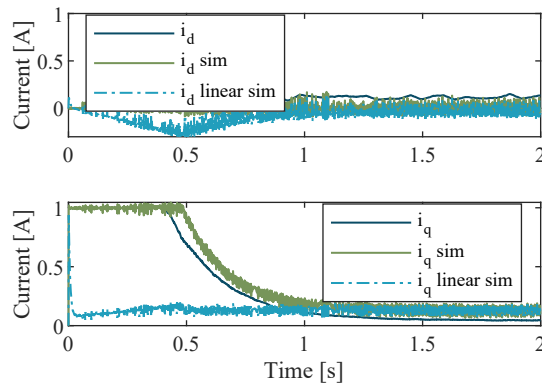


Figure F.5. Current validation of closed-loop current test with PWM modulation

Varying control on different inertia G

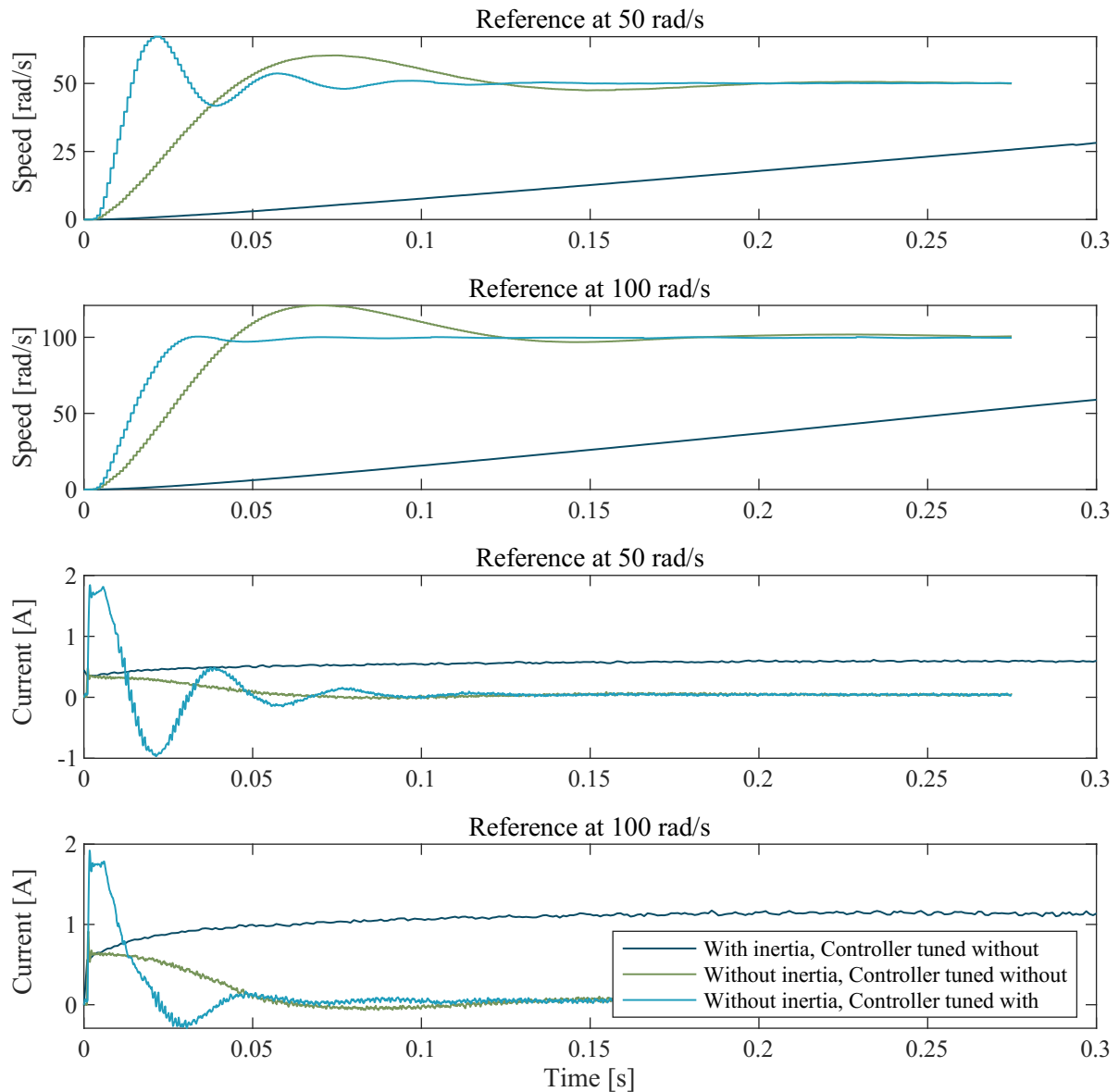


Figure G.1. Plots of speed and current responses for different controllers for different mechanical system inertias


```
1 /* USER CODE BEGIN Header */
2 /**
3
4  * @file           : main.c
5  * @brief          : Main program body
6
7  * @attention
8  *
9  * Copyright (c) 2024 STMicroelectronics.
10 * All rights reserved.
11 *
12 * This software is licensed under terms that can be found in the
    LICENSE file
13 * in the root directory of this software component.
14 * If no LICENSE file comes with this software, it is provided AS-
    IS.
15 *
16
17 */
18 /* USER CODE END Header */
19 /* Includes
    -----*/
20 #include "main.h"
21
22 /* Private includes
    -----*/
23 /* USER CODE BEGIN Includes */
24 #include "math.h"
25 #include "string.h"
26 /* USER CODE END Includes */
27
28 /* Private typedef
    -----*/
29 /* USER CODE BEGIN PTD */
30
31 /* USER CODE END PTD */
32
33 /* Private define
    -----*/
34 /* USER CODE BEGIN PD */
35 #define PI 3.14159265
36
37 // Data sampling
38 #define ARRAY_SIZE 2750 // This defines the amount of data points
    saved
```

```
39 int Data_rate = 10; // Defines the number of 10kHz iterations for
    each data sample. So if it is 10 then we save at 1kHz
40 int Data_indx; // Counter using for the data_rate
41 int count = 0; // Used as index for the arrays the data is saved in.
42 int sample_data = 0; // Should be set to 1 to start data
    acquisition.
43 int w = 0; // Defines the number of 10kHz iterations before the data
    starts sampling
44
45 int Global_enable = 0; // Variable that is changed to globally
    enable or disable the inverter and control loop.
46
47 float omega_ref = 0; //[Rad/s]
48 float i_qref = 0;
49 int pos_loop = 0; // 1 if position controller is active.
50 int Speed_loop = 0; // 1 if speed controller is active.
51 float phi_ref = 0; // Reference position.
52 int send_data = 0;
53
54 // Controller Variables
55 float T_e = 0.0001;
56 float T_m = 0.001;
57
58 // For controllers using Kp and Ki with antiwindup
59 float K_p_m = 0.0792; // 1.5842; // 0.045;
60 float K_i_m = 0.0200; // 0.035;
61
62 float K_p_e = 7;
63 float K_i_e = 700;
64
65
66 // Define arrays to save the data in
67 float array1[ARRAY_SIZE]; //
68 float array2[ARRAY_SIZE]; //
69 float array3[ARRAY_SIZE];
70 float array4[ARRAY_SIZE];
71 float array5[ARRAY_SIZE];
72 float array6[ARRAY_SIZE];
73 float array7[ARRAY_SIZE];
74 float array8[ARRAY_SIZE];
75 float array9[ARRAY_SIZE];
76 float array10[ARRAY_SIZE];
77 float array11[ARRAY_SIZE];
78 //uint16_t array12[ARRAY_SIZE];
79 //uint16_t array13[ARRAY_SIZE];
80 //uint16_t array14[ARRAY_SIZE];
81
82 int j; // Counter used to see that the while loop is executed
83 int k; //
84
85 int location = 0;
```

```
86
87
88 float V_alpha;
89 float V_beta;
90 float t = 0;
91 float omega_dot = 62.83;
92 int constant_acc = 0;
93
94
95 // Variables for SVPWM
96 #define V_s 16 // Supply voltage [V]
97 float V_s_factor = 1.0/(2.0*16.0/3.0); // TH
98 double T_1;
99 double T_2;
100 double T_0;
101 uint32_t T1_count;
102 uint32_t T2_count;
103 uint32_t T0_count;
104 int N; // Used to determine the sector
105 int Sector; // Variable for which sector we are in. Only used for
    validation.
106
107
108 // Angle measurements
109 int angle;
110 float angle_abs = 0;
111 float abs_add = 0;
112 uint16_t angle_HS;
113 uint16_t angle_HS_shift;
114 float angleRad;
115 uint16_t angle_encoder;
116
117 // Function definitions for functions defined later in the code
118 float getSine(unsigned int angle);
119 float getCoSine(unsigned int angle);
120
121 float FilterSpeed(float speed, float LP_old);
122 void readAngleFromMA730(uint16_t *angle_HS);
123
124 //Terkel
125 // Current variables.
126 float i1 = 0; // This is the current converted from ADC measurement
127 float i2 = 0;
128 float i3 = 0;
129
130 // Currents converted to dq-system
131 float i_d = 0;
132 float i_q = 0;
133
134 float V_dref = 0;
135 float V_dref_sat = 0;
```

```
136 float V_qref = 0;
137 float V_qref_sat = 0;
138 float C_omega_i_old = 0;
139 float C_omega_i = 0;
140 float i_qref_sat = 0;
141
142 float C_dCurr_old = 0;
143 float C_qCurr_old = 0;
144 float C_dCurr_i = 0;
145 float C_qCurr_i = 0;
146
147
148 // Position controller variables
149
150
151 float e_omega;
152
153 float e_dCurr = 0;
154 float e_qCurr = 0;
155
156
157 // speed calculation variables
158 float phi_old = 0;
159 int indx = 0;
160 float speed_old = 0;
161 float speed_LP;
162 float LP_old = 0;
163 float alpha = 0.9101; // Discretization
164
165
166
167
168
169
170
171 // Speed_rad, Position_rad_encoder, Position_rad_hall, i1, i2, i3,
    speed_reference, position_reference.
172
173
174
175
176 // Generate empty array for sine lookup and the step size.
177 #define NUM_POINTS 536 // Number of data points in lookup array.
178 float SinTable[NUM_POINTS];
179 float CosTable[NUM_POINTS];
180 const double step = 2.0 * M_PI / ( 4*NUM_POINTS); // Angle step size
181
182 // Variables used to store the sine and cosine lookup value for the
    given iteration.
183 float sin_lookup;
184 float cos_lookup;
```

```
185
186
187 /* USER CODE END PD */
188
189 /* Private macro
-----*/
190 /* USER CODE BEGIN PM */
191
192 /* USER CODE END PM */
193
194 /* Private variables
-----*/
195 ADC_HandleTypeDef hadc1;
196 DMA_HandleTypeDef hdma_adc1;
197
198 SPI_HandleTypeDef hspi3;
199
200 TIM_HandleTypeDef htim1;
201 TIM_HandleTypeDef htim2;
202 TIM_HandleTypeDef htim3;
203 TIM_HandleTypeDef htim11;
204
205 UART_HandleTypeDef huart2;
206
207 /* USER CODE BEGIN PV */
208
209 /* USER CODE END PV */
210
211 /* Private function prototypes
-----*/
212 void SystemClock_Config(void);
213 static void MX_GPIO_Init(void);
214 static void MX_DMA_Init(void);
215 static void MX_TIM11_Init(void);
216 static void MX_TIM1_Init(void);
217 static void MX_ADC1_Init(void);
218 static void MX_TIM3_Init(void);
219 static void MX_TIM2_Init(void);
220 static void MX_USART2_UART_Init(void);
221 static void MX_SPI3_Init(void);
222 /* USER CODE BEGIN PFP */
223
224 /* USER CODE END PFP */
225
226 /* Private user code
-----*/
227 /* USER CODE BEGIN 0 */
228
229 uint16_t tim1_cnt = 0;
230 uint16_t tim11_cnt = 0;
231 uint32_t tim2_cnt = 0;
```

```
232 uint16_t enc_cnt = 0;
233
234
235 float phi = 0;
236 float phi_nomod = 0;
237 float phi_el = 0;
238 float speed = 0;
239 float speed_rad = 0;
240 int rpm = 0;
241 int ext = 0;
242 float filter_coeff = 0.1;
243 float speed_LP = 0;
244 //float speed_kal = 0;
245
246 //int EMFA = 0;
247 //int EMFB = 0;
248 //int EMFC = 0;
249
250 uint16_t adc[3];
251
252 float i1_raw;
253 float i2_raw;
254 float i3_raw;
255
256 #define VSENSE 3.3/4096
257
258 void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
259 {
260     TIM3->CNT = 0;
261     ext = ext + 1;
262 }
263
264 /* USER CODE END 0 */
265
266 /**
267  * @brief The application entry point.
268  * @retval int
269  */
270 int main(void)
271 {
272     /* USER CODE BEGIN 1 */
273
274     /* USER CODE END 1 */
275
276     /* MCU
    Configuration-----
    */
277
278     /* Reset of all peripherals, Initializes the Flash interface and
    the SysTick. */
279     HAL_Init();
```

```
280
281  /* USER CODE BEGIN Init */
282  /* USER CODE END Init */
283
284  /* Configure the system clock */
285  SystemClock_Config();
286
287  /* USER CODE BEGIN SysInit */
288
289  /* USER CODE END SysInit */
290
291  /* Initialize all configured peripherals */
292  MX_GPIO_Init();
293  MX_DMA_Init();
294  MX_TIM11_Init();
295  MX_TIM1_Init();
296  MX_ADC1_Init();
297  MX_TIM3_Init();
298  MX_TIM2_Init();
299  MX_USART2_UART_Init();
300  MX_SPI3_Init();
301  /* USER CODE BEGIN 2 */
302
303  // Create sin angle Table.
304  for (int i = 0; i < NUM_POINTS; ++i) {
305      double angle = i * step;
306      SinTable[i] = sin(angle);
307  }
308
309  // Create cosine angle Table.
310  for (int i = 0; i < NUM_POINTS; ++i) {
311      double angle = i * step;
312      CosTable[i] = cos(angle);
313  }
314
315  // Global enable from the begining
316
317  // enable the transistors from the begining.
318  HAL_GPIO_WritePin(GPIOC, GPIO_PIN_10, 1);
319  HAL_GPIO_WritePin(GPIOC, GPIO_PIN_11, 1);
320  HAL_GPIO_WritePin(GPIOC, GPIO_PIN_12, 1);
321
322  // initialize counter compare value to 0 and start the timer PWM
  for timer 1 channel 1.
323  TIM1->CCR1 = 0;
324  TIM1->CCR2 = 0;
325  TIM1->CCR3 = 0;
326  HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
327  HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_2);
328  HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_3);
329  HAL_TIM_OC_Start(&htim1, TIM_CHANNEL_4); //slet
```

```
330
331 HAL_TIM_Base_Start(&htim2);
332 HAL_TIM_Base_Start_IT(&htim11);
333
334 //Global_enable = 1;
335
336 //Terkel
337 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
338 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET);
339 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_RESET);
340
341 HAL_ADC_Start_DMA(&hadcl1, (uint32_t*) adc, 3);
342
343 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, 1);
344 #define VREFINT 1.21
345 #define ADCMAX 4095.0
346
347 /* USER CODE END 2 */
348
349 /* Infinite loop */
350 /* USER CODE BEGIN WHILE */
351 while (1)
352 {
353     /* USER CODE END WHILE */
354
355     /* USER CODE BEGIN 3 */
356
357     if(Global_enable == 0 && sample_data == 0){
358         count = 0;
359         w = 0;
360         Data_indx = 0;
361         //indx = 0;
362         //k = 0;
363         t = 0;
364
365
366     }
367     j = j +1;
368 }
369 /* USER CODE END 3 */
370 }
371
372 /**
373  * @brief System Clock Configuration
374  * @retval None
375  */
376 void SystemClock_Config(void)
377 {
378     RCC_OscInitTypeDef RCC_OscInitStruct = {0};
379     RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
380
```

```
381  /** Configure the main internal regulator output voltage
382  */
383  __HAL_RCC_PWR_CLK_ENABLE();
384  __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);
385  /** Initializes the RCC Oscillators according to the specified
    parameters
386  * in the RCC_OscInitTypeDef structure.
387  */
388  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
389  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
390  RCC_OscInitStruct.HSICalibrationValue =
    RCC_HSICALIBRATION_DEFAULT;
391  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
392  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
393  RCC_OscInitStruct.PLL.PLLM = 8;
394  RCC_OscInitStruct.PLL.PLLN = 96;
395  RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
396  RCC_OscInitStruct.PLL.PLLQ = 4;
397  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
398  {
399      Error_Handler();
400  }
401  /** Initializes the CPU, AHB and APB buses clocks
402  */
403  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|
    RCC_CLOCKTYPE_SYSCLK
404                                |RCC_CLOCKTYPE_PCLK1|
    RCC_CLOCKTYPE_PCLK2;
405  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
406  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
407  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
408  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
409
410  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_3) !=
    HAL_OK)
411  {
412      Error_Handler();
413  }
414 }
415
416 /**
417  * @brief ADC1 Initialization Function
418  * @param None
419  * @retval None
420  */
421 static void MX_ADC1_Init(void)
422 {
423
424  /* USER CODE BEGIN ADC1_Init 0 */
425
426  /* USER CODE END ADC1_Init 0 */
```

```
427
428  ADC_ChannelConfTypeDef sConfig = {0};
429
430  /* USER CODE BEGIN ADC1_Init 1 */
431
432  /* USER CODE END ADC1_Init 1 */
433  /** Configure the global features of the ADC (Clock, Resolution,
434      Data Alignment and number of conversion)
435  */
436  hadc1.Instance = ADC1;
437  hadc1.Init.ClockPrescaler = ADC_CLOCK_SYNC_PCLK_DIV4;
438  hadc1.Init.Resolution = ADC_RESOLUTION_12B;
439  hadc1.Init.ScanConvMode = ENABLE;
440  hadc1.Init.ContinuousConvMode = DISABLE;
441  hadc1.Init.DiscontinuousConvMode = ENABLE;
442  hadc1.Init.NbrOfDiscConversion = 3;
443  hadc1.Init.ExternalTrigConvEdge =
444      ADC_EXTERNALTRIGCONVEDGE_FALLING;
445  hadc1.Init.ExternalTrigConv = ADC_EXTERNALTRIGCONV_T2_TRGO;
446  hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT;
447  hadc1.Init.NbrOfConversion = 3;
448  hadc1.Init.DMAContinuousRequests = ENABLE;
449  hadc1.Init.EOCSelection = ADC_EOC_SEQ_CONV;
450  if (HAL_ADC_Init(&hadc1) != HAL_OK)
451  {
452      Error_Handler();
453  }
454  /** Configure for the selected ADC regular channel its
455      corresponding rank in the sequencer and its sample time.
456  */
457  sConfig.Channel = ADC_CHANNEL_0;
458  sConfig.Rank = 1;
459  sConfig.SamplingTime = ADC_SAMPLETIME_3CYCLES;
460  if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK)
461  {
462      Error_Handler();
463  }
464  /** Configure for the selected ADC regular channel its
465      corresponding rank in the sequencer and its sample time.
466  */
467  sConfig.Channel = ADC_CHANNEL_11;
468  sConfig.Rank = 2;
469  if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK)
470  {
471      Error_Handler();
472  }
473  /** Configure for the selected ADC regular channel its
474      corresponding rank in the sequencer and its sample time.
475  */
476  sConfig.Channel = ADC_CHANNEL_10;
477  sConfig.Rank = 3;
```

```
473  if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK)
474  {
475      Error_Handler();
476  }
477  /* USER CODE BEGIN ADC1_Init 2 */
478
479  /* USER CODE END ADC1_Init 2 */
480
481 }
482
483 /**
484  * @brief SPI3 Initialization Function
485  * @param None
486  * @retval None
487  */
488 static void MX_SPI3_Init(void)
489 {
490
491  /* USER CODE BEGIN SPI3_Init 0 */
492
493  /* USER CODE END SPI3_Init 0 */
494
495  /* USER CODE BEGIN SPI3_Init 1 */
496
497  /* USER CODE END SPI3_Init 1 */
498  /* SPI3 parameter configuration*/
499  hspi3.Instance = SPI3;
500  hspi3.Init.Mode = SPI_MODE_MASTER;
501  hspi3.Init.Direction = SPI_DIRECTION_2LINES;
502  hspi3.Init.DataSize = SPI_DATASIZE_16BIT;
503  hspi3.Init.CLKPolarity = SPI_POLARITY_LOW;
504  hspi3.Init.CLKPhase = SPI_PHASE_1EDGE;
505  hspi3.Init.NSS = SPI_NSS_SOFT;
506  hspi3.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_2;
507  hspi3.Init.FirstBit = SPI_FIRSTBIT_MSB;
508  hspi3.Init.TIMode = SPI_TIMODE_DISABLE;
509  hspi3.Init.CRCCalculation = SPI_CRCCALCULATION_DISABLE;
510  hspi3.Init.CRCPolynomial = 10;
511  if (HAL_SPI_Init(&hspi3) != HAL_OK)
512  {
513      Error_Handler();
514  }
515  /* USER CODE BEGIN SPI3_Init 2 */
516
517
518
519
520
521  /* USER CODE END SPI3_Init 2 */
522
523 }
```

```
524
525 /**
526  * @brief TIM1 Initialization Function
527  * @param None
528  * @retval None
529  */
530 static void MX_TIM1_Init(void)
531 {
532
533     /* USER CODE BEGIN TIM1_Init 0 */
534
535     /* USER CODE END TIM1_Init 0 */
536
537     TIM_MasterConfigTypeDef sMasterConfig = {0};
538     TIM_OC_InitTypeDef sConfigOC = {0};
539     TIM_BreakDeadTimeConfigTypeDef sBreakDeadTimeConfig = {0};
540
541     /* USER CODE BEGIN TIM1_Init 1 */
542
543     /* USER CODE END TIM1_Init 1 */
544     htim1.Instance = TIM1;
545     htim1.Init.Prescaler = 96-1;
546     htim1.Init.CounterMode = TIM_COUNTERMODE_CENTERALIGNED1;
547     htim1.Init.Period = 49;
548     htim1.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
549     htim1.Init.RepetitionCounter = 0;
550     htim1.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
551     if (HAL_TIM_PWM_Init(&htim1) != HAL_OK)
552     {
553         Error_Handler();
554     }
555     if (HAL_TIM_OC_Init(&htim1) != HAL_OK)
556     {
557         Error_Handler();
558     }
559     sMasterConfig.MasterOutputTrigger = TIM_TRGO_UPDATE;
560     sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_ENABLE;
561     if (HAL_TIMEx_MasterConfigSynchronization(&htim1, &sMasterConfig)
        != HAL_OK)
562     {
563         Error_Handler();
564     }
565     sConfigOC.OCMode = TIM_OCMODE_PWM1;
566     sConfigOC.Pulse = 0;
567     sConfigOC.OCpolarity = TIM_OCPOLARITY_HIGH;
568     sConfigOC.OCNPolarity = TIM_OCNPOLARITY_HIGH;
569     sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
570     sConfigOC.OCIdleState = TIM_OCIDLESTATE_RESET;
571     sConfigOC.OCNIdleState = TIM_OCNIDLESTATE_RESET;
572     if (HAL_TIM_PWM_ConfigChannel(&htim1, &sConfigOC, TIM_CHANNEL_1) !=
        HAL_OK)
```

```
573 {
574     Error_Handler();
575 }
576 if (HAL_TIM_PWM_ConfigChannel(&htim1, &sConfigOC, TIM_CHANNEL_2) !=
    = HAL_OK)
577 {
578     Error_Handler();
579 }
580 if (HAL_TIM_PWM_ConfigChannel(&htim1, &sConfigOC, TIM_CHANNEL_3) !=
    = HAL_OK)
581 {
582     Error_Handler();
583 }
584 sConfigOC.OCMode = TIM_OCMode_ACTIVE;
585 sConfigOC.Pulse = 1200;
586 if (HAL_TIM_OC_ConfigChannel(&htim1, &sConfigOC, TIM_CHANNEL_4) !=
    HAL_OK)
587 {
588     Error_Handler();
589 }
590 sBreakDeadTimeConfig.OffStateRunMode = TIM_OSSR_DISABLE;
591 sBreakDeadTimeConfig.OffStateIDLEMode = TIM_OSSI_DISABLE;
592 sBreakDeadTimeConfig.LockLevel = TIM_LOCKLEVEL_OFF;
593 sBreakDeadTimeConfig.DeadTime = 0;
594 sBreakDeadTimeConfig.BreakState = TIM_BREAK_DISABLE;
595 sBreakDeadTimeConfig.BreakPolarity = TIM_BREAKPOLARITY_HIGH;
596 sBreakDeadTimeConfig.AutomaticOutput =
    TIM_AUTOMATICOUTPUT_DISABLE;
597 if (HAL_TIMEx_ConfigBreakDeadTime(&htim1, &sBreakDeadTimeConfig) !=
    = HAL_OK)
598 {
599     Error_Handler();
600 }
601 /* USER CODE BEGIN TIM1_Init 2 */
602
603 /* USER CODE END TIM1_Init 2 */
604 HAL_TIM_MspPostInit(&htim1);
605
606 }
607
608 /**
609  * @brief TIM2 Initialization Function
610  * @param None
611  * @retval None
612  */
613 static void MX_TIM2_Init(void)
614 {
615
616     /* USER CODE BEGIN TIM2_Init 0 */
617
618     /* USER CODE END TIM2_Init 0 */
```

```
619
620 TIM_SlaveConfigTypeDef sSlaveConfig = {0};
621 TIM_MasterConfigTypeDef sMasterConfig = {0};
622
623 /* USER CODE BEGIN TIM2_Init 1 */
624
625 /* USER CODE END TIM2_Init 1 */
626 htim2.Instance = TIM2;
627 htim2.Init.Prescaler = 192-1;
628 htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
629 htim2.Init.Period = 48;
630 htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
631 htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
632 if (HAL_TIM_Base_Init(&htim2) != HAL_OK)
633 {
634     Error_Handler();
635 }
636 sSlaveConfig.SlaveMode = TIM_SLAVEMODE_TRIGGER;
637 sSlaveConfig.InputTrigger = TIM_TS_ITR0;
638 if (HAL_TIM_SlaveConfigSynchro(&htim2, &sSlaveConfig) != HAL_OK)
639 {
640     Error_Handler();
641 }
642 sMasterConfig.MasterOutputTrigger = TIM_TRGO_UPDATE;
643 sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
644 if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig)
    != HAL_OK)
645 {
646     Error_Handler();
647 }
648 /* USER CODE BEGIN TIM2_Init 2 */
649
650 /* USER CODE END TIM2_Init 2 */
651
652 }
653
654 /**
655  * @brief TIM3 Initialization Function
656  * @param None
657  * @retval None
658  */
659 static void MX_TIM3_Init(void)
660 {
661
662     /* USER CODE BEGIN TIM3_Init 0 */
663
664     /* USER CODE END TIM3_Init 0 */
665
666     TIM_Encoder_InitTypeDef sConfig = {0};
667     TIM_MasterConfigTypeDef sMasterConfig = {0};
668
```

```
669  /* USER CODE BEGIN TIM3_Init 1 */
670
671  /* USER CODE END TIM3_Init 1 */
672  htim3.Instance = TIM3;
673  htim3.Init.Prescaler = 0;
674  htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
675  htim3.Init.Period = 15000;
676  htim3.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
677  htim3.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
678  sConfig.EncoderMode = TIM_ENCODERMODE_TI1;
679  sConfig.IC1Polarity = TIM_ICPOLARITY_RISING;
680  sConfig.IC1Selection = TIM_ICSELECTION_DIRECTTI;
681  sConfig.IC1Prescaler = TIM_ICPSC_DIV1;
682  sConfig.IC1Filter = 0;
683  sConfig.IC2Polarity = TIM_ICPOLARITY_RISING;
684  sConfig.IC2Selection = TIM_ICSELECTION_DIRECTTI;
685  sConfig.IC2Prescaler = TIM_ICPSC_DIV1;
686  sConfig.IC2Filter = 0;
687  if (HAL_TIM_Encoder_Init(&htim3, &sConfig) != HAL_OK)
688  {
689      Error_Handler();
690  }
691  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
692  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
693  if (HAL_TIMEx_MasterConfigSynchronization(&htim3, &sMasterConfig)
    != HAL_OK)
694  {
695      Error_Handler();
696  }
697  /* USER CODE BEGIN TIM3_Init 2 */
698
699  HAL_TIM_Encoder_Start(&htim3, TIM_CHANNEL_ALL);
700
701  /* USER CODE END TIM3_Init 2 */
702
703 }
704
705 /**
706  * @brief TIM11 Initialization Function
707  * @param None
708  * @retval None
709  */
710 static void MX_TIM11_Init(void)
711 {
712
713     /* USER CODE BEGIN TIM11_Init 0 */
714
715     /* USER CODE END TIM11_Init 0 */
716
717     /* USER CODE BEGIN TIM11_Init 1 */
718
```

```
669  /* USER CODE BEGIN TIM3_Init 1 */
670
671  /* USER CODE END TIM3_Init 1 */
672  htim3.Instance = TIM3;
673  htim3.Init.Prescaler = 0;
674  htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
675  htim3.Init.Period = 15000;
676  htim3.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
677  htim3.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
678  sConfig.EncoderMode = TIM_ENCODERMODE_TI1;
679  sConfig.IC1Polarity = TIM_ICPOLARITY_RISING;
680  sConfig.IC1Selection = TIM_ICSELECTION_DIRECTTI;
681  sConfig.IC1Prescaler = TIM_ICPSC_DIV1;
682  sConfig.IC1Filter = 0;
683  sConfig.IC2Polarity = TIM_ICPOLARITY_RISING;
684  sConfig.IC2Selection = TIM_ICSELECTION_DIRECTTI;
685  sConfig.IC2Prescaler = TIM_ICPSC_DIV1;
686  sConfig.IC2Filter = 0;
687  if (HAL_TIM_Encoder_Init(&htim3, &sConfig) != HAL_OK)
688  {
689      Error_Handler();
690  }
691  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
692  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
693  if (HAL_TIMEx_MasterConfigSynchronization(&htim3, &sMasterConfig)
    != HAL_OK)
694  {
695      Error_Handler();
696  }
697  /* USER CODE BEGIN TIM3_Init 2 */
698
699  HAL_TIM_Encoder_Start(&htim3, TIM_CHANNEL_ALL);
700
701  /* USER CODE END TIM3_Init 2 */
702
703 }
704
705 /**
706  * @brief TIM11 Initialization Function
707  * @param None
708  * @retval None
709  */
710 static void MX_TIM11_Init(void)
711 {
712
713     /* USER CODE BEGIN TIM11_Init 0 */
714
715     /* USER CODE END TIM11_Init 0 */
716
717     /* USER CODE BEGIN TIM11_Init 1 */
718
```

```
719  /* USER CODE END TIM11_Init 1 */
720  htim11.Instance = TIM11;
721  htim11.Init.Prescaler = 192-1;
722  htim11.Init.CounterMode = TIM_COUNTERMODE_UP;
723  htim11.Init.Period = 48;
724  htim11.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
725  htim11.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
726  if (HAL_TIM_Base_Init(&htim11) != HAL_OK)
727  {
728      Error_Handler();
729  }
730  /* USER CODE BEGIN TIM11_Init 2 */
731
732  /* USER CODE END TIM11_Init 2 */
733
734 }
735
736 /**
737  * @brief USART2 Initialization Function
738  * @param None
739  * @retval None
740  */
741 static void MX_USART2_UART_Init(void)
742 {
743
744  /* USER CODE BEGIN USART2_Init 0 */
745
746  /* USER CODE END USART2_Init 0 */
747
748  /* USER CODE BEGIN USART2_Init 1 */
749
750  /* USER CODE END USART2_Init 1 */
751  huart2.Instance = USART2;
752  huart2.Init.BaudRate = 115200;
753  huart2.Init.WordLength = UART_WORDLENGTH_8B;
754  huart2.Init.StopBits = UART_STOPBITS_1;
755  huart2.Init.Parity = UART_PARITY_NONE;
756  huart2.Init.Mode = UART_MODE_TX_RX;
757  huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
758  huart2.Init.OverSampling = UART_OVERSAMPLING_16;
759  if (HAL_UART_Init(&huart2) != HAL_OK)
760  {
761      Error_Handler();
762  }
763  /* USER CODE BEGIN USART2_Init 2 */
764
765  /* USER CODE END USART2_Init 2 */
766
767 }
768
769 /**
```

```
770  * Enable DMA controller clock
771  */
772 static void MX_DMA_Init(void)
773 {
774
775  /* DMA controller clock enable */
776  __HAL_RCC_DMA2_CLK_ENABLE();
777
778  /* DMA interrupt init */
779  /* DMA2_Stream0_IRQn interrupt configuration */
780  HAL_NVIC_SetPriority(DMA2_Stream0_IRQn, 0, 0);
781  HAL_NVIC_EnableIRQ(DMA2_Stream0_IRQn);
782
783 }
784
785 /**
786  * @brief GPIO Initialization Function
787  * @param None
788  * @retval None
789  */
790 static void MX_GPIO_Init(void)
791 {
792  GPIO_InitTypeDef GPIO_InitStructure = {0};
793
794  /* GPIO Ports Clock Enable */
795  __HAL_RCC_GPIOC_CLK_ENABLE();
796  __HAL_RCC_GPIOA_CLK_ENABLE();
797  __HAL_RCC_GPIOB_CLK_ENABLE();
798  __HAL_RCC_GPIOD_CLK_ENABLE();
799
800  /*Configure GPIO pin Output Level */
801  HAL_GPIO_WritePin(GPIOC, GPIO_PIN_3|GPIO_PIN_9|GPIO_PIN_10|
    GPIO_PIN_11
802                      |GPIO_PIN_12, GPIO_PIN_RESET);
803
804  /*Configure GPIO pin Output Level */
805  HAL_GPIO_WritePin(GPIOA, LED_Pin|GPIO_PIN_6, GPIO_PIN_RESET);
806
807  /*Configure GPIO pin Output Level */
808  HAL_GPIO_WritePin(GPIOD, GPIO_PIN_2, GPIO_PIN_RESET);
809
810  /*Configure GPIO pins : PC3 PC9 PC10 PC11
811                          PC12 */
812  GPIO_InitStructure.Pin = GPIO_PIN_3|GPIO_PIN_9|GPIO_PIN_10|
    GPIO_PIN_11
813                          |GPIO_PIN_12;
814  GPIO_InitStructure.Mode = GPIO_MODE_OUTPUT_PP;
815  GPIO_InitStructure.Pull = GPIO_NOPULL;
816  GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_LOW;
817  HAL_GPIO_Init(GPIOC, &GPIO_InitStructure);
818
```

```
819  /*Configure GPIO pins : LED_Pin PA6 */
820  GPIO_InitStruct.Pin = LED_Pin|GPIO_PIN_6;
821  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
822  GPIO_InitStruct.Pull = GPIO_NOPULL;
823  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
824  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
825
826  /*Configure GPIO pin : PB1 */
827  GPIO_InitStruct.Pin = GPIO_PIN_1;
828  GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;
829  GPIO_InitStruct.Pull = GPIO_NOPULL;
830  HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
831
832  /*Configure GPIO pin : PD2 */
833  GPIO_InitStruct.Pin = GPIO_PIN_2;
834  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
835  GPIO_InitStruct.Pull = GPIO_NOPULL;
836  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
837  HAL_GPIO_Init(GPIOD, &GPIO_InitStruct);
838
839  /* EXTI interrupt init*/
840  HAL_NVIC_SetPriority(EXTI1_IRQn, 0, 0);
841  HAL_NVIC_EnableIRQ(EXTI1_IRQn);
842
843 }
844
845 /* USER CODE BEGIN 4 */
846 // Callback: timer has rolled over
847
848 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
849 {
850     // Check which version of the timer triggered. Calculate the new
851     // times for for the different gates.
852     if (htim == &htim11)
853     {
854
855
856
857
858
859
860 //         readAngleFromMA730(&angle_HS);
861
862 //         angleRad = (angle_HS * 0.000095873799);
863 //         angle_HS_shift = angle_HS >> 2;
864 //         uint16_t enc_cnt_old = enc_cnt;
865         enc_cnt = __HAL_TIM_GET_COUNTER(&htim3);
866
867
868         angle = (enc_cnt + (14238)) % 2143; // 15000 - 762 = 14238
```

```
869         angle_encoder = ((enc_cnt + (14238)) % 15000);
870         phi_nomod = (float)((enc_cnt + (14238)) % 15000) *
0.000418879;
871
872         indx++;
873         int indx_max = 10;
874         float phi_diff = phi_nomod - phi_old;
875         if (indx == indx_max){
876             if(phi_diff > 3.1415){
877                 speed_rad = (phi_diff - 6.283185)/(indx_max*0.0001);
878             }
879             else if(phi_diff < -3.1415){
880                 speed_rad = (phi_diff + 6.283185)/(indx_max*0.0001);
881             }
882             else {
883                 speed_rad = phi_diff/(indx_max*0.0001);
884             }
885             phi_old = phi_nomod;
886             indx = 0;
887
888             LP_old = speed_LP;
889             speed_LP = FilterSpeed(speed_rad, LP_old);
890             if(phi_diff > 3.1415){
891                 abs_add = abs_add - 6.28318;
892             }
893             else if(phi_diff < -3.1415){
894                 abs_add = abs_add + 6.28318;
895             }
896             angle_abs = phi_nomod + abs_add;
897
898
899         }
900         if (Global_enable == 1){
901             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, 1);
902             t = t + 0.0001;
903
904             // Lookup table local variable - Fixed point
905             sin_lookup = getSine(angle);
906             cos_lookup = getCoSine(angle);
907
908             // Current
909             i1 = 0.001595690196*adc[0] - 3.08973;
910             i2 = 0.001595690196*adc[1] - 3.08973;
911             i3 = 0.001595690196*adc[2] - 3.08973;
912
913
914 //      Park Transform (abc to dq0) - alpha-d alligned
915         i_d = (0.667)*(i1*cos_lookup + i2*(0.866*sin_lookup -
0.5*cos_lookup) + i3*(-0.5*cos_lookup - 0.866*sin_lookup)); // I
think this can be speeded up from multiplying into the parenteces
ourselves and simplifying.
```

```
916         i_q = (0.667)*(- i1*sin_lookup - i2*(-0.866*cos_lookup -  
0.5*sin_lookup) - i3*(0.866*cos_lookup - 0.5*sin_lookup));  
917         // i_d = 0.667*cos_lookup*i1 - 0.3335*cos_lookup*i2 -  
0.3335*cos_lookup*i3 + 0.57762*i2*sin_lookup - 0.57762*i3*sin_lookup  
918         // i_d = cos_lookup * (0.667*i1 - 0.3335*i2 - 0.3335*i3)  
+ sin_lookup * (0.57762*i2 - 0.57762*i3); // potential improvement.  
919  
920  
921         // i_0 = (0.667)*0.5*(i1 + i2 + i3); // Commented out  
since it is not used  
922  
923  
924  
925 //          Controller - speed indx == 0 correspond to indx = 10  
before. This is Validated  
926         if (Speed_loop == 1 && indx == 0){  
927             // The position controller is inside the speed  
controller to sync them.  
928             if (pos_loop == 1){  
929                 if (k == 10){  
930                     float K_pos = 20;  
931                     omega_ref = K_pos*(phi_ref - angle_abs);  
932                     k = 0;  
933                 }  
934                 else{  
935                     k++;  
936                 }  
937             }  
938  
939  
940             if(constant_acc == 1){  
941                 omega_ref = t*omega_dot;  
942  
943                 HAL_GPIO_WritePin(GPIOD, GPIO_PIN_2, 1);  
944             }  
945             if( constant_acc == 1 && t > 2){  
946                 omega_ref = -5;  
947                 HAL_GPIO_WritePin(GPIOD, GPIO_PIN_2, 0);  
948             }  
949             float K_AW_m = 0.001;  
950             e_omega = omega_ref - speed_rad;  
951  
952             C_omega_i = K_i_m * T_m *e_omega + C_omega_i_old +  
K_AW_m*(i_qref_sat - i_qref);  
953             i_qref = K_p_m * e_omega + C_omega_i;  
954             C_omega_i_old = C_omega_i;  
955  
956             i_qref_sat = i_qref;  
957             if (i_qref > 2.85){  
958                 i_qref_sat = 2.85;  
959             }
```

```

960         else if(i_qref < -2.85){
961             i_qref_sat = -2.85;
962         }
963         else{
964             i_qref_sat = i_qref;
965         }
966     }
967
968
969
970     // Controllers - Current (i_dref = 0)
971
972     e_dCurr = (0 - i_d);
973     e_qCurr = (i_qref_sat - i_q);
974
975     //PI controller
976
977
978     float K_AW = 0.1;
979     C_dCurr_i = K_i_e * T_e *e_dCurr + C_dCurr_old + K_AW*
(V_dref_sat - V_dref);
980     V_dref = K_p_e * e_dCurr + C_dCurr_i;// -
speed_LP*7*0.0024*i_q;
981     C_dCurr_old = C_dCurr_i;
982
983     C_qCurr_i = K_i_e * T_e *e_qCurr + C_qCurr_old + K_AW*
(V_qref_sat - V_qref);
984     V_qref = K_p_e * e_qCurr +C_qCurr_i;// + speed_LP*7*
(0.0024*i_q + 0.0102);
985     C_qCurr_old = C_qCurr_i;
986     // P-controller
987 // V_dref = e_dCurr *28;
988 // V_qref = e_qCurr *28;
989
990     // Saturation of the vector magnitude
991     float mag_V = sqrtf(V_dref*V_dref + V_qref*V_qref);
992     if (mag_V > 9.2376){
993         V_dref_sat = V_dref/mag_V*9.2376;
994         V_qref_sat = V_qref/mag_V*9.2376;
995     }
996     else{
997         V_dref_sat = V_dref;
998         V_qref_sat = V_qref;
999     }
1000
1001     // Inverse Park - d-alpha aligned
1002     V_alpha = cos_lookup*V_dref_sat - sin_lookup*V_qref_sat;
1003     V_beta = sin_lookup*V_dref_sat + cos_lookup*V_qref_sat;
1004
1005
1006     if(sample_data == 1){

```

```
1007         w = w+1;
1008     }
1009
1010     // Data sampling
1011
1012     if(sample_data == 1 && w > 0){
1013         // This functions saves the variables to an array each
iteration. Remember to have the array be the right type.
1014         Data_indx = Data_indx + 1;
1015         if(Data_indx == Data_rate && count < ARRAY_SIZE ){
1016             array1[count] = (float) phi_nomod;
1017             array2[count] = (float) angle_abs;
1018             array3[count] = (float) phi_ref;
1019             array4[count] = (float) speed;
1020             array5[count] = (float) speed_LP;
1021             array6[count] = (float) omega_ref;
1022             array7[count] = (float) i_qref_sat;
1023             array8[count] = (float) i_q;
1024             array9[count] = (float) i_qref;
1025             array10[count] = (float) V_qref;
1026             array11[count] = (float) V_qref_sat;
1027 //             array12[count] = (uint16_t) angle_encoder;
1028 //             array13[count] = (uint16_t) angle_HS;
1029 //             array14[count] = (uint16_t) angle_HS_shift;
1030             Data_indx = 0;
1031             count = count + 1; }
1032         }
1033
1034
1035         if(count == ARRAY_SIZE && send_data == 1){
1036             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, 0); //
Disable global enable when the data is saved. This is to stop
current flowing in the motor.
1037
1038             HAL_UART_Transmit(&huart2, (uint8_t *)array1,
sizeof(float)*ARRAY_SIZE, HAL_MAX_DELAY); // This sends the array
over uart. Remember to specify correct datatype in sizeof(). >The
other should always be uint8_t *
1039             HAL_UART_Transmit(&huart2, (uint8_t *)array2,
sizeof(float)*ARRAY_SIZE, HAL_MAX_DELAY);
1040             HAL_UART_Transmit(&huart2, (uint8_t *)array3,
sizeof(float)*ARRAY_SIZE, HAL_MAX_DELAY);
1041             HAL_UART_Transmit(&huart2, (uint8_t *)array4,
sizeof(float)*ARRAY_SIZE, HAL_MAX_DELAY);
1042             HAL_UART_Transmit(&huart2, (uint8_t *)array5,
sizeof(float)*ARRAY_SIZE, HAL_MAX_DELAY);
1043             HAL_UART_Transmit(&huart2, (uint8_t *)array6,
sizeof(float)*ARRAY_SIZE, HAL_MAX_DELAY);
1044             HAL_UART_Transmit(&huart2, (uint8_t *)array7,
sizeof(float)*ARRAY_SIZE, HAL_MAX_DELAY);
1045             HAL_UART_Transmit(&huart2, (uint8_t *)array8,
```

```

    sizeof(float)*ARRAY_SIZE, HAL_MAX_DELAY);
1046     HAL_UART_Transmit(&huart2, (uint8_t *)array9,
    sizeof(float)*ARRAY_SIZE, HAL_MAX_DELAY);
1047     HAL_UART_Transmit(&huart2, (uint8_t *)array10,
    sizeof(float)*ARRAY_SIZE, HAL_MAX_DELAY);
1048     HAL_UART_Transmit(&huart2, (uint8_t *)array11,
    sizeof(float)*ARRAY_SIZE, HAL_MAX_DELAY);
1049
1050 //     HAL_UART_Transmit(&huart2, (uint8_t *)array12,
    sizeof(uint16_t)*ARRAY_SIZE, HAL_MAX_DELAY);
1051 //     HAL_UART_Transmit(&huart2, (uint8_t *)array13,
    sizeof(uint16_t)*ARRAY_SIZE, HAL_MAX_DELAY);
1052 //     HAL_UART_Transmit(&huart2, (uint8_t *)array14,
    sizeof(uint16_t)*ARRAY_SIZE, HAL_MAX_DELAY);
1053
1054     // transmit controller values
1055     float controller[4];
1056     controller[0] = K_p_m;
1057     controller[1] = K_i_m;
1058     controller[2] = K_p_e;
1059     controller[3] = K_i_e;
1060
1061     HAL_UART_Transmit(&huart2, (uint8_t *)controller,
    sizeof(float)*4, HAL_MAX_DELAY);
1062
1063
1064     count = 0;
1065     w = 0;
1066     }
1067
1068
1069     float V_ref2 = 0.866025*V_alpha - 0.5*V_beta;
1070     float V_ref3 = -0.866025*V_alpha - 0.5*V_beta;
1071     N = copysign(1,V_beta) + 2*copysign(1,V_ref2) +
    4*copysign(1,V_ref3);
1072     float C1 = 0.57735027;
1073     float C2 = 1.15470054;
1074     int CP = 48;
1075     switch(N) {
1076     case -1:
1077         Sector = 1;
1078         T_1 = ( V_alpha - C1*V_beta )*V_s_factor;
1079         T_2 = (          C2*V_beta )*V_s_factor;
1080         T_0 = 1 - (T_1 + T_2);
1081         T1_count = T_1*CP ;//- 1;
1082         T2_count = T_2*CP ;//- 1;
1083         T0_count = T_0*CP ;//- 1;
1084         TIM1->CCR1 = 0.5*T0_count;
1085         TIM1->CCR2 = 0.5*T0_count + T1_count;
1086         TIM1->CCR3 = 0.5*T0_count + T1_count + T2_count;
1087         break;

```

```
1088     case -5:
1089         Sector = 2;
1090         T_1 = ( V_alpha + C1*V_beta )*V_s_factor;
1091         T_2 = (-V_alpha + C1*V_beta )*V_s_factor;
1092         T_0 = 1 - (T_1 + T_2);
1093         T1_count = T_1*CP ;//- 1;
1094         T2_count = T_2*CP ;//- 1;
1095         T0_count = T_0*CP ;//- 1;
1096         TIM1->CCR1 = 0.5*T0_count + T2_count;
1097         TIM1->CCR2 = 0.5*T0_count;
1098         TIM1->CCR3 = 0.5*T0_count + T2_count + T1_count;
1099         break;
1100
1101     case 3:
1102         Sector = 3;
1103         T_1 = (          C2*V_beta )*V_s_factor;
1104         T_2 = (-V_alpha - C1*V_beta )*V_s_factor;
1105         T_0 = 1 - (T_1 + T_2);
1106         T1_count = T_1*CP ;//- 1;
1107         T2_count = T_2*CP ;//- 1;
1108         T0_count = T_0*CP ;//- 1;
1109         TIM1->CCR1 = 0.5*T0_count + T1_count + T2_count;
1110         TIM1->CCR2 = 0.5*T0_count;
1111         TIM1->CCR3 = 0.5*T0_count + T1_count;
1112         break;
1113     case 1:
1114         Sector = 4;
1115         T_1 = (-V_alpha + C1*V_beta )*V_s_factor;
1116         T_2 = (          - C2*V_beta )*V_s_factor;
1117         T_0 = 1 - (T_1 + T_2);
1118         T1_count = T_1*CP ;//- 1;
1119         T2_count = T_2*CP ;//- 1;
1120         T0_count = T_0*CP ;//- 1;
1121         TIM1->CCR1 = 0.5*T0_count + T2_count + T1_count;
1122         TIM1->CCR2 = 0.5*T0_count + T2_count;
1123         TIM1->CCR3 = 0.5*T0_count;
1124         break;
1125     case 5:
1126         Sector = 5;
1127         T_1 = (-V_alpha - C1*V_beta )*V_s_factor;
1128         T_2 = ( V_alpha - C1*V_beta )*V_s_factor;
1129         T_0 = 1 - (T_1 + T_2);
1130         T1_count = T_1*CP ;//- 1;
1131         T2_count = T_2*CP ;//- 1;
1132         T0_count = T_0*CP ;//- 1;
1133         TIM1->CCR1 = 0.5*T0_count + T1_count;
1134         TIM1->CCR2 = 0.5*T0_count + T1_count + T2_count;
1135         TIM1->CCR3 = 0.5*T0_count;
1136         break;
1137     case -3:
1138         Sector = 6;
```

```
1139         T_1 = (          - C2*V_beta )*V_s_factor;
1140         T_2 = ( V_alpha + C1*V_beta )*V_s_factor;
1141         T_0 = 1 - (T_1 + T_2);
1142         T1_count = T_1*CP ;//- 1;
1143         T2_count = T_2*CP ;//- 1;
1144         T0_count = T_0*CP ;//- 1;
1145         TIM1->CCR1 = 0.5*T0_count ;
1146         TIM1->CCR2 = 0.5*T0_count + T2_count + T1_count;
1147         TIM1->CCR3 = 0.5*T0_count + T2_count;
1148         break;
1149     }
1150 }
1151 else{
1152     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, 0);
1153 }
1154 //     HAL_GPIO_WritePin(GPIOD, GPIO_PIN_2, 0);
1155
1156
1157
1158 }
1159
1160 }
1161 float getSine(unsigned int angle) {
1162     angle = angle % 2143; //angle &= 15000; // Reduce angle to the
    range of 1 circle
1163     if (angle < 536){
1164         return SinTable[angle];
1165     } else if(angle >= 536 && angle < 1072 ) {
1166         return SinTable[1072 - (angle + 1)];
1167     } else if(angle >= 1072 && angle < 1607){
1168         return -SinTable[angle-1072];
1169     } else if(angle >= 1607 && angle < 2143){
1170         return -SinTable[2143-(angle + 1)];
1171     }
1172 }
1173 float getCoSine(unsigned int angle) {
1174     angle = angle % 2143; //angle &= 15000; // Reduce angle to the
    range of 1 circle
1175     if (angle < 536){
1176         return CosTable[angle];
1177     } else if(angle >= 536 && angle < 1072 ) {
1178         return -CosTable[1072 - (angle + 1)];
1179     } else if(angle >= 1072 && angle < 1607){
1180         return -CosTable[angle-1072];
1181     } else if(angle >= 1607 && angle < 2143){
1182         return CosTable[2143-(angle + 1)];
1183     }
1184 }
1185
1186
1187 float FilterSpeed(float speed, float LP_old) {
```

```
1188     speed_LP = alpha*LP_old + (1-alpha)*speed;
1189 }
1190
1191 void readAngleFromMA730(uint16_t *angle_HS) { // Hall sensor
1192 //     uint8_t rxBuffer[2] = {0}; // Buffer to store the received
        data
1193     HAL_GPIO_WritePin(GPIOC, GPIO_PIN_9, GPIO_PIN_RESET); // CS Low
1194     HAL_SPI_Receive(&hspi3, angle_HS, 2, 100); // Receive 2 bytes
        (16 bits) of data
1195     HAL_GPIO_WritePin(GPIOC, GPIO_PIN_9, GPIO_PIN_SET); // CS High
1196
1197
1198 }
1199
1200
1201
1202 //Terkel
1203 void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)
1204 {
1205     HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_3);
1206 //     HAL_Delay(1);
1207 //     HAL_GPIO_WritePin(GPIOC, GPIO_PIN_3, 0);
1208 }
1209
1210 /* USER CODE END 4 */
1211
1212 /**
1213  * @brief This function is executed in case of error occurrence.
1214  * @retval None
1215  */
1216 void Error_Handler(void)
1217 {
1218     /* USER CODE BEGIN Error_Handler_Debug */
1219     /* User can add his own implementation to report the HAL error
        return state */
1220     __disable_irq();
1221     while (1)
1222     {
1223
1224
1225
1226
1227     }
1228     /* USER CODE END Error_Handler_Debug */
1229 }
1230
1231 #ifndef USE_FULL_ASSERT
1232 /**
1233  * @brief Reports the name of the source file and the source line
        number
1234  *
        where the assert_param error has occurred.
```

```
1235  * @param file: pointer to the source file name
1236  * @param line: assert_param error line source number
1237  * @retval None
1238  */
1239 void assert_failed(uint8_t *file, uint32_t line)
1240 {
1241     /* USER CODE BEGIN 6 */
1242     /* User can add his own implementation to report the file name and
       line number,
1243        ex: printf("Wrong parameters value: file %s on line %d\r\n",
           file, line) */
1244     /* USER CODE END 6 */
1245 }
1246 #endif /* USE_FULL_ASSERT */
1247
1248
```